

# Sicurezza

Alessio Marini, 2122855

Appunti presi durante il corso di **Sicurezza** nell'anno **2025/2026** del professore Emiliano Casalicchio.

Gli appunti li scrivo principalmente per rendere il corso più comprensibile **a me** e anche per imparare il linguaggio Typst. Se li usate per studiare verificate sempre le informazioni 🙏.

## Contatti:

📀 alem1105

✉ marini.2122855@studenti.uniroma1.it

September 27, 2025

# Indice

|                                                                  |    |
|------------------------------------------------------------------|----|
| 1. CIA - Key Objective / Requirements .....                      | 5  |
| 1.1. Levels of Impact .....                                      | 5  |
| 1.2. Computer Security Challenges .....                          | 5  |
| 1.3. Un modello per la Sicurezza .....                           | 6  |
| 2. Classificazione degli Attacchi .....                          | 7  |
| 2.1. Requisiti di Sicurezza .....                                | 7  |
| 2.2. Principi fondamentali di Design per la sicurezza .....      | 8  |
| 2.3. Superfici e Alberi di Attacco .....                         | 8  |
| 2.4. Computer Security Strategies .....                          | 9  |
| 3. Cryptographic Tools .....                                     | 10 |
| 3.1. Symmetric Encryption .....                                  | 10 |
| 3.2. Type of Symmetric Encryption .....                          | 11 |
| 3.3. Message Authentication .....                                | 11 |
| 3.4. Funzioni di Hash .....                                      | 12 |
| 3.5. Crittografia a Chiave Pubblica (Asimmetrica) .....          | 13 |
| 3.6. Digital Signatures .....                                    | 14 |
| 3.7. Digital Envelope .....                                      | 17 |
| 3.8. Random Numbers .....                                        | 18 |
| 4. Cifratura Simmetrica .....                                    | 19 |
| 4.1. Cryptographics Systems .....                                | 19 |
| 4.1.1. Block Cipher .....                                        | 19 |
| 4.1.1.1. DES - Data Encryption Standard .....                    | 21 |
| 4.1.1.2. 3DES .....                                              | 21 |
| 4.1.1.3. AES - Advanced Encryption Standard .....                | 22 |
| 4.1.2. Stream Ciphers .....                                      | 27 |
| 4.1.2.1. RC4 .....                                               | 27 |
| 4.2. Block Cipher Mode Operation .....                           | 29 |
| 4.2.1. Electronic Codebook Mode (ECB) .....                      | 29 |
| 4.2.2. Cipher Block Chaining Mode (CBC) .....                    | 29 |
| 4.2.2.1. Propagazione degli errori .....                         | 31 |
| 4.2.3. Cipher Feedback Mode (CFB) .....                          | 31 |
| 4.2.4. Counter Mode (CTR) .....                                  | 33 |
| 4.3. Cryptanalysis .....                                         | 33 |
| 4.4. Key Distribution .....                                      | 34 |
| 5. Cifratura a Chiave Asimmetrica e Message Authentication ..... | 36 |
| 5.1. Secure Hash Functions .....                                 | 36 |
| 5.1.1. Simple Hash Functions .....                               | 36 |
| 5.1.2. The SHA Secure Hash Function .....                        | 37 |
| 5.2. HMAC .....                                                  | 40 |
| 5.2.1. Secutity of HMAC .....                                    | 42 |
| 5.3. Authenticated Encryption (AE) .....                         | 42 |
| 5.4. RSA Public-Key Encryption .....                             | 45 |
| 5.4.1. Sicurezza dell'RSA .....                                  | 46 |

|          |                                                             |    |
|----------|-------------------------------------------------------------|----|
| 5.4.1.1. | Il problema della fattorizzazione .....                     | 47 |
| 5.4.1.2. | Timing Attacks .....                                        | 48 |
| 5.5.     | Diffie-Hellman .....                                        | 49 |
| 5.5.1.   | Sicurezza del Diffie-Hellman .....                          | 49 |
| 6.       | User Authentication .....                                   | 51 |
| 6.1.     | Authentication with something you know .....                | 51 |
| 6.2.     | Authentication Based on Biometrics: Something you are ..... | 52 |
| 6.3.     | Authentication Based on Tokens: Something you have .....    | 54 |
| 6.4.     | Federated Identity Management (FIM) .....                   | 55 |
| 6.5.     | Authentication Security Issues .....                        | 56 |
| 7.       | Access Control .....                                        | 57 |
| 7.1.     | Access Policies (AP) .....                                  | 57 |
| 7.2.     | Implementing Access Control .....                           | 58 |
| 7.2.1.   | Access Control Directory .....                              | 58 |
| 7.2.2.   | Access Control Matrix .....                                 | 59 |
| 7.2.3.   | Access Control List .....                                   | 60 |
| 7.2.4.   | Privilege List .....                                        | 60 |
| 7.3.     | Capability (non ho capito molto dal libro / slide) .....    | 60 |
| 7.4.     | Domain .....                                                | 61 |
| 7.5.     | Capabilities at Work .....                                  | 62 |
| 7.6.     | Procedure Oriented Access Control .....                     | 62 |
| 7.7.     | Role-Based Access Control .....                             | 62 |
| 7.8.     | Attribute-Based Access Control .....                        | 64 |
| 7.8.1.   | Attributes .....                                            | 64 |
| 8.       | Programs And Programming .....                              | 66 |
| 8.1.     | Buffer Overflow .....                                       | 66 |
| 8.2.     | Incomplete Mediation .....                                  | 67 |
| 8.3.     | Time-of-Check to Time-of-Use - TOCTTOU .....                | 67 |
| 8.4.     | Undocumented Access Point / Backdoor .....                  | 67 |
| 8.5.     | Race Condition .....                                        | 67 |
| 9.       | Malware .....                                               | 68 |
| 9.1.     | Harm .....                                                  | 68 |
| 9.2.     | Trasmissione e Propagazione .....                           | 68 |
| 9.3.     | Attivazione .....                                           | 69 |
| 9.4.     | Stealth .....                                               | 69 |
| 9.5.     | Contromisure .....                                          | 69 |
| 9.6.     | Tecniche di Testing e Validazione .....                     | 69 |
| 10.      | Internet Security .....                                     | 71 |
| 10.1.    | Browser Attacks .....                                       | 71 |
| 10.1.1.  | Man-in-the-Browser .....                                    | 71 |
| 10.1.2.  | Keystroke Logger .....                                      | 71 |
| 10.1.3.  | Page-in-the-Middle .....                                    | 72 |
| 10.1.4.  | Program Download Substitution .....                         | 72 |
| 10.1.5.  | User-in-the-Middle .....                                    | 72 |
| 10.2.    | How Browser Attacks Succeed .....                           | 72 |

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 10.2.1. Successful Identification and Authentication .....        | 72 |
| 10.3. Web Attacks Targeting Users .....                           | 73 |
| 10.3.1. False or Misleading Content .....                         | 73 |
| 10.3.2. Fake Web Site .....                                       | 73 |
| 10.3.3. Fake Code .....                                           | 73 |
| 10.3.3.1. Protecting Web Sites Against Change .....               | 73 |
| 10.3.3.1.1. Integrity Checksums .....                             | 73 |
| 10.3.3.1.2. Signed Code or Data .....                             | 73 |
| 10.3.4. Web Bug .....                                             | 73 |
| 10.4. Furto di Dati: Scripting e SQL Injection .....              | 73 |
| 10.5. Sicurezza delle Applicazioni Mobile (App) .....             | 74 |
| 10.6. Attacchi tramite Email e Messaggi .....                     | 75 |
| 11. OS Security .....                                             | 76 |
| 11.1. Separation and Sharing .....                                | 77 |
| 11.2. Hardware Protection of Memory .....                         | 77 |
| 11.3. Security in the Design of Operating System .....            | 81 |
| 12. Network Security .....                                        | 83 |
| 12.1. Interception .....                                          | 83 |
| 12.2. Network Security Attacks .....                              | 83 |
| 12.2.1. Intercettazione .....                                     | 83 |
| 12.2.2. Data Corruption .....                                     | 84 |
| 12.2.3. Denial of Service .....                                   | 84 |
| 12.3. Port Scanning .....                                         | 84 |
| 12.4. Wireless Security .....                                     | 85 |
| 12.5. Denial of Service .....                                     | 87 |
| 12.5.1. Routing Attacks .....                                     | 89 |
| 12.5.2. Distributed Denial of Service .....                       | 89 |
| 12.6. Contromisure .....                                          | 90 |
| 12.6.1. Network Criptography .....                                | 90 |
| 12.6.2. IPsec .....                                               | 91 |
| 12.7. Firewalls .....                                             | 93 |
| 12.7.1. Tipologie di Firewall .....                               | 94 |
| 12.7.2. Limitazioni dei Firewall .....                            | 96 |
| 12.7.3. Network Address Translation (NAT) .....                   | 96 |
| 12.8. Intrusion Detection and Prevention .....                    | 97 |
| 12.8.1. Posizionamento e Tipologie di Implementazione .....       | 99 |
| 12.9. Intrusion Prevention Systems (IPS) e Tipi di Risposta ..... | 99 |



# 1. CIA - Key Objective / Requirements

## Sicurezza Informatica - NIST

Le misure che garantiscono **confidenzialità, integrità e disponibilità** di un sistema informatico nel suo interesse quindi hardware, software, firmware, le informazioni che memorizza, processa e comunica

Queste tre proprietà vengono indicate nella **triade CIA** e significano:

- **Confidenzialità:** Garantire che le informazioni private non vengano divulgate a persone non autorizzate e garantire anche che ogni utente abbia il controllo su quali dei suoi dati personali vengono memorizzati nel sistema o a chi vengono inviati. L'utente deve quindi essere in grado di gestire la sua **privacy**.
- **Integrità:** Assicurare che i dati o il sistema vengano modificati solo in modo autorizzato.
- **Disponibilità:** Garantire l'accesso affidabile alle informazioni e ai servizi, non devono esserci interruzioni del servizio.

La triade è in costante conflitto con usabilità e costi del sistema, il lavoro di un esperto di sicurezza è quindi quello di trovare il giusto compromesso fra queste variabili.

Nei sistemi moderni, ai tre pilastri della triade, si aggiungono:

- **Autenticità:** La garanzia che un messaggio o un utente provengano effettivamente dalla fonte dichiarata.
- **Accountability:** La capacità di tracciare in modo univoco le azioni di un'entità, è utile per indagini forensi, isolare i guasti e la **nonrepudiation** ovvero quando qualcuno nega di aver compiuto un'azione.

## 1.1. Levels of Impact

Possiamo classificare l'impatto di eventuali violazioni:

- *Basso:* Danni minori o perdite finanziarie limitate, il sistema continua comunque a funzionare anche se magari in modo ridotto.
- *Moderato:* Danni significativi alle capacità operative e perdite finanziarie importanti.
- *Alto:* Il sistema non funziona più, danni finanziari enormi e ci sono rischi catastrofici o letali per le persone.

## 1.2. Computer Security Challenges

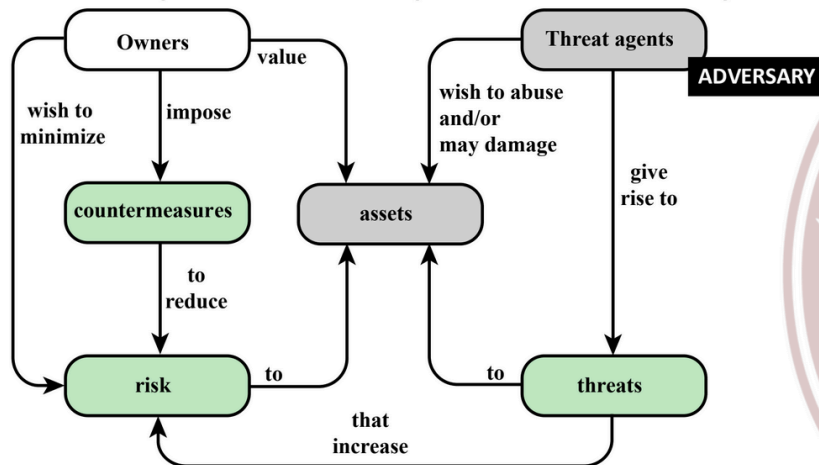
La sicurezza informatica presenta diverse sfide che la rendono complessa:

1. Non è semplice per chi si trova alla prima armi e spesso le procedure di sicurezza risultano controintuitive
2. Richiede una gestione di **informazioni riservate** e anche una decisione su dove posizionare le difese.
3. Dagli utenti viene vista come **un ostacolo** e dai manager come uno spreco di risorse fino a quando non subiscono un attacco.
4. Spesso viene aggiunta alla fine dell'implementazione di un sistema come un «ripensamento» invece di essere implementata dall'inizio.

### Dilemma del Difensore

Chi difende deve eliminare **tutte le debolezze** di un sistema (praticamente impossibile), mentre a chi attacca ne basta una sola.

## 1.3. Un modello per la Sicurezza



Nel diagramma sono indicati:

- **Asset (Risorsa):** È quello che ha un valore nel sistema (Hardware, software, Dati ecc...)
- **Vulnerabilità (di un asset):** Una debolezza nel sistema che può essere sfruttata. Ne esistono di diversi tipi:
  - **Corrupted:** Dati alterati
  - **Leaky:** Dati trapelati
  - **Unavailable:** Sistema non disponibile
- **Threat:** Una qualsiasi «situazione» che potrebbe causare danni di qualsiasi tipo.
- **Attack:** Quando un *Threat Agent* ovvero un avversario sfrutta attivamente una vulnerabilità.
- **Countermeasure:** Tecniche o dispositivi utili a prevenire gli attacchi o a ridurne gli effetti.

Possiamo anche calcolare il rischio grazie alla formula:

$$R = P \times D$$

Dove:

- *P*: Probabilità che l'evento si verifichi
- *D*: Impatto dell'evento

### Nota

L'obiettivo delle contromisure **non è azzerare il rischio** dato che sarebbe troppo costoso o comunque impossibile, ma è quello di portarlo ad un livello accettabile definito **Rischio Residuo**.

## 2. Classificazione degli Attacchi

La prima distinzione che possiamo fare è quella della provenienza dell'attacco, o dall'interno (**insider**) o dall'esterno (**outsider**). Gli attacchi si dividono poi in:

- **Passivi:** L'attaccante osserva e raccoglie informazioni senza alterare il sistema, questi sono molto difficile da rilevare e l'unica difesa è la **prevenzione**. Ne esistono di due tipi:
  - *Release of Message Contents:* Leggere il contenuto di un messaggio
  - *Traffic Analysis:* Anche se il messaggio è cifrato, l'attaccante osserva chi sta parlando con chi e guardando anche la dimensione del messaggio, la frequenza e altri dati può ricavare qualche informazione.
- **Attivi:** L'attaccante altera i dati o il funzionamento del sistema:
  - *Replay:* Catturare un dato e reinviarlo successivamente per ottenere accesso.
  - *Masquerade:* Fingersi qualcun altro
  - *Modification of Messages:* Alterare parti legittime di un messaggio o ritardarlo.
  - *Denial of Service (DoS):* Interrompere il servizio di un sistema sovraccaricandolo di richieste.

Vediamo adesso le conseguenze e i tipi di attacco che le producono (secondo lo standard RFC4949):

- **Unauthorized Disclosure:** Vengono divulgati dati da persone non autorizzate, attacchi possibili:
  - *Exposure:* Esposizione diretta
  - *Interception*
  - *Inference:* Deduzione di dati da altri segnali
  - *Intrusion*
- **Deception:** Vengono fornite informazioni ad un utente non autorizzato
  - *Masquerade:* Fingersi un'altra persona
  - *Falsification:* Alterare i dati per ingannare
  - *Repudiation:* Negare di aver compiuto un'azione
- **Disruption:** Interruzione delle funzionalità del sistema:
  - *Incapacitation:* Disabilitare un componente
  - *Corruption:* Alterare i dati che permettono il funzionamento del sistema
  - *Obstruction:* Ostacolare i servizi
- **Usurpation:** Viene preso il controllo del sistema da parte di utenti non autorizzati.
  - *Misappropriation:* Prendere il controllo logico o fisico di una risorsa
  - *Misuse:* Utilizzare un componente per svolgere operazioni dannose.

### 2.1. Requisiti di Sicurezza

Le contromisure alle vulnerabilità e minacce possono essere classificate e categorizzate in differenti modi, possiamo classificarle in funzione dei requisiti funzionali, il FIPS 200 le divide in:

- **Requisiti di Sicurezza Tecnici:** Ad esempio controllo degli accessi, autenticazione e integrità del sistema.
- **Funzionali:** Formazione del personale, valutazione dei rischi, sicurezza fisica
- **Misti:** Gestione della configurazione, risposta agli incidenti e protezione dei media.

## 2.2. Principi fondamentali di Design per la sicurezza

Questi principi sono fondamentali per chi progetta architetture di rete o sviluppa software:

- **Economy of Mechanism:** I sistemi di sicurezza devono essere piccoli e semplici, più codice scrivi e più bug introduci.
- **Fail-safe default:** L'accesso deve basarsi sui permessi e non sulle esclusioni, se qualcosa va storto il sistema deve bloccarsi in uno stato sicuro negando l'accesso.
- **Complete Mediation:** Ogni singolo accesso a una risorsa deve essere verificato senza fidarsi ciecamente della cache.
- **Open Design:** La sicurezza non deve basarsi sulla segretezza del codice ma sulla forza dell'algoritmo o delle chiavi.
- **Separation of Privilege:** Richiedere più condizioni per sbloccare un'azione (2FA).
- **Least Privilege:** Utenti e processi devono avere solo i permessi strettamente necessari per fare il loro lavoro.
- **Layering:** Usare ostacoli multipli e sovrapposti, se l'attaccante buca il firewall deve trovare altri blocchi.

### Esempio Fail-Safe

Prendiamo questo esempio in pseudocodice:

```
1  DWORD dwRet = IsAccessAllowed( ... );
2  if (dwRet == ERROR_ACCESS_DENIED) {
3      // Security check failed
4  } else {
5      // Security check OK
6  }
```

Questo codice non va bene perché se per qualsiasi motivo la funzione `IsAccessAllowed()` fallisce e ritorna un dato non previsto noi stiamo garantendo l'accesso al sistema.

## 2.3. Superfici e Alberi di Attacco

Per difendere un sistema è utile capire come possiamo attaccarlo.

Definiamo come **Attack Surface** l'insieme di tutte le vulnerabilità raggiungibili in un sistema, si divide in:

- *Rete*: Porte aperte e server esposti
- *Software*: Bug nel codice
- *Umana*: Social engineering o errore del personale

Il nostro obiettivo è quello di ridurre il più possibile questa superficie.

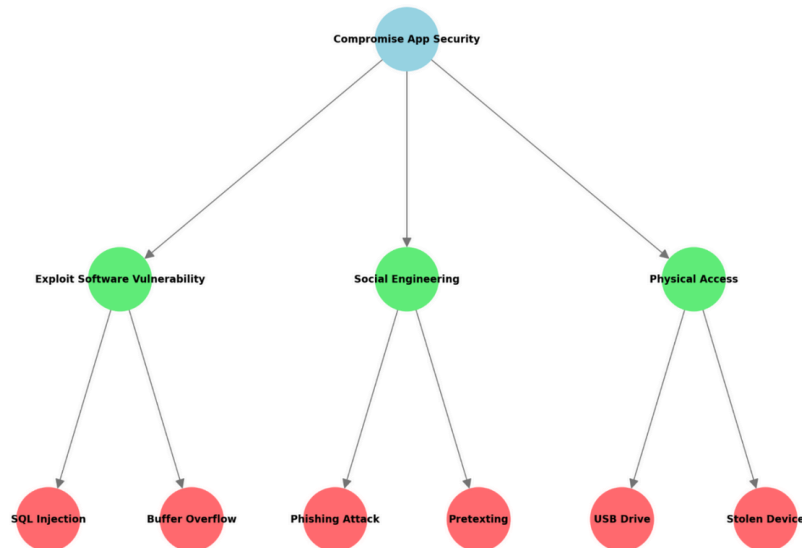
Per analizzare le debolezze del sistema possiamo costruire un **albero di attacco** ovvero un albero che rappresenta le minacce al nostro sistema:

- Il nodo radice è l'obiettivo dell'attaccante
- I nodi intermedi sono i sotto-obiettivi
- Le foglie sono gli attacchi che può fare per colpire un sotto-obiettivo.

I sotto-obiettivi possono essere in AND o OR fra di loro.

Questi alberi sono utilizzati dai team di sicurezza per capire qual è il percorso «più economico» o «più facile» per un attaccante, questo permette quindi di posizionare le difese (layering) dove sono più necessarie.

*Esempio di albero:*



## 2.4. Computer Security Strategies

Come si gestisce quindi la sicurezza a livello aziendale?

- **Specification / Policy (Cosa Fare?):** Le regole che indicano cosa deve fare il sistema (es. lunghezza minima di una password). Queste devono bilanciare il costo della sicurezza col costo di un eventuale attacco.
- **Implementation / Mechanisms (Come Farlo?):** Le tecnologie usate per prevenire, rilevare, rispondere e recuperare i dati.
- **Assurance ed Evaluation (Funziona?):** Il livello di confidenza che i meccanismi implementati rispettino effettivamente le policy. Questo si ottiene tramite test, certificazioni e analisi.

## 3. Cryptographic Tools

### 3.1. Symmetric Encryption

È la tecnica di crittografia universale per garantire la **confidenzialità** dei dati. Il mittente e il destinatario, per scambiarsi messaggi, devono condividere la stessa chiave segreta  $K$ . Questa tecnica, per essere sicura, richiede:

- Un algoritmo crittografico forte.
- Che la condivisione della chiave sia avvenuta in modo sicuro.

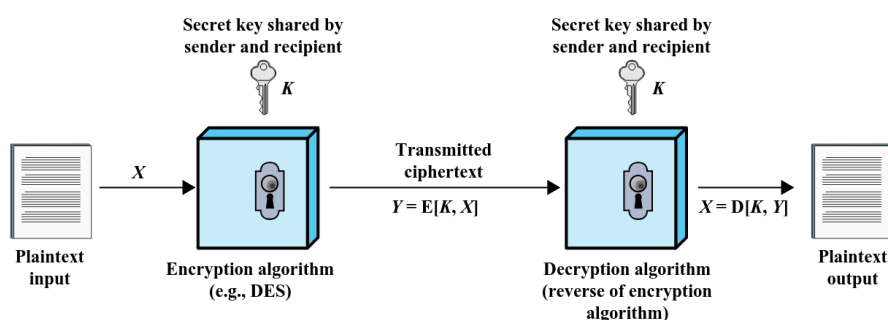


Figure 2.1 Simplified Model of Symmetric Encryption

I principali tipi di attacco a queste tecniche sono:

- **Crittoanalisi:** Sfruttare la natura matematica dell'algoritmo o se si conoscono alcune caratteristiche del testo non cifrato per dedurre la chiave.
- **Brute-Force:** Si provano tutte le chiavi possibili finché non si ottiene un testo sensato. In media è necessario provare almeno la metà di tutte le chiavi possibili.

#### La distribuzione delle chiavi

Il vero problema di questa tecnica è la distribuzione della chiave, infatti, come fanno due persone a scambiarsi la chiave in modo sicuro se il canale su cui parlano non è protetto? Questo problema verrà risolto dalla **crittografia asimmetrica**.

Tra i principali algoritmi per la crittografia simmetrica troviamo:

- **DES (Data Encryption Standard):** Utilizza blocchi da 64-bit e una chiave da 56-bit. Anche se è l'algoritmo più studiato la sua chiave da 56-bit lo rende molto debole contro i moderni processori di oggi.
- **3DES (Triple DES):** Ripete il DES tre volte usando 2 o 3 chiavi diverse portando quindi la lunghezza effettiva della chiave a 112 o 168 bit. I problemi principali sono che è molto lento e usa ancora blocchi piccoli da 64-bit.
- **AES (Advanced Encryption Standard):** Creato per sostituire il 3DES. È molto più efficiente, utilizza blocchi da 128-bit e chiavi da 128, 192 o 256-bit.

L'AES è più efficiente perché utilizza un'architettura chiamata **Rijndael** che applica le sue trasformazioni all'intero blocco di 128-bit simultaneamente organizzandolo in una matrice, questa struttura è ottimizzata per eseguire calcoli paralleli ed è perfetta per come lavorano i

processori moderni. Questo rende l'algoritmo più veloce rispetto ad un DES o 3DES che invece utilizzano la **Rete di Feistel**, questa divide il blocco a metà e, a ogni passaggio, cripta solo una metà alla volta incrociandola con l'altra.

*Tempi medi richiesti per rompere un algoritmo:*

| Key size (bits) | Cipher     | Number of Alternative Keys           | Time Required at $10^9$ decryptions/s     | Time Required at $10^{13}$ decryptions/s |
|-----------------|------------|--------------------------------------|-------------------------------------------|------------------------------------------|
| 56              | DES        | $2^{56} \approx 7.2 \times 10^{16}$  | $2^{55}$ ns = 1.125 years                 | 1 hour                                   |
| 128             | AES        | $2^{128} \approx 3.4 \times 10^{38}$ | $2^{127}$ ns = $5.3 \times 10^{21}$ years | $5.3 \times 10^{17}$ years               |
| 168             | Triple DES | $2^{168} \approx 3.7 \times 10^{50}$ | $2^{167}$ ns = $5.8 \times 10^{33}$ years | $5.8 \times 10^{29}$ years               |
| 192             | AES        | $2^{192} \approx 6.3 \times 10^{57}$ | $2^{191}$ ns = $9.8 \times 10^{40}$ years | $9.8 \times 10^{36}$ years               |
| 256             | AES        | $2^{256} \approx 1.2 \times 10^{77}$ | $2^{255}$ ns = $1.8 \times 10^{60}$ years | $1.8 \times 10^{56}$ years               |

Oggi però i **computer quantistici** minacciano gravemente gli algoritmi con chiave pubblica, infatti questi si basano su problemi matematici come la fattorizzazione che un QC può risolvere facilmente. Oggi si cercano quindi algoritmi **Quantum - resistant**.

### 3.2. Type of Symmetric Encryption

Ovviamente dobbiamo cifrare file più grandi di un qualsiasi blocco, per questo esistono diverse tipologie di cifratura:

- **Block Cipher:** Cifra il testo elaborando un blocco alla volta, può riutilizzare le chiavi ed è il più comune.
- **Stream Cipher:** Cifra il flusso di dati continuamente, byte alla volta, generando un flusso pseudocasuale, è molto più veloce e usa meno codice.

Un metodo di cifratura che utilizza il cifrario a blocchi è il **ECB**:

- **Electronic Codebook (ECB):** È la modalità base per usare i cifrari a blocchi su messaggi lunghi. Ogni blocco viene cifrato singolarmente con la stessa chiave, il problema è che se il testo in chiaro ha delle regolarità allora queste appariranno anche nel testo cifrato.

### 3.3. Message Authentication

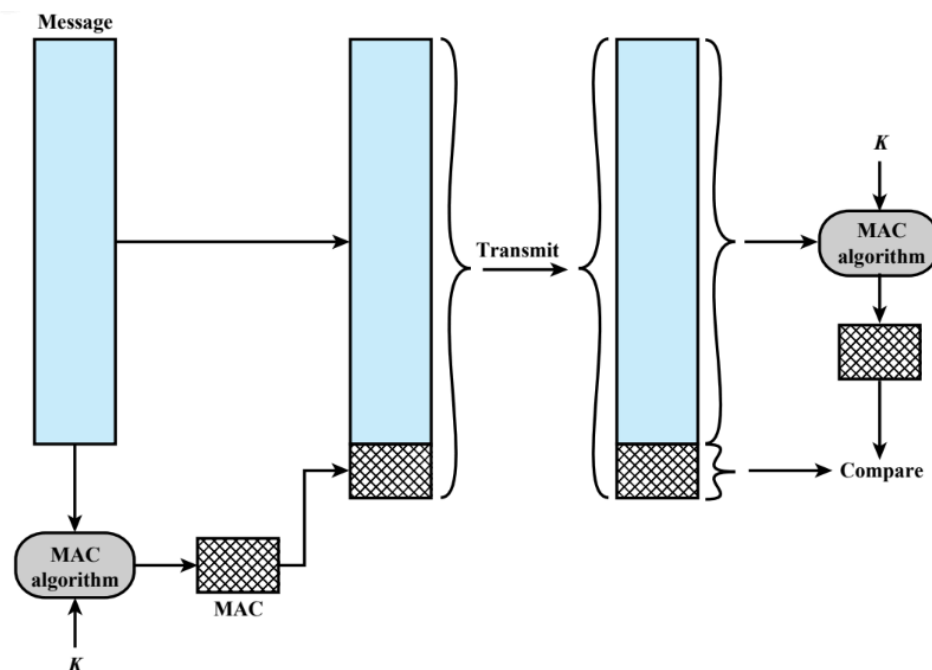
La cifratura dei messaggi protegge dalla lettura non autorizzata ma non dalla manomissione del messaggio ovvero gli attacchi attivi. Con l'autenticazione dei messaggi garantiamo:

- Il contenuto non è stato alterato
- Il messaggio proviene da una fonte legittima

Per garantire che il messaggio non sia stato alterato si utilizza un **MAC (Message Authentication Code)**. Attraverso la chiave  $K$  e il messaggio  $M$  si calcola un «tag»:

$$\text{MAC} = F(K, M)$$

Se il ricevente, ricalcolando il MAC con la stessa chiave segreta, ottiene lo stesso risultato, ha la garanzia che il messaggio non sia stato modificato.



Ovviamente, anche qui, la chiave deve essere in possesso solo delle persone fidate.

Possiamo inoltre distinguere due casi:

- **Without Confidentiality:** Il messaggio è autenticato ma non cifrato, viene soltanto aggiunto il TAG al messaggio.
- **With Confidentiality:** Il messaggio è autenticato e cifrato, quindi anche il tag viene cifrato.

Quando è utile avere autenticazione ma non confidenzialità?

- Sistemi broadcast o molto carichi
- Protocolli di rete, l'header del pacchetto IP non deve essere alterato ma non abbiamo bisogno di nascondere dato che i router devono leggerlo.

### 3.4. Funzioni di Hash

Una funzione Hash  $H(M)$  è un'alternativa al MAC che non richiede una chiave in input, prende un messaggio di dimensione variabile e restituisce un'impronta di dimensione fissa (*digest*). Esistono 6 proprietà da ricordare per le hash function. Data una funzione  $H$ :

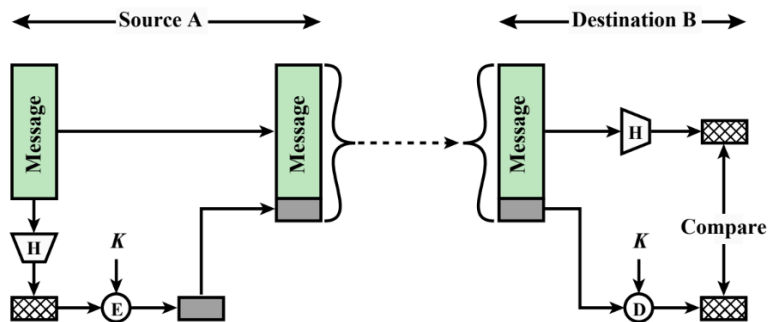
1.  $H$  può essere applicata a blocchi di qualsiasi dimensione.
2.  $H$  produce un output di lunghezza fissa.
3.  $H(x)$  è relativamente semplice da calcolare per ogni  $x$  in input.
4. Per ogni codice hash  $h$  dato, deve essere difficile calcolare un  $x$  tale che  $H(x) = h$ . Una funzione con questa proprietà si dice **one-way** o **pre-image resistant**.
5. Per ogni blocco  $x$  dato, deve essere difficile trovare un  $y$  con  $y \neq x$  tale che  $H(y) = H(x)$ . Una funzione con questa proprietà si dice **second preimage resistant** o **weak collision resistant**.
6. Deve essere difficile trovare una coppia  $(x, y)$  tale che  $H(x) = H(y)$ . Una funzione che soddisfa questa proprietà si dice **collision resistant** o **strong collision resistant**.

Le proprietà 1,2 e 3 sono necessarie soltanto per le hash function utilizzare per message authentication mentre 4, 5 e 6 sono generali ma comunque vanno rispettate se vogliamo avere una buona funzione di hash.

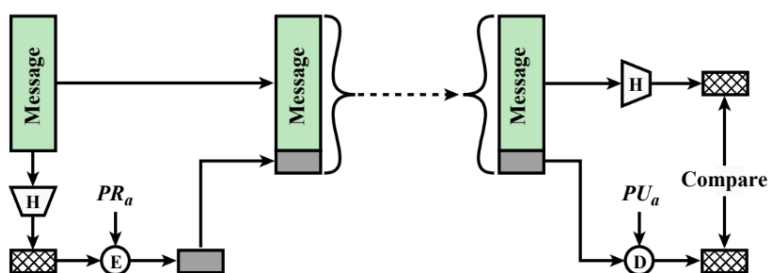


Possiamo utilizzare l'hash a scopo di autenticazione in diversi modi:

- **One-Way Hash function + Symmetric Encryption**

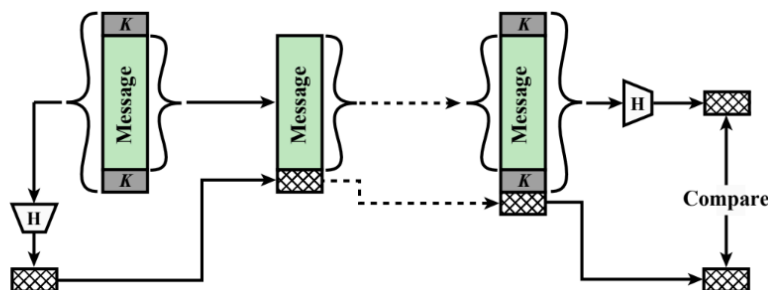


- **One-Way Hash function + Public Key Encryption**



Qui il vantaggio è che non è richiesta la distribuzione delle chiavi in modo sicuro.

- **One-Way Hash function + Secret Value (no encryption)**



Non abbiamo bisogno di crittografia, risparmiando quindi computazione. Chi invia il messaggio deve calcolare  $MD_M = H(K \parallel M \parallel K)$  e inviare  $MD_M \parallel M$ , a questo punto il destinatario, che conosce  $K$ , deve calcolare  $H(K \parallel M \parallel K)$  e verificare  $MD_M$ .

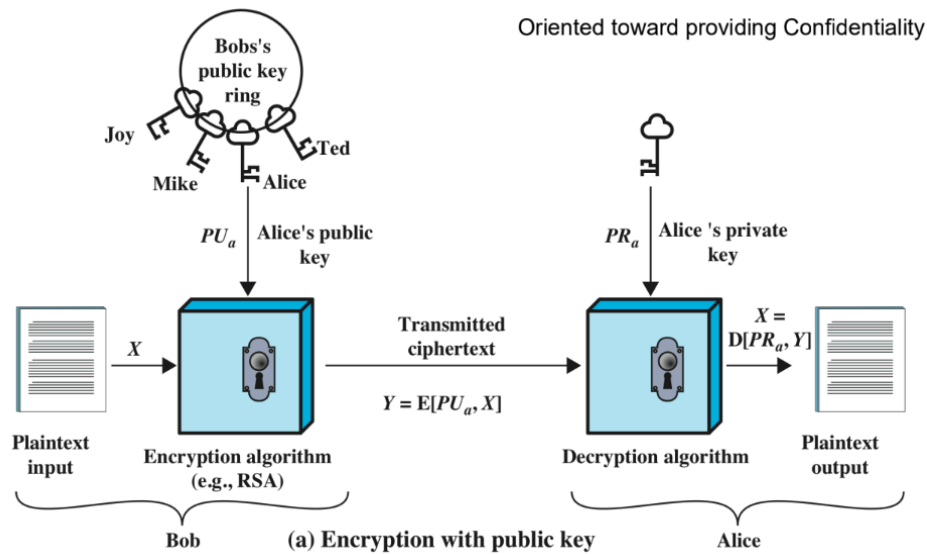
Lo standard più utilizzato è lo **SHA** anche se oggi è considerato debole e si utilizza **SHA-2** o **SHA-3**.

### 3.5. Crittografia a Chiave Pubblica (Asimmetrica)

È un algoritmo di crittografia proposto da **Diffie e Hellman** nel 1976, risolve principalmente il problema della distribuzione della chiave. Viene utilizzata una coppia di chiavi collegate fra loro matematicamente, una **pubblica** e una **privata**. Queste garantiscono:

- **Confidenzialità:** Io cifro con la chiave pubblica di Bob e solo Bob con la sua chiave privata sarà in grado di decifrare il messaggio.

- **Autenticazione:** Cifro l'hash con la mia chiave privata, adesso tutti possono decifrarlo con la mia chiave pubblica garantendo quindi che sono stato io a generare quel messaggio.



Tra i principali algoritmi a chiave asimmetrica troviamo:

- **RSA:** Utilizzato per la cifratura, firme digitali e scambio di chiavi.
- **Diffie-Hellman:** Serve *solo* per permettere a due entità di generare una chiave simmetrica condivisa attraverso un canale sicuro.
- **DSS (Digital Signature Standard):** Garantisce firma digitale con SHA-1
- **ECC (Elliptic Curve):** Sicuro quanto RSA ma usa chiavi più corte, è quindi più veloce e leggero.

| Algorithm      | Digital Signature | Symmetric Key Distribution | Encryption of Secret Keys |
|----------------|-------------------|----------------------------|---------------------------|
| RSA            | Yes               | Yes                        | Yes                       |
| Diffie-Hellman | No                | Yes                        | No                        |
| DSS            | Yes               | No                         | No                        |
| Elliptic Curve | Yes               | Yes                        | Yes                       |

### Vero vantaggio della chiave asimmetrica

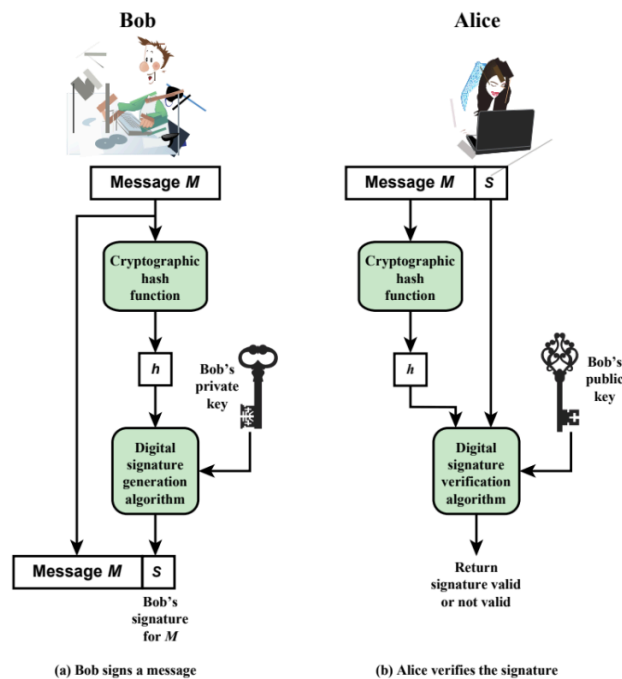
La crittografia asimmetrica è **computazionalmente lentissima** rispetto a quella simmetrica, nel mondo reale infatti viene utilizzata ma soltanto all'inizio delle comunicazioni per scambiare in modo sicuro una chiave segreta temporanea simmetrica, dopo questo scambio il resto della comunicazione avviene con cifratura simmetrica tramite AES.

*Si vede più avanti*

## 3.6. Digital Signatures

Per firma digitale intendiamo gli algoritmi che ci permettono di verificare l'autenticità del mittente di un messaggio, la sua integrità e la non repudiabilità.

L'idea è quella di creare un pacchetto di dati che accompagna il messaggio. Per fare questo vengono utilizzati algoritmi di cifratura e i 3 più importanti sono *DSA*, *RSA* e *Algoritmi a curva ellittica ECDSA*.



Il mittente deve:

- Calcolare l'hash del messaggio.
- Cifriamo l'hash appena ottenuto.
- Inviare il messaggio in chiaro insieme all'hash cifrato.

Il destinatario, per leggere il messaggio deve:

- Effettuare l'hash del messaggio in chiaro.
- Decifrare tramite la chiave pubblica del mittente l'hash ricevuto.
- Verificare che i due hash combacino.

Il problema principale in questo tipo di algoritmi è che qualcuno potrebbe essersi finto Bob quando ha pubblicato la sua chiave pubblica. Come risolviamo questo problema? utilizziamo un **certificato a chiave pubblica**.

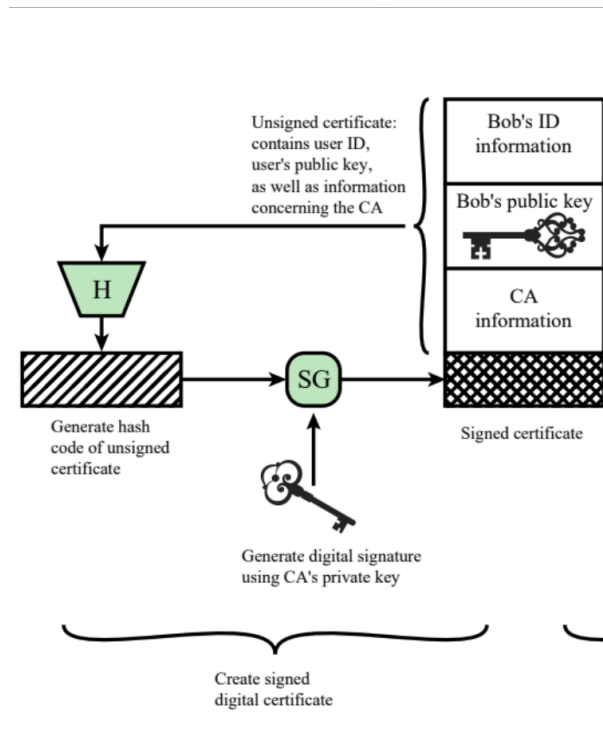
I certificati a chiave pubblica sono composti da:

- La chiave pubblica da distribuire.
- Informazioni sul possessore di questa chiave (ID)
- Informazioni sul verificatore dell'identità che chiamiamo **Certification Authority (CA)** (Aruba, Infocert, PosteItaliane ...), ovvero un'entità fidata che si occupa di verificare l'appartenenza delle chiavi.
- Periodo di validità di questo certificato.

Ma come si crea un certificato?

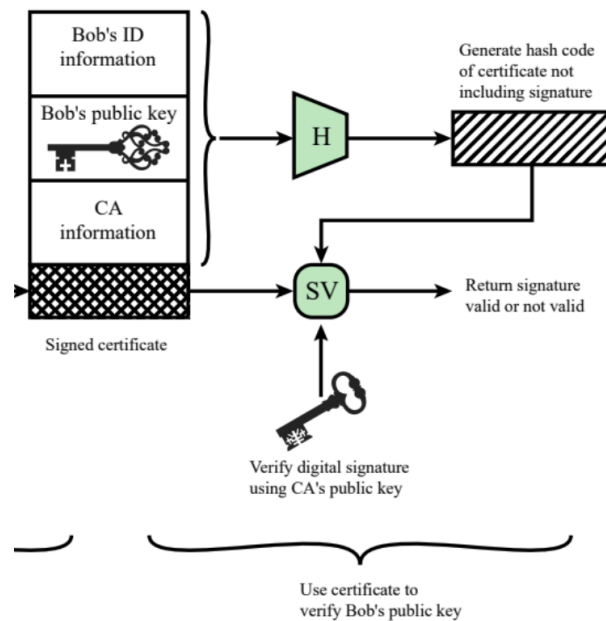
1. L'utente crea la coppia di chiavi, una pubblica e una privata.
2. Prepara un certificato non verificato che include il suo ID e la sua chiave pubblica.
3. Fornisce questo certificato ad una CA in **modo sicuro**.

4. La CA crea il certificato seguendo questi passaggi:
  - Usa una funzione hash su tutto il certificato.
  - Genera una firma digitale utilizzando la propria chiave privata.
5. La CA allega la firma digitale al certificato non verificato.
6. Invia il certificato verificato all'utente



Adesso il certificato potrà essere pubblicato dall'utente e chiunque può verificarlo utilizzando la chiave pubblica della CA, nello specifico:

1. L'utente calcola l'hash del certificato (senza la firma).
2. Tramite la chiave pubblica della CA decifra la firma e verifica che combacia con l'hash ottenuto.



Lo standard per la formattazione dei certificati è X.509

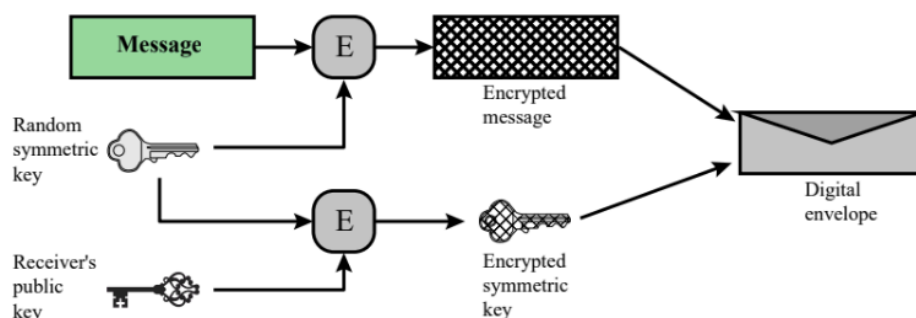
### 3.7. Digital Envelope

Utilizzare algoritmi a chiave asimmetrica è molto pesante dal punto di vista delle prestazioni. Per alleggerire il sistema ma mantenere comunque una trasmissione sicura viene utilizzata una chiave asimmetrica per scambiare una chiave simmetrica che verrà poi utilizzata per il resto della conversazione.

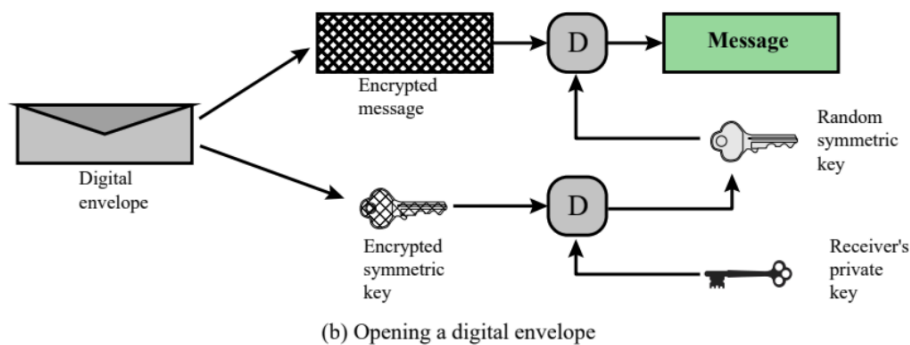
Questo metodo di cifratura non viene utilizzato solo per questo ma in generale per scambiare messaggi cifrati con chiave simmetrica.

I passaggi da seguire sono:

1. Si prepara il messaggio da inviare.
2. Generiamo una chiave simmetrica casuale che verrà usata soltanto in questa comunicazione.
3. Si cifra il messaggio utilizzando la chiave simmetrica appena generata.
4. Si cifra la chiave simmetrica utilizzando la chiave pubblica del destinatario.
5. Si allega la chiave cifrata al messaggio cifrato e si invia.



Quindi il destinatario per leggere il messaggio dovrà semplicemente decifrare prima la chiave simmetrica e poi con questa il messaggio. Ovviamente per garantire autenticità si possono utilizzare i certificati visti prima.



### 3.8. Random Numbers

Vengono usati moltissimo nel mondo della crittografia in algoritmi a chiave pubblica o cifratura a flusso ma anche per generare token per sessioni o altro ...

Quali caratteristiche devono soddisfare?

- **Randomness**

- **Uniform Distribution:** La frequenza con cui un determinato numero viene generato deve essere *circa* la stessa per ogni numero.
- **Independence:** Non deve essere possibile prevedere la sequenza di numeri che vengono generati. (È difficile da provare)

- **Unpredictability**

- Ogni numero è statisticamente indipendente dagli altri numeri della sequenza. Un attaccante non deve essere in grado di predire i numeri futuri basandosi su quelli usciti.

Generare numeri completamente casuali in informatica è molto difficile e per questo si generano, invece, dei numeri detti **pseudocasuali** ovvero delle sequenze di numeri che soddisfano dei **test di randomicità**. Per generare questi numeri ci si basa su delle fonti non deterministiche o in generale eventi fisici, ad esempio gli **Hardware RNG** ovvero delle statistiche della macchina come la temperatura, il jitter dei circuiti o i quantum effects.

## 4. Cifratura Simmetrica

È conosciuta come il metodo convenzionale di cifratura, ha **5 componenti** principali:

- **Plaintext**: Il testo in chiaro da dare all'algoritmo.
- **Encryption Algorithm**: Algoritmo che effettua *sostituzioni e trasformazioni* al plaintext.
- **Secret Key**: Anche questa viene data in input all'algoritmo e da questa dipendono tutte le trasformazioni effettuate nell'algoritmo.
- **Ciphertext**: Il testo di output dell'algoritmo, per ogni testo se utilizziamo chiavi diverse dobbiamo ottenere risultati diversi.
- **Decryption Algorithm**: L'algoritmo di cifratura inverso, dato il ciphertext e la chiave deve fornire in output il plaintext.

### 4.1. Cryptographics Systems

Sono meccanismi che utilizzano algoritmi, chiavi e protocolli per proteggere la **confidenzialità, l'integrità, l'autenticazione** e la **non ripudiabilità**.

Vengono classificati su 3 *dimensioni*

- Tipo di operazioni utilizzate per cifrare il plaintext
  - **Sostituzione**: Prende un insieme di byte e li rimpiazza con altri byte.
  - **Trasposizione**: I byte del testo vengono mischiati fra di loro.

Queste sono indipendenti dalla chiave e vengono ripetute un certo numero di volte (*round*) sul testo. Nell'effettuare queste operazioni non dobbiamo **perdere informazione**, questo significa che le operazioni devono essere invertibili.

- Tipo di chiave utilizzata:
  - **Simmetrica** o **asimmetrica**
- Modo in cui viene trattato il testo in chiaro:
  - **Block**: Viene processato un blocco alla volta. Quindi da un blocco in input otteniamo un blocco in output.
  - **Stream**: L'input viene continuamente processato, ogni elemento produce un elemento in output.

#### 4.1.1. Block Cipher

I meccanismi di cifratura simmetrici a blocchi consistono in una **serie di round** composti da **sostituzioni e permutazioni** basati su una chiave.

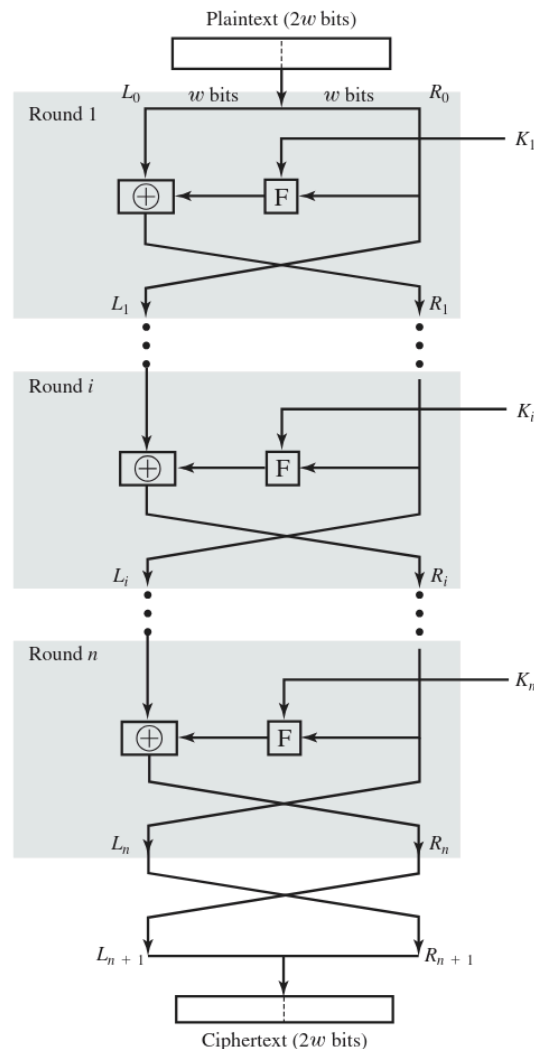
Una delle strutture più famose è la **Feistel Cipher Structure** utilizzata anche nell'algoritmo DES.

In questa struttura dobbiamo tenere a mente che:

- È divisa in round e ogni round effettua le stesse operazioni.
- Si parte con una chiave ma poi a partire da questa vengono generate più chiavi, nello specifico una diversa per ogni round.

A ogni round il testo viene diviso in due parti uguali, la funzione  $F$  prende in input la parte destra ed effettua delle sostituzioni, l'output della funzione viene poi messo in **XOR** con la

parte sinistra del testo. A questo punto viene effettuata una sostituzione tra le due parti ovvero mettiamo quella sinistra a destra e quella di destra a sinistra, in questo modo il round successivo effettua le stesse operazioni perché si troverà a destra la parte non cifrata e a sinistra quella cifrata.



Per progettare una struttura di cifrario a blocchi dobbiamo considerare diversi parametri:

- **Block Size:** Se il blocco è grande ho più sicurezza ma meno velocità nelle operazioni, un tradeoff ottimale è 128-bit.
- **Key Size:** Più è grande e più è alta la sicurezza ma riduciamo la velocità delle operazioni. Un tradeoff ottimale è 128 - 256-bits.
- **Numero di round:** Aumenta la sicurezza ad aumentare il numero di round. Anche qui dobbiamo trovare un giusto tradeoff che è 16 round.
- **Algoritmo di generazione delle sottochiavi:** Più è complesso, più è difficile fare **cryptanalysis**
- **Funzione** utilizzata nei **round:** Stesso ragionamento dell'algoritmo di generazione delle sottochiavi.
- **Velocità dell'algoritmo** di cifratura e decifratura.
- L'algoritmo deve essere **facile da analizzare:** Serve ad individuare meglio le vulnerabilità però le componenti dell'algoritmo devono essere tali che anche se conosciute da chi fa cryptanalysis sia impossibile invertire le operazioni senza la chiave.



## Cryptanalysis

Processo utilizzato per cercare di scoprire il testo in chiaro o la chiave, sfruttando soltanto la conoscenza dell'algoritmo di cifratura, conoscendo quest'ultimo ed i suoi punti deboli si cerca di risalire al testo in chiaro.

### 4.1.1.1. DES - Data Encryption Standard

Il DES era un algoritmo complesso da analizzare.

Il **processo di cifratura** era composto da:

- Block cipher con blocchi da 64-bits e chiave da 56-bits.
- Utilizzava una variante della struttura Feistel a 16 rounds.
- Dalla chiave a 56 bit venivano generate 16 sottochiavi

Il **processo di decifratura** era semplicemente l'inverso della cifratura, questo ha in input il ciphertext e poi utilizza in ordine inverso rispetto alla cifratura le sottochiavi generate dalla chiave.

Per i moderni processori di oggi la chiave a 56-bits è considerata obsoleta e si è passati al 3DES.

### 4.1.1.2. 3DES

- Si estende la chiave fino a 168-bits che veniva suddivisa in 3 chiavi da 56-bits
- Si cifra con le tre chiavi in modo sequenziale, ovvero applicare il DES 3 volte.
- Per decifrare si fa l'opposto.

Nello specifico definiamo come:

- $E$  funzione di cifratura
- $D$  funzione di decifratura

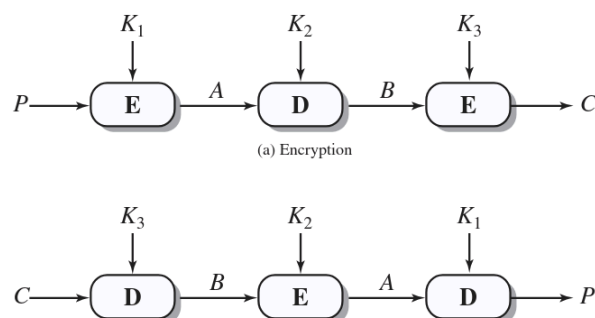
Per effettuare una cifratura con 3DES si applica:

- $C = E(K_3, D(K_2, E(K_1, P)))$

Mentre per la decifratura:

- $P = D(K_1, E(K_2, D(K_3, C)))$

*Graficamente:*



Utilizzare la funzione di decifratura come secondo step in 3DES non offre vantaggi in termini di crittografia ma permette la retrocompatibilità con il DES classico, o meglio permette a chi usa 3DES di decriptare anche messaggi cifrati con DES.

Per migliore sicurezza si è passati poi all'AES.

#### 4.1.1.3. AES - Advanced Encryption Standard

AES utilizza blocchi da 128-bits e chiavi da 128, 192 o 256-bits, negli esempi più avanti la assumiamo da 128. È strutturato in questo modo:

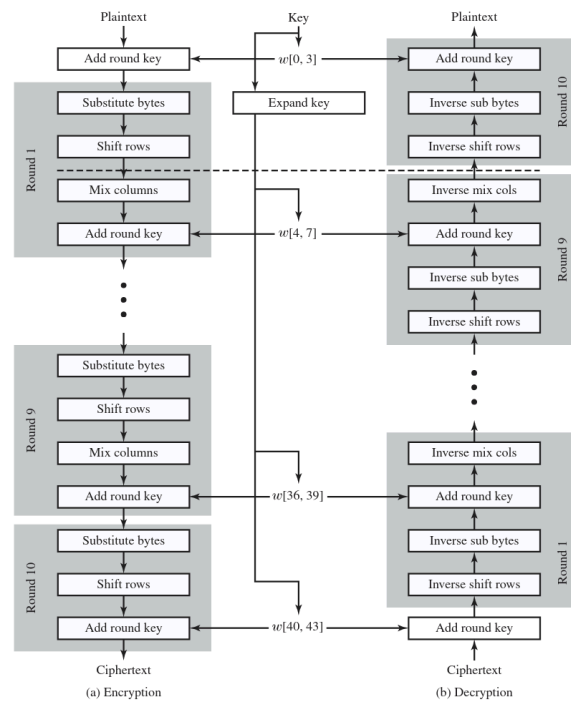
Vuole come input un blocco da 128-bit, questo blocco viene rappresentato come una matrice di byte. Durante l'esecuzione dell'algoritmo, questo blocco viene copiato nello **State array** che viene modificato ad ogni stage di cifratura o decifratura. Dopo l'ultimo stage l'array viene copiato in una matrice di output. Anche la chiave viene rappresentata in una matrice di bytes, questa chiave viene estesa in un array di words, ogni words è composta da 4 bytes e l'array in totale contiene 44 words, ovviamente aumentando i bit della chiave aumentiamo anche le words nell'array.

L'ordine usato nelle matrici è per colonne, quindi ad esempio i primi 4 bytes del plaintext occuperanno la prima colonna della matrice:

|    |    |     |     |
|----|----|-----|-----|
| B1 | B5 | B9  | B13 |
| B2 | B6 | B10 | B14 |
| B3 | B7 | B11 | B15 |
| B4 | B8 | B12 | B16 |

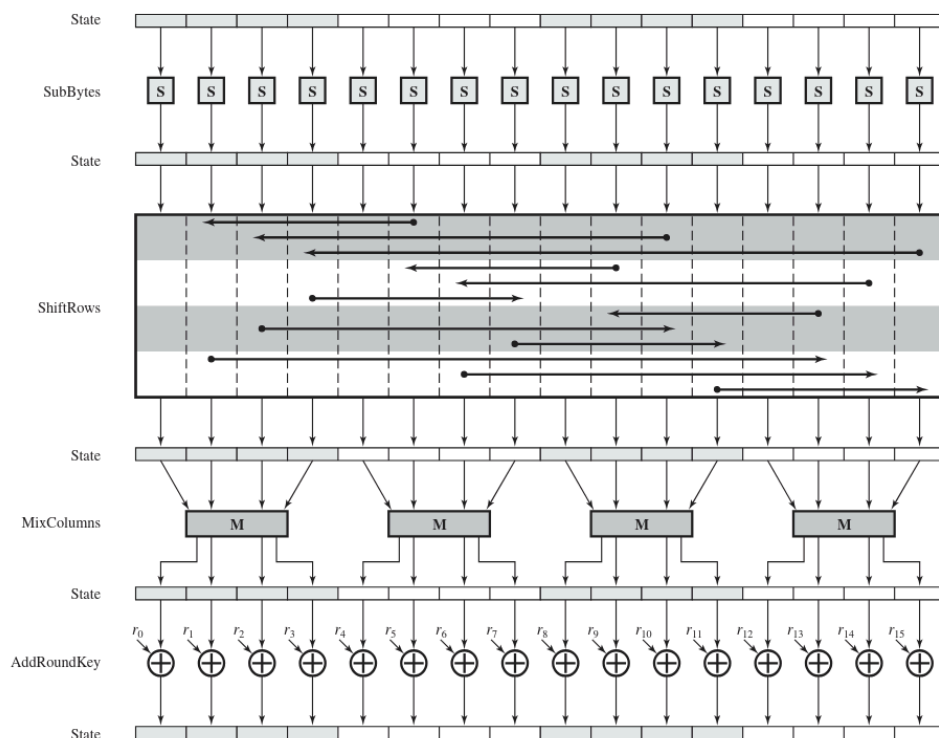
In questo algoritmo i round:

- Non seguono architettura Feistel.
- La chiave iniziale viene estesa in un array di 44 words da 32bit, 4 words (128 bit) costituiscono una chiave utilizzabile in un round.
- Vengono utilizzati 4 diversi stage, 1 di permutazione e 3 sostituzioni:
  - **Substitute bytes:** Utilizza delle tabelle per fare sostituzioni byte-a-byte.
  - **Shift Rows:** Permutazione effettuata sulle righe.
  - **Mix Columns:** Sostituzione che altera ogni byte in una colonna andandoli a smistare sulle restanti colonne.
  - **Add Round Key:** Un semplice *XOR* del blocco corrente con la chiave del round corrente.



La struttura è relativamente semplice sia per la cifratura che la decifratura. La cifratura inizia con una *Add Round Key* seguita da 9 round contenenti tutti gli stage visti sopra, seguiti infine da un round di tre stage.

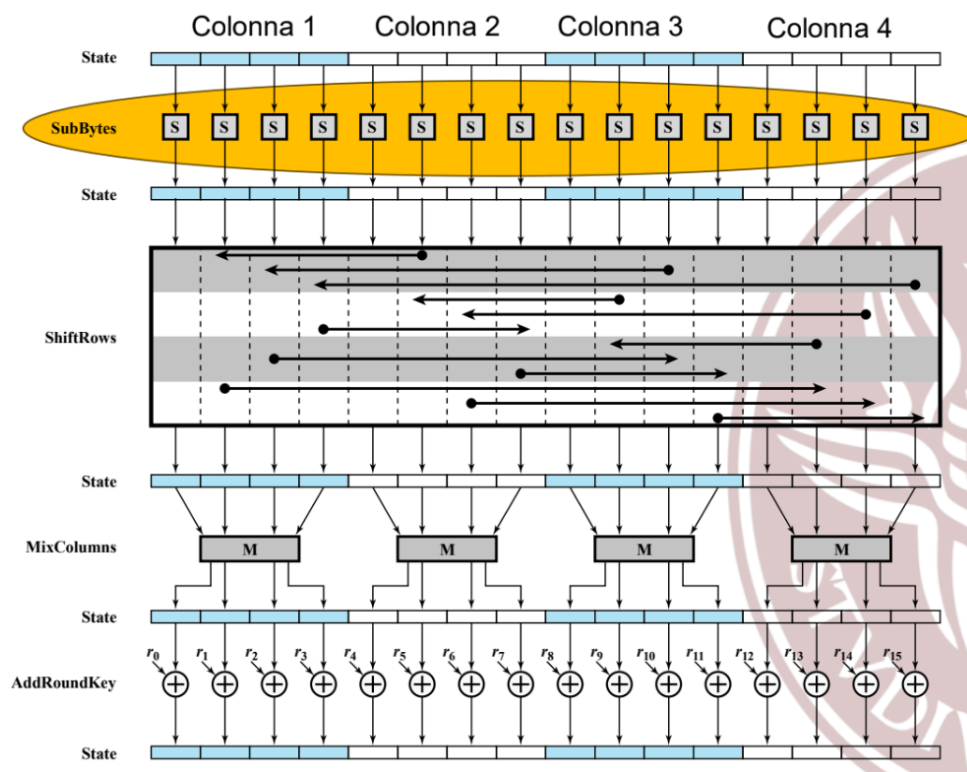
Nella figura sono rappresentati tutti gli stage di un round:



- Solo lo stage *Add Round Key* utilizza la chiave e per questo viene svolto all'inizio e alla fine della cifratura, tutti gli altri stage applicati all'inizio o alla fine sono invertibili senza conoscenza della chiave e non aggiungerebbero sicurezza.

- Anche gli altri sono utili però perchè servono a creare confusione nei dati, ma come detto prima da soli non bastano perchè non utilizzano la chiave.
- L'algoritmo di decifratura utilizza le funzioni inverse degli stage e per la chiave basta fare lo *XOR* come per la cifratura, inoltre va anche utilizzata la chiave estesa ma in ordine inverso.
- Dato che tutti gli stage sono invertibili, noteremo che allo stesso *livello verticale* nello *state array* avremo sempre lo stesso array.

Vediamo adesso i principali passaggi:



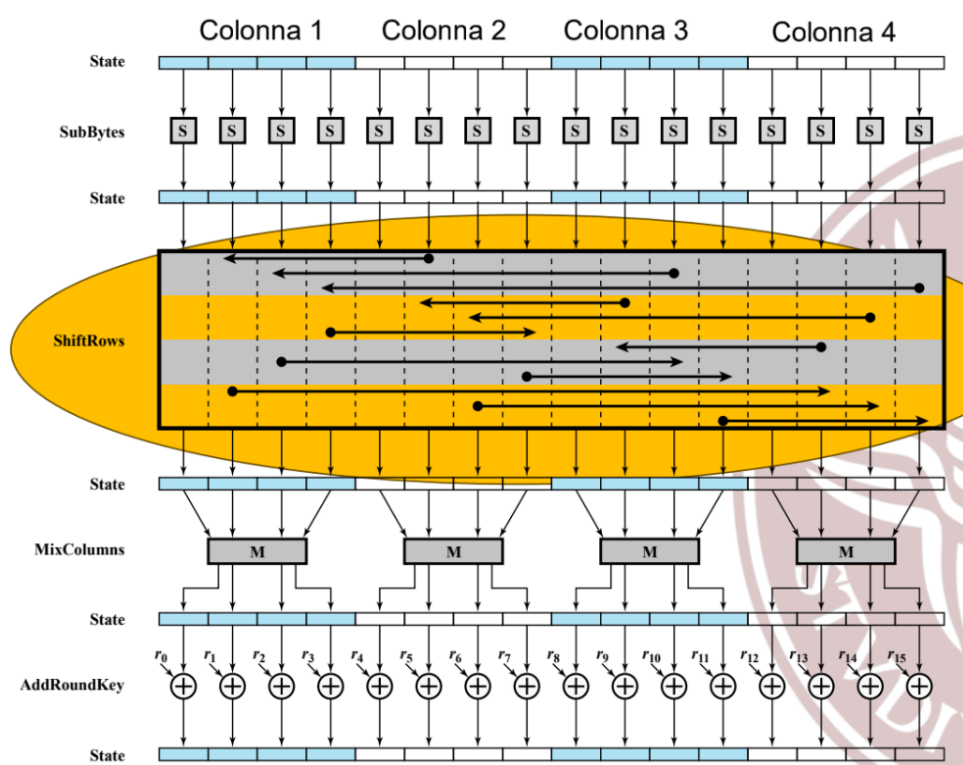
1. Effettuiamo una **Substitute Bytes**, una sostituzione byte a byte del blocco attraverso una tabella. Ovviamente esiste anche la matrice per la decifratura e queste due devono essere progettate in modo che ci sia poca correlazione fra byte in input e di output. Inoltre non deve essere possibile definire una relazione matematica per capire l'output dall'input.

La tabella è nella forma:

|   |   | y  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|   |   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
| x | 0 | 63 | 7C | 77 | 7B | F2 | 6B | 6F | C5 | 30 | 01 | 67 | 2B | FE | D7 | AB | 76 |
|   | 1 | CA | 82 | C9 | 7D | FA | 59 | 47 | F0 | AD | D4 | A2 | AF | 9C | A4 | 72 | C0 |
|   | 2 | B7 | FD | 93 | 26 | 36 | 3F | F7 | CC | 34 | A5 | E5 | F1 | 71 | D8 | 31 | 15 |
|   | 3 | 04 | C7 | 23 | C3 | 18 | 96 | 05 | 9A | 07 | 12 | 80 | E2 | EB | 27 | B2 | 75 |
|   | 4 | 09 | 83 | 2C | 1A | 1B | 6E | 5A | A0 | 52 | 3B | D6 | B3 | 29 | E3 | 2F | 84 |
|   | 5 | 53 | D1 | 00 | ED | 20 | FC | B1 | 5B | 6A | CB | BE | 39 | 4A | 4C | 58 | CF |
|   | 6 | D0 | EF | AA | FB | 43 | 4D | 33 | 85 | 45 | F9 | 02 | 7F | 50 | 3C | 9F | A8 |
|   | 7 | 51 | A3 | 40 | 8F | 92 | 9D | 38 | F5 | BC | B6 | DA | 21 | 10 | FF | F3 | D2 |
|   | 8 | CD | 0C | 13 | EC | 5F | 97 | 44 | 17 | C4 | A7 | 7E | 3D | 64 | 5D | 19 | 73 |
|   | 9 | 60 | 81 | 4F | DC | 22 | 2A | 90 | 88 | 46 | EE | B8 | 14 | DE | 5E | 0B | DB |
|   | A | E0 | 32 | 3A | 0A | 49 | 06 | 24 | 5C | C2 | D3 | AC | 62 | 91 | 95 | E4 | 79 |
|   | B | E7 | C8 | 37 | 6D | 8D | D5 | 4E | A9 | 6C | 56 | F4 | EA | 65 | 7A | AE | 08 |
|   | C | BA | 78 | 25 | 2E | 1C | A6 | B4 | C6 | E8 | DD | 74 | 1F | 4B | BD | 8B | 8A |
|   | D | 70 | 3E | B5 | 66 | 48 | 03 | F6 | 0E | 61 | 35 | 57 | B9 | 86 | C1 | 1D | 9E |
|   | E | E1 | F8 | 98 | 11 | 69 | D9 | 8E | 94 | 9B | 1E | 87 | E9 | CE | 55 | 28 | DF |
|   | F | 8C | A1 | 89 | 0D | BF | E6 | 42 | 68 | 41 | 99 | 2D | 0F | B0 | 54 | BB | 16 |

I primi 4 bit vengono usati per la coordinata  $x$  mentre gli ultimi 4 per la  $y$ . Ad esempio il byte con rappresentazione esadecimale 95 viene tradotto in 2A ovvero il byte presente alla riga 9, colonna 5.

A questo punto salviamo il risultato nello state array e passiamo all'operazione successiva:



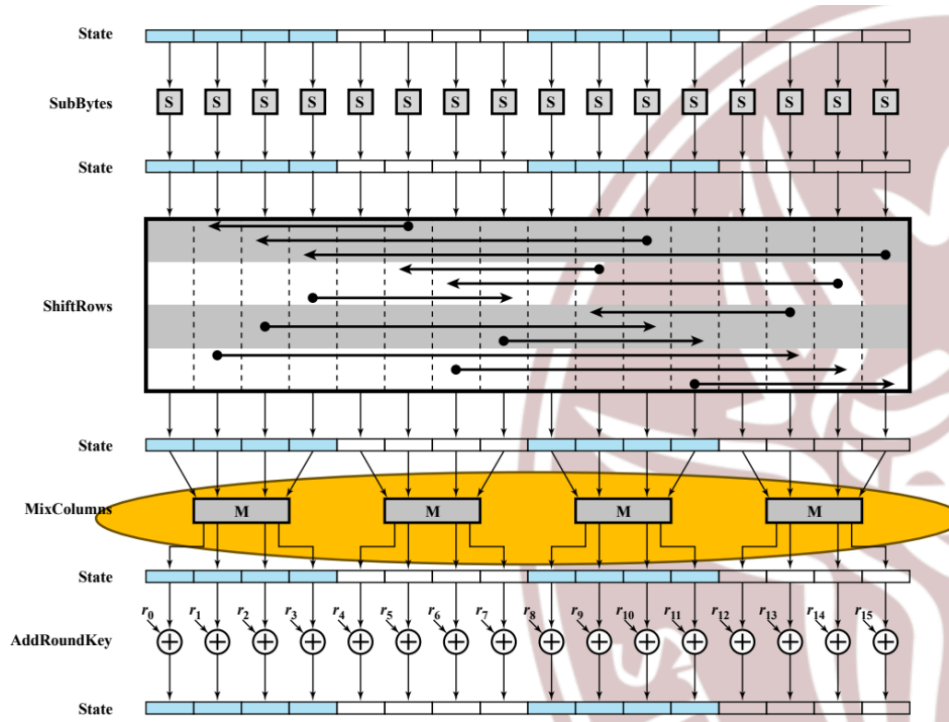
2. Effettuiamo uno **Shift Rows**, nello specifico:

- La prima riga non viene modificata.
- La seconda viene spostata di un byte a sinistra.
- La terza di 2 byte a sinistra.
- La quarta di 3 byte a sinistra.

Gli shift sono circolari quindi chi arriva al limite sinistro «sbuca» all'estremità destra. Questa trasformazioni si assicura che i 4 byte di una colonna vengano smistati in 4 colonne diverse.

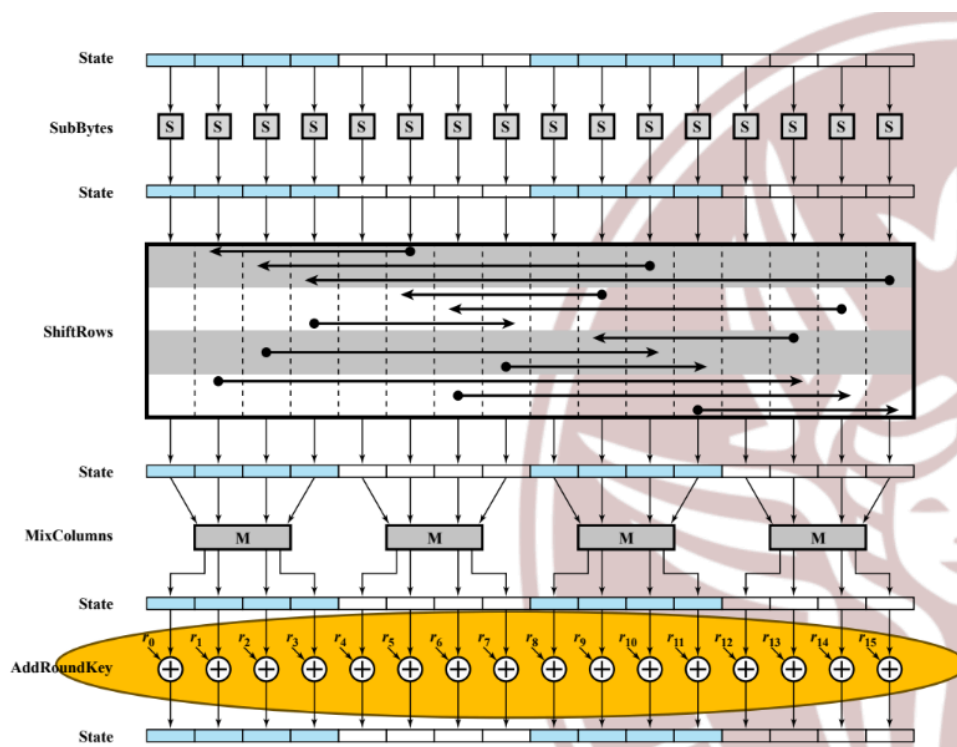
Una volta finito salviamo il risultato nello state array.

3. A questo punto, possiamo passare alla **Mix Column**:



In questa operazione prendiamo in input i 4 byte di una colonna e otteniamo in output i nuovi 4 byte, non ci deve essere perdita di informazione, questo significa che l'**output dipende dall'input**.

4. Come ultima operazione facciamo l'**Add Round Key**:



Per quanto riguarda l'algoritmo di espansione della chiave:



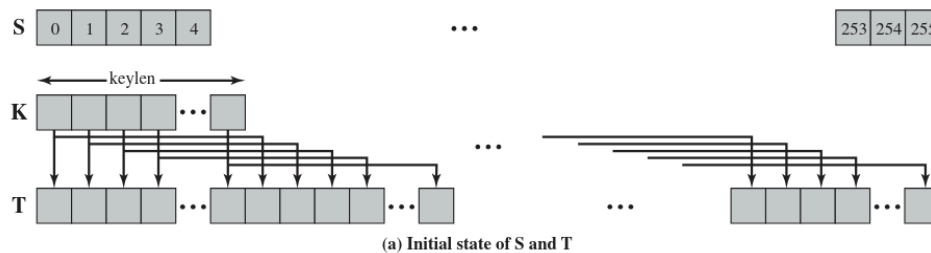
- **Inizializzazione:** Viene inizializzato un array  $S$  con tutti i valori da 0 a 255 in modo che  $S[0] = 0$ ,  $S[1] = 1$ , ...,  $S[255] = 255$ .

Viene anche inizializzato anche un vettore  $T$ , se la lunghezza della chiave  $K$  è 256bytes allora  $K$  viene trasferita su  $T$ , altrimenti per una chiave di lunghezza  $keylen$  bytes, i primi  $keylen$  bytes di  $K$  vengono copiati in  $T$  e poi  $K$  viene ripetuta tante volte quanto necessario per riempire  $T$ . Possiamo riassumere questa operazione con il seguente pseudocodice:

```

1 for i = 0 to 255 do
2   S[i] = i;
3   T[i] = K[i mod keylen]

```

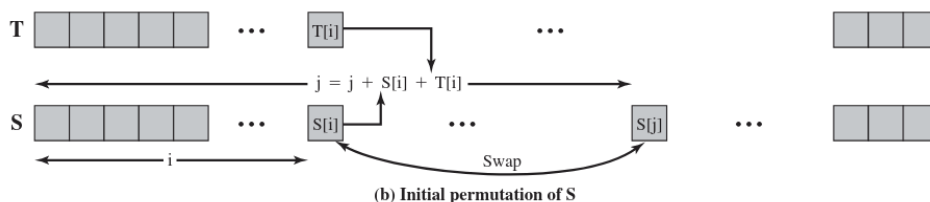


Viene poi effettuata la **prima permutazione**: Per fare questa utilizziamo  $T$ , scorriamo tutto  $S$  e per ogni  $S[i]$  lo swappiamo con un altro byte in  $S$  basandoci sullo schema fornito da  $T[i]$ . Nello specifico:

```

1 j = 0;
2 for i = 0 to 255 do
3   j = (j + S[i] + T[i]) mod 256;
4   Swap (S[i], S[j]);

```



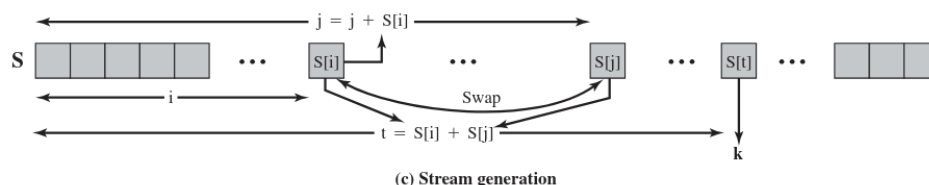
Iniziamo la **generazione delle chiavi**: A questo punto la chiave non è più necessaria e la generazione avviene ciclando su  $S$  e per ogni elemento  $S[i]$  lo swappiamo con un altro elemento di  $S$  basandoci sulla corrente configurazione di  $S$ . Se raggiungiamo l'elemento in posizione 255 torniamo a 0. Possiamo usare il seguente pseudocodice:

```

1 i, j = 0;
2 while (true)
3   i = (i + 1) mod 256;
4   j = (j + S[i]) mod 256;
5   Swap (S[i], S[j]);
6   t = (S[i] + S[j]) mod 256;
7   k = S[t];

```





Per cifrare effettuiamo uno *XOR* del valore  $k$  con il successivo byte del plaintext mentre per decifrare usiamo sempre lo *XOR* del valore  $k$  con il byte successivo del ciphertext.

## 4.2. Block Cipher Mode Operation

Gli algoritmi con chiave simmetrica e a blocchi processano un blocco alla volta, quindi quando il testo è più lungo è necessario spezzarlo in più blocchi. Possiamo applicare i block cipher in 5 **modes of operation** individuate dal NIST:

| Mode                        | Description                                                                                                                                                                                                | Typical Application                                                                                                                       |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Electronic Code book (ECB)  | Each block of 64 plaintext bits is encoded independently using the same key.                                                                                                                               | <ul style="list-style-type: none"> <li>Secure transmission of single values (e.g., an encryption key)</li> </ul>                          |
| Cipher Block Chaining (CBC) | The input to the encryption algorithm is the XOR of the next 64 bits of plaintext and the preceding 64 bits of ciphertext.                                                                                 | <ul style="list-style-type: none"> <li>General-purpose block-oriented transmission</li> <li>Authentication</li> </ul>                     |
| Cipher Feedback (CFB)       | Input is processed $s$ bits at a time. Preceding ciphertext is used as input to the encryption algorithm to produce pseudorandom output, which is XORed with plaintext to produce next unit of ciphertext. | <ul style="list-style-type: none"> <li>General-purpose stream-oriented transmission</li> <li>Authentication</li> </ul>                    |
| Output Feedback (OFB)       | Similar to CFB, except that the input to the encryption algorithm is the preceding DES output.                                                                                                             | <ul style="list-style-type: none"> <li>Stream-oriented transmission over noisy channel (e.g., satellite communication)</li> </ul>         |
| Counter (CTR)               | Each block of plaintext is XORed with an encrypted counter. The counter is incremented for each subsequent block.                                                                                          | <ul style="list-style-type: none"> <li>General-purpose block-oriented transmission</li> <li>Useful for high-speed requirements</li> </ul> |

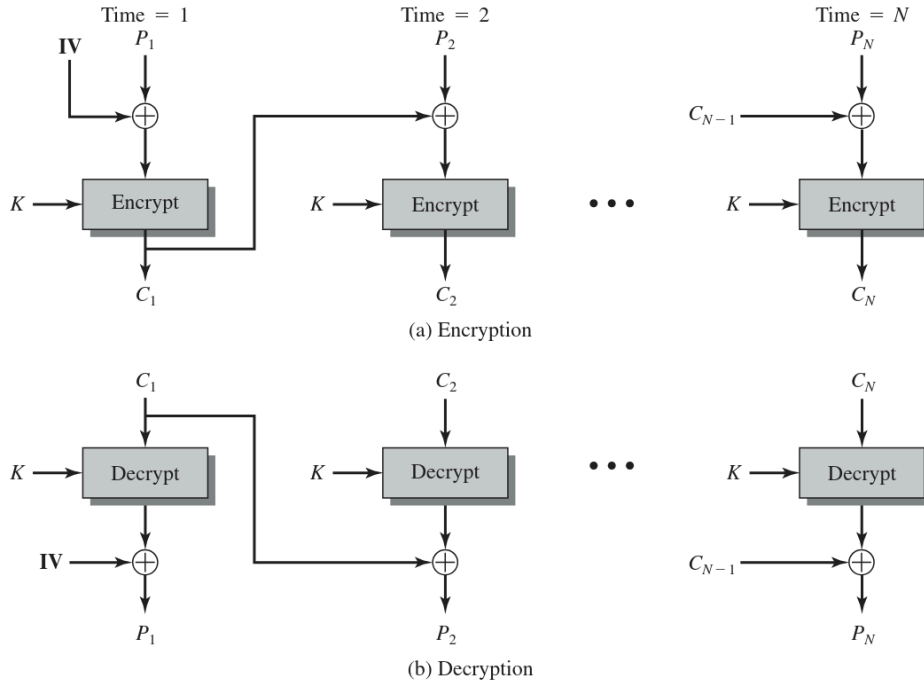
### 4.2.1. Electronic Codebook Mode (ECB)

È il metodo più semplice, infatti se i blocchi hanno dimensione  $b$  bits allora processiamo il testo  $b$  bits alla volta cifrandoli sempre con la stessa chiave. Per messaggi molto lunghi questa tecnica non è molto sicura infatti se ci sono ripetizioni nel testo queste avranno la stessa cifratura; oppure se il messaggio è molto strutturato potrebbe essere possibile effettuare cryptanalysis, ad esempio se ogni pagina inizia con gli stessi header abbiamo già delle combinazioni che possiamo sfruttare.

Per risolvere questo problema abbiamo bisogno di una tecnica che, anche se troviamo blocchi uguali, produca una cifratura diversa.

### 4.2.2. Cipher Block Chaining Mode (CBC)

In questa modalità di cifratura a blocchi l'input dell'algoritmo è lo *XOR* del blocco in plaintext corrente e il precedente blocco cifrato, viene comunque utilizzata la stessa chiave.



Per la decifratura ogni blocco cifrato viene passato all'algoritmo, il risultato viene messo in XOR con il precedente blocco cifrato per produrre il blocco in chiaro.

Per capire come funziona e soprattutto dimostrare il funzionamento possiamo scrivere:

$$C_j = E(K, [C_{j-1} \oplus P_j])$$

Dove  $E[K, X]$  è la funzione di cifratura effettuata sul blocco  $X$  utilizzando la chiave  $K$ . Continuiamo la dimostrazione scrivendo:

$$D(K, C_j) = D(K, E(K, [C_{j-1} \oplus P_j]))$$

$$D(K, C_j) = C_{j-1} \oplus P_j$$

$$C_{j-1} \oplus D(K, C_j) = C_{j-1} \oplus C_{j-1} \oplus P_j = P_j$$

Per produrre il primo blocco cifrato abbiamo bisogno di un **initilization vector** (IV) con cui effettuare lo XOR con il primo blocco di plaintext. Per la decifratura facciamo lo stesso ragionamento quindi il vettore IV viene messo in XOR con l'output dell'algoritmo di decifratura per recuperare il primo blocco di plaintext. Il vettore IV deve essere noto a mittente e destinatario e per avere più sicurezza possiamo proteggere anch'esso con una chiave. Possiamo farlo mandando il vettore usando cifratura ECB. Perché è importante proteggere IV? Perché anche se un attaccante non conosce la chiave ma riesce a leggere e modificare IV può modificare il primo blocco del messaggio, infatti:

$$C_1 = E(K, [IV \oplus P_1])$$

$$P_1 = IV \oplus D(K, C_1)$$

Adesso usiamo la notazione  $X[j]$  per indicare il bit in posizione  $j$ :

$$P_1[i] = IV[i] \oplus D(K, C_1)[i]$$

E utilizzando la proprietà dello XOR:

$$P_1[i]' = IV[i]' \oplus D(K, C_1)[i]$$

Dove la notazione con  $<'>$  indica il complemento, questo significa che se un attaccante può modificare i bit in IV allora può modificare anche i bit nel primo blocco senza conoscere la chiave.

#### 4.2.2.1. Propagazione degli errori

In ECB se avviene un errore di trasmissione nel testo cifrato, solo il corrispondente blocco in chiaro viene compromesso. In CBC l'errore si propaga, se avviene un errore in  $C_k$  allora corrompe sicuramente anche  $P_k$  e  $P_{k+1}$ , ma corrompe anche i successivi?

No, supponiamo  $C_k$  corrotto e notiamo come il blocco in chiaro  $P_{k+2}$  dipenda soltanto da  $C_{k+1}$  e  $C_{k+2}$ :

- $C_k' \neq C_k$  corrotto
- $P_k = C_{k-1} \oplus D(K, C_k)$  corrotto
- $P_{k+1} = C_k \oplus D(K, C_{k+1})$  corrotto
- $P_{k+2} = C_{k+1} \oplus D(K, C_{k+2})$  non corrotto

Se invece c'è un errore nella versione originale di  $P_k$ ? Attraverso quanti blocchi si propaga l'errore? Che effetti nota il destinatario? Un errore in  $P_k$  corrompe  $C_k$  ma dato che  $C_k$  è l'input per calcolare  $C_{k+1}$  anche quest'ultimo viene corrotto. Questo errore viene portato avanti in modo indefinito su tutti i blocchi cifrati successivi a  $C_k$ . Alla fine della ricezione però l'algoritmo di decifratura ricostruisce il messaggio corretto per tutti i blocchi ad eccezione del blocco contenente l'errore.

*Cifratura (sempre corrotti)*

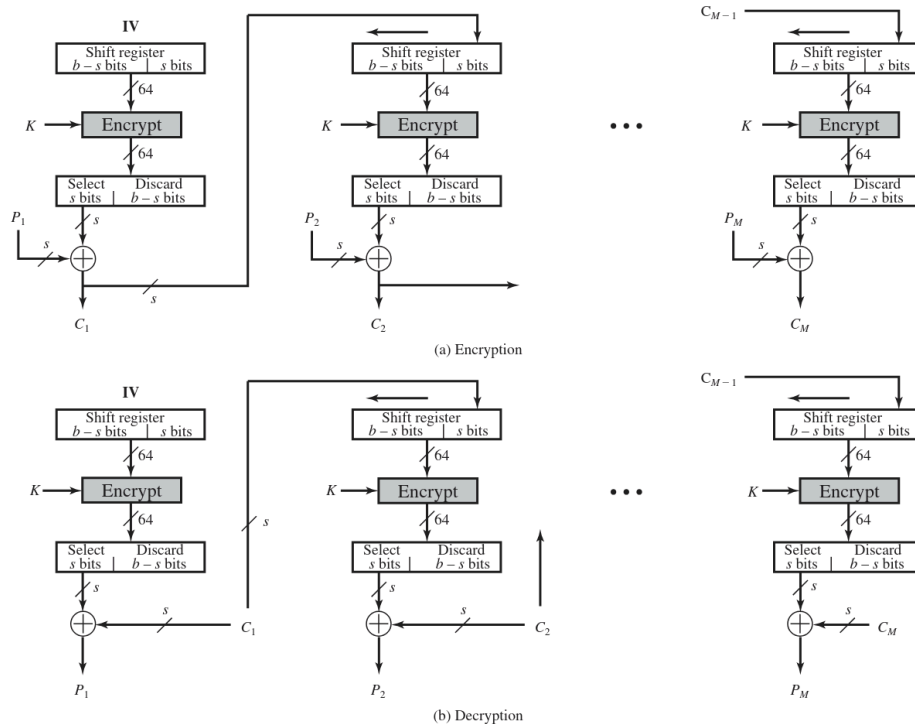
- $P_k' \neq P_k$
- $C_k = E(K, [C_{k-1} \oplus P_k'])$
- $C_{k+1} = E(K, [C_k \oplus P_{k+1}])$
- $C_{k+2} = E(K, [C_{k+1} \oplus P_{k+2}])$

*Decifratura*

- $P_k = C_{k-1} \oplus D(K, C_k) = C_{k-1} \oplus C_{k-1} \oplus P_k'$  corrotto
- $P_{k+1} = C_k \oplus D(K, C_{k+1}) = C_k \oplus C_k \oplus P_{k+1}$  ok
- $P_{k+2} = C_{k+1} \oplus D(K, C_{k+2}) = C_{k+1} \oplus C_{k+1} \oplus P_{k+2}$  ok

#### 4.2.3. Cipher Feedback Mode (CFB)

Con questa modalità possiamo convertire qualsiasi block cipher in uno stream cipher.

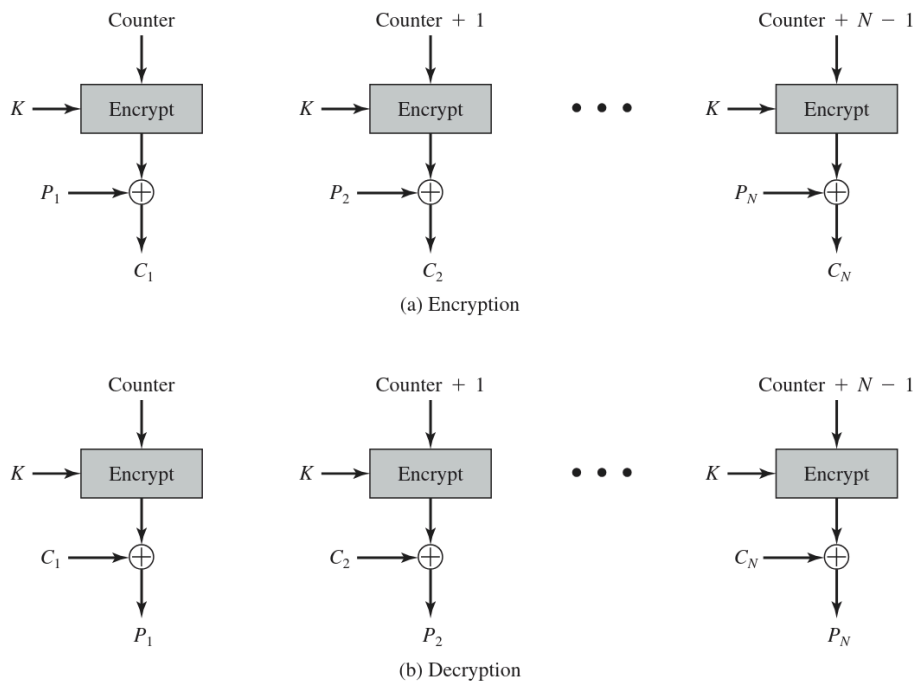


Nell'immagine sopra viene assunta come unità di trasmissione  $s$ , di solito sono 8 bits. Anche qui il plaintext viene concatenato nella funzione, quindi il testo cifrato di ogni unità di testo in chiaro è in funzione dell'unità precedente.

La funzione di cifratura prende in input uno shift di  $b$  bit che per la prima unità viene applicato ad un vettore IV, i  $s$  bit più significativi del risultato della funzione vengono messi in  $XOR$  con la prima unità di plaintext  $P_1$  per produrre la prima unità di testo cifrato  $C_1$  che verrà trasmessa. Il contenuto dello shift register viene shiftato a sinistra di  $s$  bits e  $C_1$  viene piazzato negli  $s$  bits più a destra del registro. Questo processo continua finché non vengono cifrate tutte le unità di plaintext.

Per la decifratura viene utilizzato lo stesso schema, l'unica differenza è che le unità di testo cifrato ricevute vengono messe in  $XOR$  con l'output che viene prodotto dalla funzione di cifratura.

#### 4.2.4. Counter Mode (CTR)



Viene utilizzato un *counter* che corrisponde alla grandezza dei blocchi del plaintext, l'unico requisito è che il valore counter sia diverso per ogni blocco plaintext che viene cifrato. Di solito questo valore viene inizializzato e poi incrementato ogni volta di 1. Per la cifratura viene cifrato il counter e poi messo in *XOR* con il blocco in chiaro, per la decifratura viene usato il processo inverso.

Questa tecnica presenta diversi vantaggi:

- **Hardware Efficiency:** Questa tecnica può essere utilizzata in parallelo su più blocchi.
- **Software Efficiency:** Praticamente uguale a quella hardware, abbiamo molte opportunità per sfruttare il parallelismo ad esempio con processori che utilizzano istruzioni SIMD.
- **Preprocessing:** La cifratura non dipende dall'input, questo significa che se abbiamo abbastanza memoria possiamo precalcolarci dei valori per poi effettuare solo gli *XOR* quando arriva l'input.
- **Random Access:** L'*i*-esimo blocco del testo in chiaro o del testo cifrato può essere processato senza dipendenze temporali da altri blocchi.
- **Provable Security:** È dimostrato essere sicuro quanto le altre tecniche di block cipher.
- **Simplicity:** A differenza di ECB e CBC, il CTR ha bisogno soltanto di implementare un algoritmo di cifratura e non di decifratura.

### 4.3. Cryptanalysis

Il processo con il quale un attaccante ricava delle informazioni su un messaggio o una chiave. La strategia dell'attaccante dipende da:

- Algoritmo di cifratura
- Informazioni a disposizione dell'attaccante (cosa viene inviato nel messaggio o altro)

Ci sono vari attacchi in base a cosa si conosce:

- Ciphertext only

- Known plaintext
- Chosen plaintext
- Chosen ciphertext
- Chosen text

Supponiamo che l'attaccante conosca sempre l'algoritmo di cifratura utilizzato (non sempre vero nella realtà)

### **Ciphertext only**

L'attaccante conosce soltanto il testo cifrato, può provare attacchi **bruteforce** sulla chiave, provare a capire dei pattern del testo se, ad esempio, conosce in che lingua è scritto il messaggio originale e quindi capire quali sono le parole più ripetute.

### **Known plaintext**

L'attaccante conosce uno o più messaggi in chiaro e la loro cifratura, oppure alcuni pattern presenti nel testo in chiaro. Con queste conoscenze l'attaccante potrebbe essere in grado di dedurre la chiave in base a come viene trasformato il testo.

### **Chosen plaintext**

L'attaccante è in grado di accedere al sistema e inserire un suo messaggio che presenta dei pattern in grado di rilevare la struttura della chiave. Ovviamente deve essere in grado di ottenere il messaggio cifrato risultante

### **Chosen ciphertext**

L'attaccante ha accesso al sistema per decifrare, seleziona un testo cifrato e lo invia al sistema, analizza il risultato per capire informazioni sulla chiave.

### **Chosen text**

L'attaccante ha accesso sia alla cifratura che alla decifratura, può far cifrare messaggi scelti da lui e anche decifrare.

### **Computationally Secure Encryption Schemes**

La cifratura è computazionalmente sicura se:

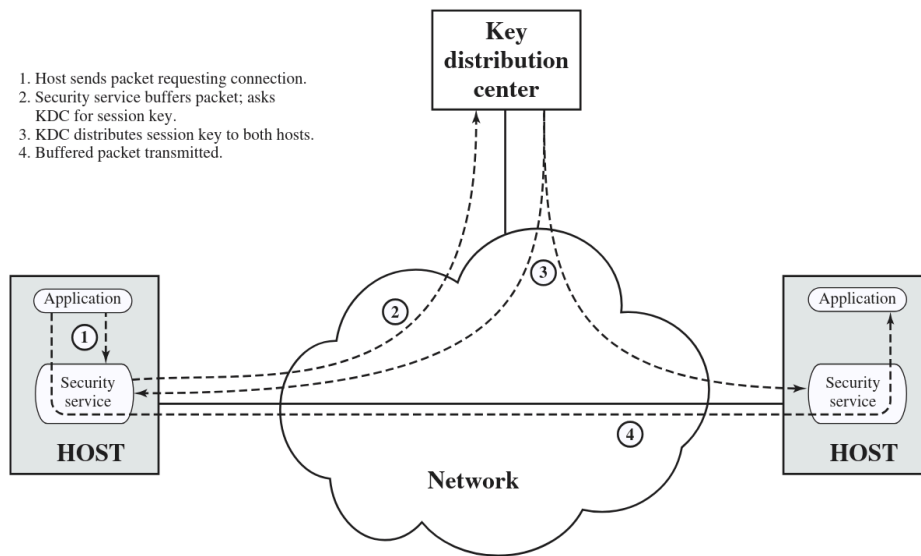
- Il costo per romperla è maggiore al valore delle informazioni che si ricavano.
- Il tempo per romperla supera il tempo utile di quell'informazione.

## **4.4. Key Distribution**

Abbiamo visto che questi algoritmi funzionano con chiave simmetrica, questo significa che mittente e destinatario devono dividerla in modo sicuro. Ci sono diversi modi per farlo, supponiamo le due parti siano  $A$  e  $B$ :

- $A$  potrebbe portare fisicamente la chiave a  $B$ .
- Qualcuno di esterno potrebbe selezionare e portare la chiave a  $B$ .
- Se  $A$  e  $B$  già avevano una chiave per parlare potrebbero usarla per trasmettere la nuova chiave.
- Se  $A$  e  $B$  hanno una connessione sicura con  $C$  questo potrebbe inviare la chiave al loro posto.

Le prime due opzioni prevedono uno scambio fisico, la terza è possibile ma se un attaccante entra in possesso di una chiave allora avrà anche tutte le successive. L'opzione 4 rimane la più attuabile:



In questo schema ignoriamo la cifratura nei collegamenti, potrebbe esserci o no, identifichiamo però due tipi di chiave:

- **Session Key:** Quando due *end systems* vogliono comunicare stabiliscono una connessione logica, per tutta la durata della sessione tutti i dati vengono cifrati con una one-time session key, alla fine della sessione la chiave viene distrutta.
- **Permanent Key:** Questa chiave viene utilizzata per distribuire le sessions key.

Questa configurazione consiste in:

- **Key Distribution Center:** Il Key Distribution Center (KDC) determina quali sistemi sono autorizzati a comunicare fra di loro, quando due di loro ottengono l'autorizzazione questo gli invia una session key per la connessione.
- **Security Service Module (SSM):** Esegue la cifratura e ottiene le session key degli utenti.

1. Un host invia una richiesta per parlare con un altro host
2. il SSM salva il pacchetto e chiede al KDC l'autorizzazione
3. Tra SSM e KDC avviene una comunicazione cifrata usando una master key condivisa solo fra loro, se il KDC approva la comunicazione genera una session key e la invia ai due SSM utilizzando una chiave permanente per ogni SSM.
4. L'SSM può adesso stabilire la connessione tra i due sistemi e rilasciare il pacchetto di connessione.

Tutti i dati che vengono scambiati adesso saranno cifrati dall'SSM utilizzando la session key.

## 5. Cifratura a Chiave Asimmetrica e Message Authentication

### 5.1. Secure Hash Functions

Abbiamo già visto precedentemente le proprietà che dovrebbero essere rispettate da una buona hash function, adesso vediamo diverse funzioni con un focus sul gruppo delle **Secure Hash Algorithms (SHA)**.

#### 5.1.1. Simple Hash Functions

Tutte le funzioni di hash operano nel seguente modo:

- L'input viene visto come una sequenza di blocchi da  $n$ -bits.

L'input viene processato un blocco alla volta in modo iterativo, alla fine dell'operazione si ottiene un risultato da  $n$ -bit. Per capire questo meccanismo si può prendere come esempio una delle funzioni più semplici ovvero con lo XOR, possiamo processarlo nel seguente modo:

$$C_i = b_{i1} \oplus b_{i2} \oplus \dots \oplus b_{im}$$

Dove:

- $C_i$ : L' $i$ -esimo bit dell'hash code con  $1 \leq i \leq n$
- $m$ : Numero dei blocchi da  $n$  bits in input
- $b_{ij}$ :  $i$ -esimo bit del  $j$ -esimo blocco
- $\oplus$ : XOR operation

Quindi ogni blocco viene messo in XOR con il successivo e alla fine otteniamo un valore grande quanto la grandezza dei blocchi.

Possiamo vedere questo processo graficamente con la seguente tabella:

|           | Bit 1    | Bit 2    | • • • | Bit $n$  |
|-----------|----------|----------|-------|----------|
| Block 1   | $b_{11}$ | $b_{21}$ |       | $b_{n1}$ |
| Block 2   | $b_{12}$ | $b_{22}$ |       | $b_{n2}$ |
|           | •        | •        | •     | •        |
|           | •        | •        | •     | •        |
|           | •        | •        | •     | •        |
| Block $m$ | $b_{1m}$ | $b_{2m}$ |       | $b_{nm}$ |
| Hash code | $C_1$    | $C_2$    |       | $C_n$    |

Questa operazione non fa altro che produrre un bit di parità per ogni bit ed è conosciuta come **longitudinal redundancy check**, è abbastanza efficace per check di integrità. Però c'è da notare che la probabilità che un errore nei dati non porti ad un cambiamento nell'hash code è di  $2^{-n}$ . Più i dati sono formattati e più perde di efficacia, ad esempio in quasi tutti i normali file di testo il primo bit di ogni ottetto è sempre 0, quindi se viene usato un valore hash da 128-bit (grandezza del risultato) invece di un'efficacia di  $2^{-128}$  avremo  $2^{-112}$ . Infatti dato che un ottetto è fatto da 8bit se abbiamo un hash value da 128 significa che abbiamo 16 ottetti (16 x 8), ma sappiamo che il primo bit di ognuno è 0 quindi ci saranno 16 bit bloccati a 0 che non cambieranno mai.

Per migliorare questo algoritmo possiamo applicare un **1-bit circular shift**, o rotazione, sull'hash value dopo che ogni blocco viene processato, possiamo riassumere l'operazione in:

1. Impostiamo l'hash value da  $n$ -bit a zero.
2. Processiamo ogni blocco successivo nel seguente modo:



- Ruotiamo il valore hash corrente a sinistra di 1-bit.
- XOR del blocco con il valore hash.

Questo serve a «randomizzare» l'input ed abbassare il numero di *regolarità* all'interno del testo, nonostante questo garantisca buona integrità dei dati non garantisce sicurezza. Questo perché lo XOR è bidirezionale e facilissima da invertire, un attaccante potrebbe:

1. Intercettare un messaggio vero e il suo Hash Value valido
2. Scrivere un messaggio a suo piacimento
3. Calcolare l'hash di questo messaggio con lo stesso algoritmo e ottenere un hash falso
4. Aggiunge in fondo al messaggio un falso blocco di bit senza senso
5. Poiché lo XOR è prevedibile l'attaccante può calcolare matematicamente l'esatto valore che questo blocco extra deve avere per far sì che sommandolo al risultato finale diventi magicamente indentico all'hash value vero.

Cifrare tutto non è la soluzione in questo caso, era stata proposta una soluzione dove oltre allo XOR applicato a blocchi da 64bits veniva cifrato tutto il messaggio con CBC, il procedimento era il seguente:

- Dato un messaggio in blocchi da 64bits  $X_1, X_2, \dots, X_N$  definiamo l'hash code  $C$  come lo XOR blocco a blocco di tutti i blocchi e con l'hash code messo come ultimo blocco:

$$C = X_{N+1} = X_1 \oplus X_2 \oplus \dots \oplus X_N$$

Poi viene cifrato il messaggio intero e inserito l'hash code per produrre il messaggio cifrato  $Y_1, Y_2, \dots, Y_{N+1}$ . Sono stati scoperti diversi modi in cui il testo cifrato può essere manipolato e non rilevato dall'hashcode. Per esempio (seguendo le regole del CBC):

$$\begin{aligned} X_1 &= IV \oplus D(K, Y_1) \\ X_i &= Y_{i-1} \oplus D(K, Y_i) \\ X_{N+1} &= Y_N \oplus D(K, Y_{N+1}) \end{aligned}$$

Ma sappiamo che  $X_{N+1}$  è l'hash code:

$$\begin{aligned} X_{N+1} &= X_1 \oplus X_2 \oplus \dots \oplus X_N \\ &= [IV \oplus D(K, Y_1)] \oplus [Y_1 \oplus D(K, Y_2)] \oplus \dots \oplus [Y_{N-1} \oplus D(K, Y_N)] \end{aligned}$$

Dato che i termini nell'equazione precedente possono essere messi in XOR in qualsiasi ordine, significa che l'hash code non cambia se il testo cifrato viene permutato.

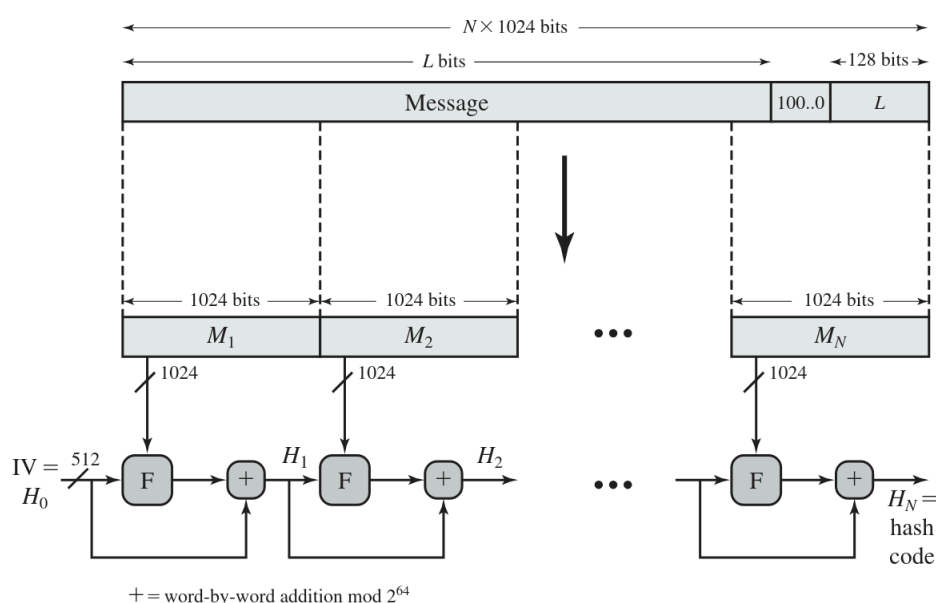
### 5.1.2. The SHA Secure Hash Function

Esistono diverse versioni di SHA:

|                     | SHA-1      | SHA-256    | SHA-384     | SHA-512     |
|---------------------|------------|------------|-------------|-------------|
| Message digest size | 160        | 256        | 384         | 512         |
| Message size        | $< 2^{64}$ | $< 2^{64}$ | $< 2^{128}$ | $< 2^{128}$ |
| Block size          | 512        | 512        | 1024        | 1024        |
| Word size           | 32         | 32         | 64          | 64          |
| Number of steps     | 80         | 64         | 80          | 80          |
| Security            | 80         | 128        | 192         | 256         |

Nel corso ci concentriamo su SHA-512. L'algoritmo prende in input un messaggio con una lunghezza massima di  $2^{128}$  bits e produce un output di 512-bit chiamato *digest*. L'input viene processato in blocchi da 1024-bit.

Graficamente possiamo vedere il processo:



Consiste in:

1. **Append padding bits:** Al messaggio viene aggiunto un padding in modo che la lunghezza sia congruente a 896 modulo 1024, il padding viene sempre aggiunto anche se il messaggio è alla lunghezza desiderata. Il numero di bit di padding quindi si trova nel range di 1-1024 bits e il padding consiste in un singolo bit impostato a 1 seguito da tutti i bit necessari impostati a 0.
2. **Append Length:** Viene aggiunto alla fine un blocco da 128bit, questo viene trattato come un intero senza segno a 128bit e contiene la lunghezza del messaggio originale (prima dell'aggiunta del padding).

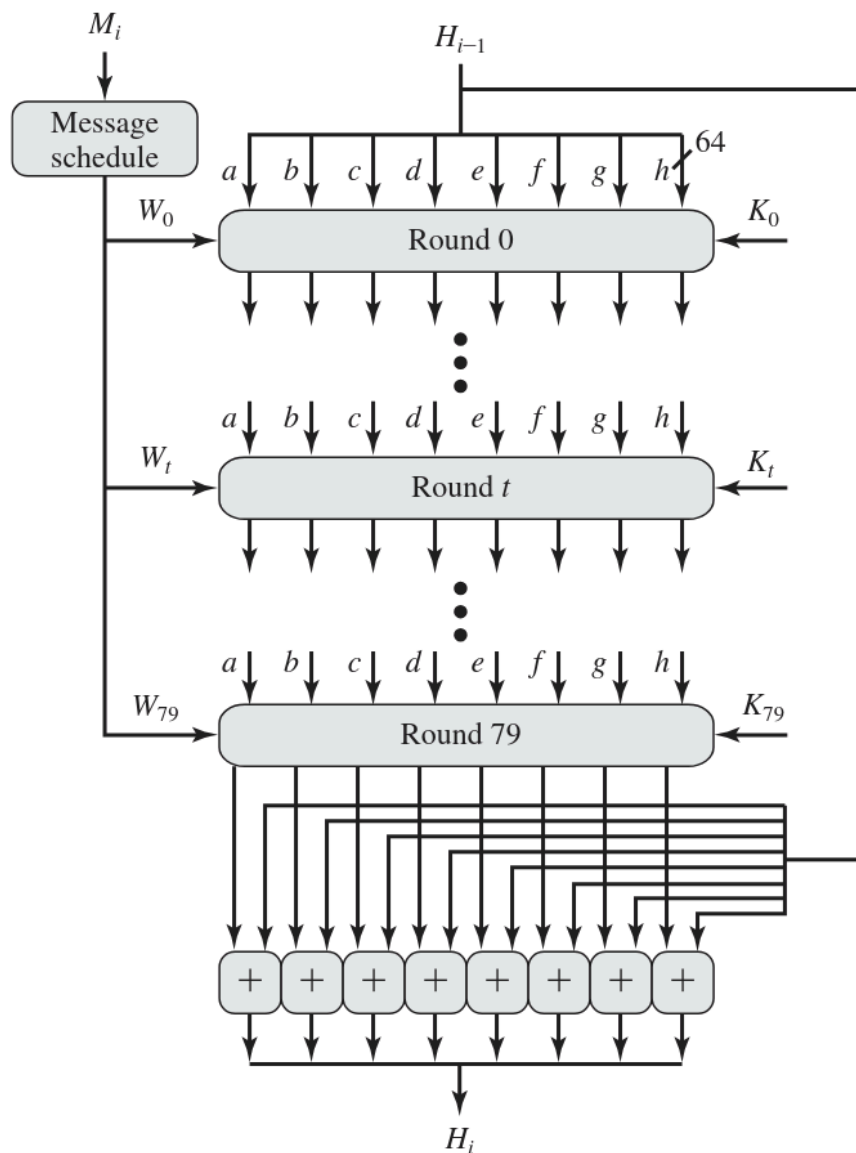
Dopo questi 2 passaggi otteniamo un messaggio che ha una lunghezza multiplo di 1024bits. Nell'immagine il messaggio viene rappresentato come una sequenza di blocchi da 1024bits  $M_1, M_2, \dots, M_N$  e la lunghezza totale è quindi  $N \times 1024$ bits.

3. **Initialize hash buffer:** Un buffer da 512bit viene utilizzato per memorizzare i risultati intermedi e finale della funzione di hash. Il buffer può essere rappresentato come 8 registri da 64bits inizializzati a questi valori esadecimali:

$a = 6A09E667F3BCC908$     $e = 510E527FADE682D1$   
 $b = BB67AE8584CAA73B$     $f = 9B05688C2B3E6C1F$   
 $c = 3C6EF372FE94F82B$     $g = 1F83D9ABFB41BD6B$   
 $d = A54FF53A5F1D36F1$     $h = 5BE0CD19137E2179$

Questi valori sono salvati in formato **big-endian** ovvero dove il bit più significativo si trova nella posizione più a sinistra. I valori sono stati ottenuti prendendo i primi 64bits delle parti frazionarie delle radici quadrate dei primi 8 numeri primi.

4. **Process message in 1024-bit (128-word) blocks:** È il cuore dell'algoritmo, consiste in 80 rounds e nella figura precedente viene indicato come  $F$ . La logica invece viene illustrata con questa figura:



Ogni round prende come input il buffer da 512bit e aggiorna il suo contenuto. Come input del primo round abbiamo il valore hash intermedio  $H_{i-1}$ , ogni round  $t$  utilizza un valore a 64bit  $W_t$  derivato dal blocco corrente (1024bit)  $M_i$ . Ad ogni round si utilizza anche una costante  $K_t$  dove  $0 \leq t \leq 79$  (una per ogni round), queste costanti sono i primi 64bits delle parti frazionarie

delle radici cubiche dei primi 80 numeri primi. Le costanti forniscono un set di patterns a 64bit randomizzati che dovrebbe rimuovere ogni regolarità presente nel testo in input. Le operazioni eseguite in un round sono *shift circolari* e operazioni booleane basate su *AND*, *OR*, *NOT* e *XOR*. L'output dell'80-esimo round viene sommato insieme all'input del primo round  $H_{i-1}$  e viene prodotto  $H_i$ , la somma viene fatta in modo indipendente per ognuna delle 8 words del buffer usando un'addizione modulo  $2^{64}$ .

5. **Output:** Dopo che tutti gli  $N$  blocchi sono stati processati abbiamo l'output da 512bit.

L'algoritmo SHA-512 ha la proprietà che ogni bit dell'hash code è una funzione di ogni bit in input, le ripetizioni molto complesse della funzione  $F$  producono risultati molto «disordinati» ed è molto difficile che due messaggi scelti a caso, anche simili, abbiano lo stesso hash code. In ogni caso trovare due messaggi con lo stesso hash code è nell'ordine delle  $2^{256}$  operazioni, mentre trovare un messaggio dato un hash code è nell'ordine delle  $2^{512}$ .

## 5.2. HMAC

Vediamo l'approccio con hash code per l'**autenticazione dei messaggi**, questa tecnica è motivo di interesse perché:

- Le funzioni hash generalmente vengono eseguite più velocemente rispetto agli algoritmi di crittografia come il *DES*.
- Sono molto più diffuse librerie di codice per funzioni hash.

Funzioni hash come *SHA-1* però non sono state progettate per essere usate come MAC e non possono essere usate in questo modo direttamente perché non si basano su una chiave segreta. Ci sono stati diversi tentativi per introdurre una chiave negli algoritmi hash esistenti e quello che ha ricevuto più supporto è l'**HMAC**, questo è stato anche usato per implementare MAC nelle connessioni internet in vari protocolli.

Obiettivi dell'HMAC:

- Utilizzare, senza modifiche, le esistenti funzioni hash.
- Permettere la sostituzione della funzione hash scelta con altre più sicure.
- Preservare le performance originali della funzione hash.
- Utilizzare le chiavi in modo semplice.
- Avere un'ottima comprensione della «forza» della funzione hash che stiamo usato, il vantaggio infatti è che se la funzione è sicura allora lo è anche HMAC.

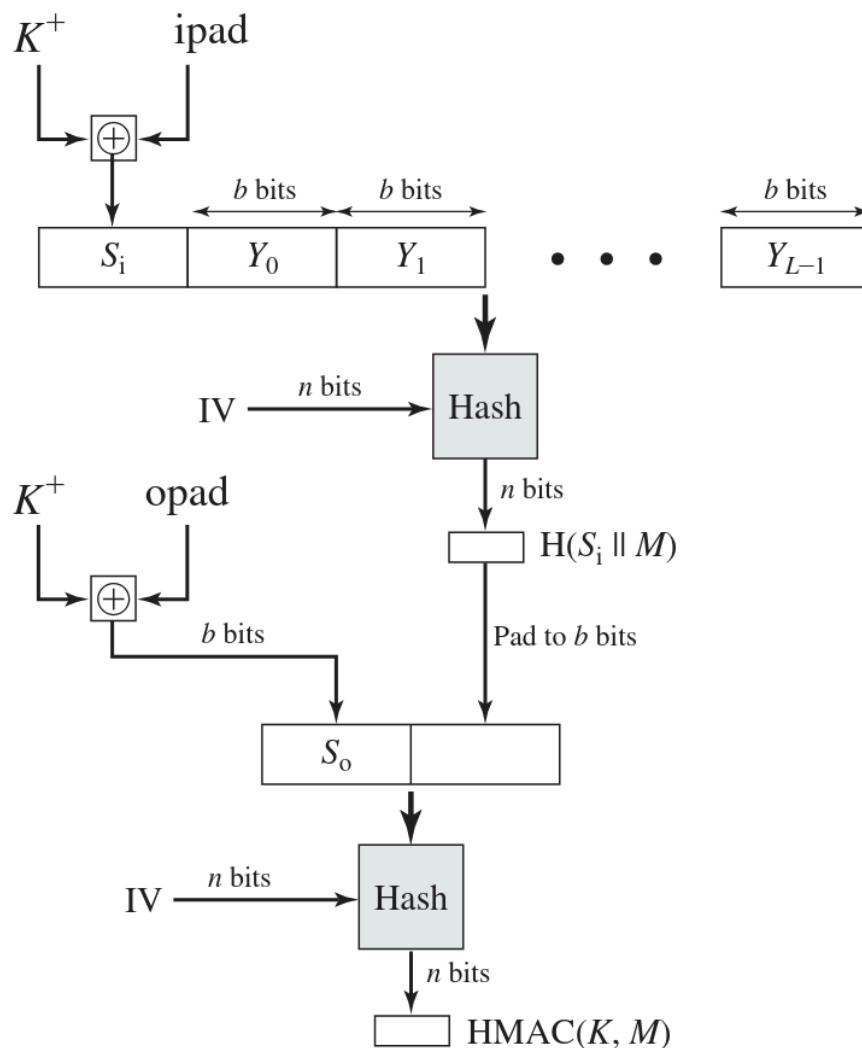
HMAC tratta la funzione hash come una «*black box*», questo ha due benefici, il primo è che parte dell'algoritmo è già pronta senza modifiche mentre il secondo è che possiamo appunto sostituire facilmente una funzione con un'altra, ad esempio se su quella in uso vengono scoperte delle debolezze.

Vediamo come funziona l'algoritmo, definiamo i seguenti termini:

- $H$  - funzione hash (SHA).
- $M$  - Messaggio input di HMAC (incluso il padding specificato nella funzione hash)
- $L$  - Numero di blocchi in  $M$
- $Y_i$  - Blocco  $i$ -esimo di  $M$  con  $0 \leq i \leq L - 1$
- $b$  - Numero di bits in un blocco
- $n$  - Lunghezza dell'hashcode prodotto dalla funzione hash

- $K$  - Chiave segreta, se la lunghezza della chiave è maggiore di  $b$ , allora la chiave viene data in input alla funzione hash per produrre una chiave da  $n$ -bit, una lunghezza consigliata è  $\geq n$ .
- $K^+$  -  $K$  con un padding di zeri a sinistra in modo che la lunghezza sia  $b$  bits.
- $ipad$  - 00110110 (36 in esadecimale) ripetuto  $\frac{b}{8}$  volte
- $opad$  - 01011100 (5C in esadecimale) ripetuto  $\frac{b}{8}$  volte

Graficamente abbiamo:



La formula formale dell'HMAC è:

$$HMAC(K, M) = H[(K^+ \oplus opad) \parallel H[k^+ \oplus ipad] \parallel M]$$

Che possiamo descrivere in questi passaggi:

1. Aggiungo zeri a sinistra di  $K$  per creare una stringa lunga  $b$  chiamata  $K^+$
2. XOR di  $K^+$  con  $ipad$  per produrre il blocco da  $b$ -bit  $S_i$ .
3. Aggiungere alla fine di  $S_i$   $M$
4. Applicare  $H$  allo stream generato dallo step 3.
5. XOR di  $K^+$  con  $opad$  per produrre il blocco da  $b$ -bit  $S_o$
6. Aggiungere alla fine del blocco  $S_o$  il risultato dello step 4.

7. Applicare  $H$  allo stream generato dallo step 6 per ottenere il risultato finale.

Da notare che lo  $XOR$  con *ipad* significa invertire una metà dei bit di  $K$  e anche con *opad* ma con un set di bit diverso. Passare  $S_i$  e  $S_o$  all'algoritmo di hash significa infatti creare due chiavi pseudocasuali da  $K$ . HMAC dovrebbe avere un tempo d'esecuzione simile alla funzione hash per lunghi messaggi, infatti aggiunge 3 esecuzione della funzione base.

### 5.2.1. Security of HMAC

HMAC fa affidamento quasi interamente sulla funzione hash che utilizza, è stato provato infatti che la probabilità di successo di un attacco contro HMAC da parte di un attaccante che può leggere l'output di un messaggio generato da un utente è equivalente a uno dei seguenti attacchi alla funzione hash:

1. L'attaccante è in grado di calcolare un output della compression function anche con IV random, segreto e sconosciuto all'attaccante.
2. L'attaccante trova collisioni nella hash function anche con IV randomico e segreto.

Nel primo attacco, possiamo vedere la compression function come la hash function applicata ad un singolo blocco da  $b$ -bit, per questo attacco l'array IV è rimpiazzato da un segreto casuale di  $n$  bits. Un attacco di questo tipo alla funzione hash richiede un attacco di forza bruta sulla chiave che è nell'ordine di  $2^n$ , o il *birthday attack* che è un caso particolare del secondo attacco. Nel secondo attacco, l'attaccante è in ricerca di due messaggi  $M$  e  $M'$  che producono lo stesso hash quindi  $H(M) = H(M')$ , questo è il *birthday attack* detto prima. Per questo tipo di attacco siamo nell'ordine di  $2^{\frac{n}{2}}$  per una lunghezza dell'hashcode di  $n$ .

Con questi dati potremmo pensare che ad esempio l'algoritmo MD5 non sia più sicuro dato che un numero di tentativi di  $2^{64}$  oggi è abbastanza fattibile, quindi MD5 renderebbe HMAC non sicuro? **No**, questo perché HMAC utilizza una chiave segreta  $K$ , per calcolare l'HMAC di un messaggio non basta il messaggio stesso ma serve anche questa chiave. Quindi MD5 da solo è debole perché l'attaccante potrebbe mettersi a genere hashcode finché non trova una collisione, ma se usato con HMAC allora l'attacco deve diventare **online** ovvero si deve mettere in ascolta e sperare di trovare due messaggi con lo stesso hash, per farlo dovrebbe catturare circa  $2^{64}$  messaggi, anche con una rete a 1Gbps che trasmette senza mai fermarsi ci vorrebbero comunque 150.000 anni di intercettazioni continue, inoltre questo presuppone che la chiave non venga **mai cambiata** per lo stesso tempo.

### 5.3. Authenticated Encryption (AE)

**NON HO TROVATO LE PAGINE DEL LIBRO SU QUESTO PARAGRAFO, DEVO CAPIRLO E SCRIVERLO MEGLIO**

Sono sistemi di cifratura che garantiscono sia confidenzialità che integrità. (Message encryption + message authentication) Sono implementati attraverso block cipher, il primo è stato **OCB AE (2014)**:

- Approvato come standard in *IEEE 802.11* wireless LAN e incluso in *MiniSec*.

Poi abbiamo **OCB** che è simile a ECB:

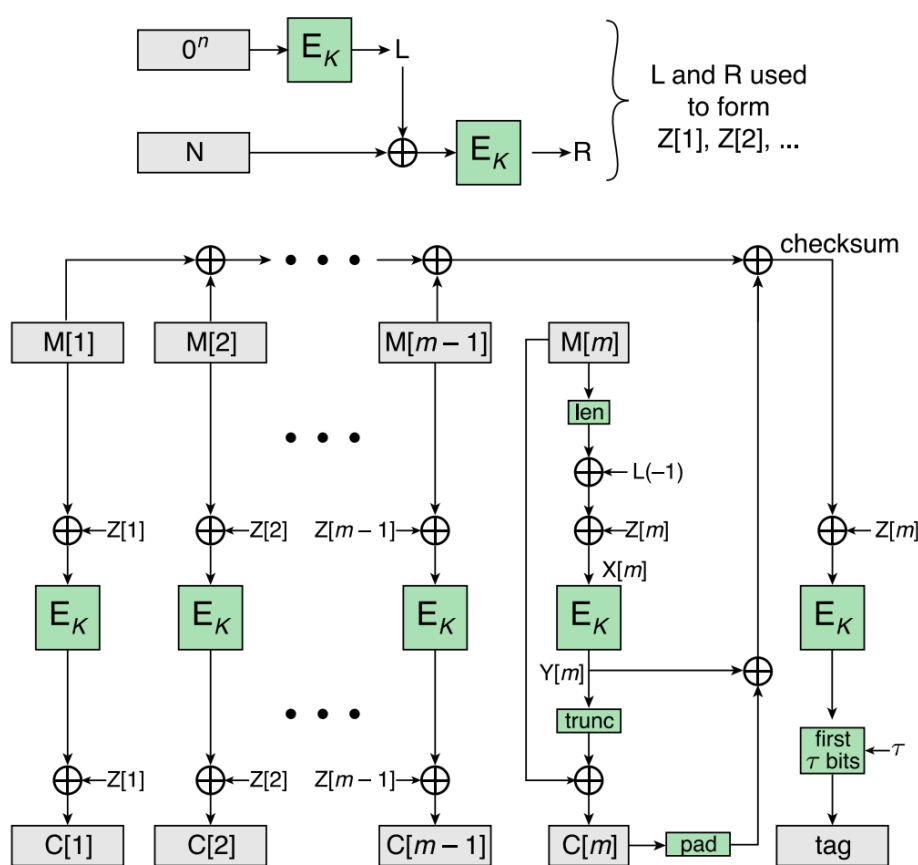
- Ha la stessa debolezza di ECB quindi stesso messaggio con stessa chiave produce lo stesso risultato, non è quindi sicuro per messaggi lunghi.

- Introduce un nuovo vettore  $Z$ , che viene utilizzato per effettuare  $XOR$  con ogni blocco  $M[i]$  nel seguente modo:

$$C[i] = E_k(M[i] \oplus Z[i]) \oplus Z[i]$$

- AES è usato come algoritmo di cifratura.
- $M$  viene diviso in blocchi da  $n$ -bits.
- Il numero totale di blocchi è  $m = \lceil \frac{\text{len}(M)}{n} \rceil$
- $N$ , un valore arbitrario di  $n$ -bit chiamato **nonce**
  - Se vengono cifrati più messaggi con la stessa chiave allora sarebbe meglio cambiare questo valore per ogni messaggio.
  - Ogni valore differente di  $N$  produce un set diverso di  $Z[i]$ .

Graficamente abbiamo:



$n$  = block length in bits

$N$  = nonce

$\text{len}(M[m])$  = length of  $M[m]$  represented as an  $n$ -bit integer

$\text{trunc}(Y[m])$  = deletes least significant bits so that result is same length as  $M[m]$

$\text{pad}$  = pad with least significant 0 bits to length  $n$

$\tau$  = length of authentication tag

Calcolo di  $Z[i]$ :

- $L[0] = L = E_k(0^n)$
- $R = E_k(N \oplus L)$
- $L[i] = 2 \cdot L[i - 1]; 1 \leq i \leq m$ ; dove la moltiplicazione è un'operazione nei *campi finiti*.
- $Z[1] = L \oplus R$

- $Z[i] = Z[i - 1] \oplus L[\text{ntz}(i)]; 2 \leq i \leq m$  con  $\text{ntz}(i)$  indichiamo il numero di 0 meno significativi in  $i$ .

### Operazioni nei campi finiti

Servono a non far «esplodere» il risultato, se il risultato sfiora il numero massimo che i bit possono contenere allora questo risultato viene riavvolto ovvero, se ad esempio abbiamo come risultato 260 e come valore massimo 255 allora il risultato sarà invece 4 perché una volta raggiunto 256 riniziamo a contare da 0.

Vediamo adesso come avviene il calcolo di  $C[m]$ , della checksum e del tag:

- $\text{len}(M[m]) \leq n$  bit
- $C[m]$  viene calcolato in modo diverso da  $C[i = \{1, m - 1\}]$
- $C[m]0^*$  è uguale a  $C[m]$  con un padding fino a raggiungere una lunghezza di  $n$ .
- $L[-1] = L \cdot 2^{-1}$
- La checksum è in funzione di  $M$  e  $K$
- Viene prodotto un Tag di lunghezza **tau**
- **tau** determina il livello di autenticazione, più è alto **tau** maggiore è il livello di autenticazione.

*Algoritmo di cifratura:*

1. Partizionare  $M$  in  $M[1], \dots, M[m]$
2.  $L = L(0) = E_K(0^n)$
3.  $R = E_k(N \oplus L)$
4. for  $(i = 1; i \leq m)$   $L(i) = 2 \cdot L(i - 1)$
5.  $L(-1) = L \cdot 2^{-1}$
6.  $Z[1] = L \oplus R$
7. for  $(i = 2; i \leq m)$   $Z[i] = Z[i - 1] \oplus L(\text{ntz}(i))$
8. for  $(i = 1; i \leq m - 1)$   $C[i] = E_K(M[i] \oplus Z[i]) \oplus Z[i]$
9.  $X[m] = \text{len}(M[m]) \oplus L(-1) \oplus Z[m]$
10.  $Y[m] = E_K(X[m])$
11.  $C[M] = M[m] \oplus (\text{first } \text{len}(M[m]) \text{ bits of } Y[m])$
12. Checksum =  $M[1] \oplus \dots \oplus M[m - 1] \oplus C[m]0^* \oplus Y[m]$
13. Tag =  $E_K(\text{Checksum} \oplus Z[m])[\text{first } \tau \text{ bits}]$

*Algoritmo di decifratura:*

1. Partizionare  $M$  in  $M[1], \dots, M[m]$
2.  $L = L(0) = E_K(0^n)$
3.  $R = E_K(N \oplus L)$
4. for  $(i = 1; i \leq m)$   $L(i) = 2 \cdot L(i - 1)$
5.  $L(-1) = L \cdot 2^{-1}$
6.  $Z[1] = L \oplus R$
7. for  $(i = 2; i \leq m)$   $Z[i] = Z[i - 1] \oplus L(\text{ntz}(i))$
8. for  $(i = 1; i \leq m - 1)$   $M[i] = D_K(C[i] \oplus Z[i]) \oplus Z[i]$
9.  $X[m] = \text{len}(M[m]) \oplus L(-1) \oplus Z[m]$



10.  $Y[m] = E_K(X[m])$
11.  $M[m] = (\text{first } \text{len}(C[m]) \text{ bits of } Y[m]) \oplus C[m]$
12.  $\text{Checksum} = M[1] \oplus \dots \oplus M[m-1] \oplus C[m]0^* \oplus Y[m]$
13.  $\text{Tag}' = E_K(\text{Checksum} \oplus Z[m])[\text{first } \tau \text{ bits}]$

## 5.4. RSA Public-Key Encryption

RSA è un block cipher dove il plaintext e il testo cifrato sono interi tra 0 ed  $n-1$  per un  $n$ . Cifratura e decifratura sono nella seguente forma, dato un blocco di testo in chiaro  $M$  e un blocco di testo cifrato  $C$ :

$$C = M^e \bmod n$$

$$M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$$

Sia mittente che destinatario devono conoscere i valori di  $n$  ed  $e$ , inoltre solo il destinatario conosce il valore di  $d$ . Questo algoritmo ha come chiave pubblica  $PU = \{e, n\}$  e come chiave privata  $PR = \{d, n\}$ , per essere un buon algoritmo di cifratura a chiave pubblica deve soddisfare i seguenti requisiti:

1. Deve essere possibile trovare valori di  $e, d, n$  tali che  $M^{ed} \bmod n = M$  per ogni  $M < n$ .
2. Deve essere relativamente facile calcolare  $M^e$  e  $C^d$  per ogni valore  $M < n$ .
3. Deve essere praticamente impossibile determinare  $d$  dato  $e$  ed  $n$ .

Riguardo al primo requisito, significa che dobbiamo trovare una relazione nella forma

$$M^{ed} \bmod n = M$$

Per soddisfare questa equazione, dato che usiamo numeri interi, è necessario soddisfare:

$$e \cdot d \bmod \Phi(n) = 1$$

Dove con  $\Phi(n)$  indichiamo la **funzione toziente di Eulero**, questa rappresenta il numero di interi positivi più piccoli di  $n$  che sono «relativamente primi» a  $n$  cioè che non hanno divisori in comune con  $n$  a parte 1. Calcolare questa funzione per numeri grandi è sì molto semplice ma praticamente impossibile dato che andrebbero controllati tutti i numeri prima di  $n$  per vedere se sono primi rispetto ad esso. È qui che sono d'aiuto i numeri primi, infatti la matematica ci dice che se prendiamo  $n = p \cdot q$  con  $p$  e  $q$  primi allora possiamo calcolare  $\Phi(n) = (p-1)(q-1)$ , infatti se un numero è primo non condivide divisori con nessuno dei numeri che lo precedono e quindi  $\Phi(q) = q-1$ . Quindi se conosciamo i due numeri che hanno «creato»  $n$  allora calcolare  $\Phi(n)$  si calcola subito altrimenti ci vorrebbero tantissimi anni.

Adesso utilizziamo quindi  $\Phi(n)$  per creare le chiavi:

- L'esponente pubblico  $e$ , lo scegliamo a caso e lo usiamo per cifrare le chiavi. L'unica regola è che non deve avere divisori in comune con  $\Phi(n)$ . La chiave pubblica che diamo a tutti è  $(e, n)$ .
- L'esponente privato  $d$  lo scegliamo quindi tale che rispetti  $e \cdot d \bmod \Phi(n) = 1$ , quindi un numero che moltiplicato per  $e$  ci dia come resto 1 se diviso per  $\Phi(n)$ .

Per cifrare il messaggio dobbiamo prendere il messaggio  $M$ , elevarlo per  $e$  e dividerlo per  $n$  prendendo il resto:

$$C = M^e \bmod n$$

Per decifrare, riceviamo  $C$  lo eleviamo per  $d$ . Matematicamente stiamo facendo:

$$(M^e)^d \bmod n = M^{ed} \bmod n$$

Grazie al fatto che abbiamo costruito  $e$  e  $d$  usando la funzione di Eulero possiamo dire che elevare alla potenza  $e$  e poi alla potenza  $d$  sono due operazioni che si annullano lasciandoci il messaggio originale  $M$ .

### Passaggi da eseguire

Prima di tutto generiamo le chiavi

1. Selezioniamo  $p, q$  primi e diversi fra loro
2. Calcoliamo  $n = p \cdot q$
3. Calcoliamo  $\Phi(n) = (p - 1)(q - 1)$
4. Selezioniamo un intero  $e$  con  $\text{M.C.D.}(\Phi(n), e) = 1; 1 < e < \Phi(n)$
5. Calcoliamo  $d$  sapendo che  $d \cdot e \bmod \Phi(n) = 1$
6. Creiamo la chiave pubblica  $KU = \{e, n\}$
7. Creiamo la chiave privata  $KR = \{d, n\}$

Poi possiamo cifrare un messaggio (un intero)

- Testo in chiaro  $M$  con  $M < n$
- Testo cifrato é dato da  $C = M^e \bmod n$

Per decifrare

- Dal testo cifrato  $C$  otteniamo  $M = C^d \bmod n$

### Esempio di utilizzo

1. Selezioniamo due numeri primi  $p = 17, q = 11$
2. Calcoliamo  $n = pq = 187$
3. Calcoliamo  $\Phi(n) = (p - 1)(q - 1) = 160$
4. Selezioniamo  $e$  tale che é primo e minore rispetto a  $\Phi(n) = 160$ , ad esempio  $e = 7$ .
5. Determiniamo  $d$  tale che  $d \cdot e \bmod 160 = 1$  e  $d < 160$ , in questo caso  $d = 23$

Otteniamo come chiavi chiave pubblica  $\{7, 187\}$  e come chiave privata  $\{23, 187\}$  Vogliamo inviare il numero 88:

1. Lo cifriamo calcolando  $88^7 \bmod 187$  e otteniamo 11
2. Per decifrare calcoliamo  $11^{23} \bmod 187$  e riotteniamo 88

#### 5.4.1. Sicurezza dell'RSA

Esistono 4 diversi modi per attaccare l'algoritmo RSA:

- **Brute Force:** Provare tutte le possibili chiavi private.
- **Mathematical Attacks:** Ci sono diversi approcci possibili, tutti equivalenti per quanto riguarda gli sforzi da fare per fattorizzare il prodotto di due numeri primi.
- **Timing Attacks:** Questi dipendono dal tempo di esecuzione dell'algoritmo di decifrazione.

- **Chosen ciphertext attacks:** Sfruttano le proprietà dell’RSA. Non affronteremo questo argomento.

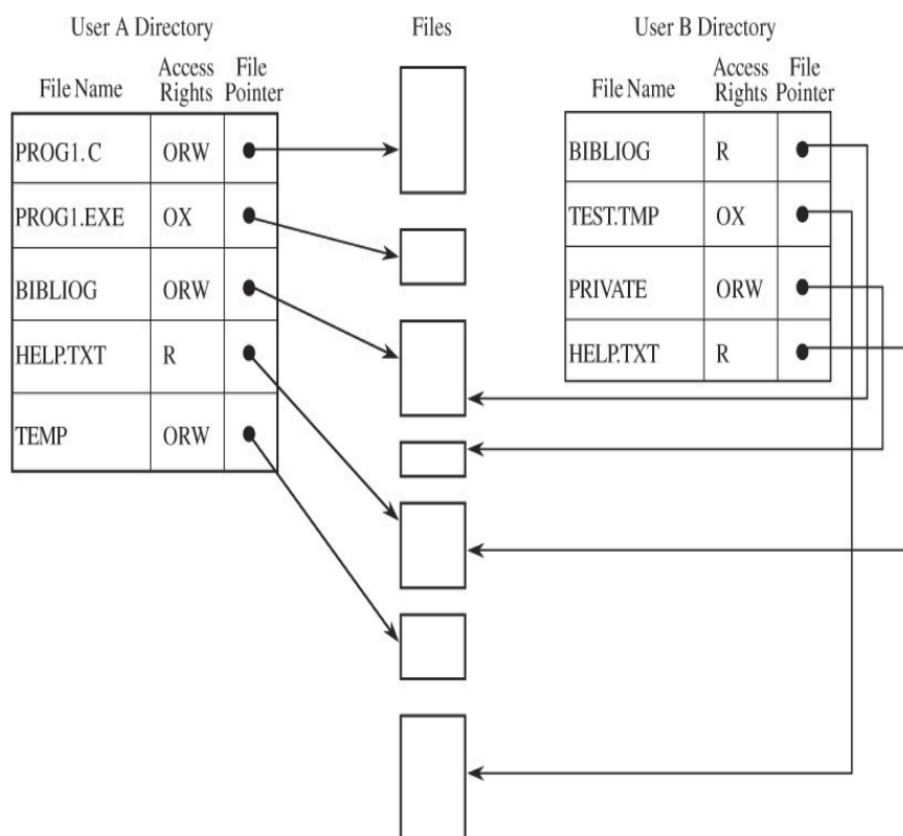
La difesa contro il primo tipo di attacchi é uguale a molti altri algoritmi ovvero usare uno spazio delle chiavi enorme, in questo caso maggiore é il numero di bit di  $d$  maggiore sarà il tempo necessario a «trovarlo», in questo caso però visti i calcoli necessari, maggiore é la grandezza della chiave e maggiore sarà il tempo necessario per cifrare e decifrare.

#### 5.4.1.1. Il problema della fattorizzazione

Ci sono 3 approcci per attaccare RSA matematicamente:

- Fattorizzare  $n$  nei suoi due fattori primi, questo permette di calcolare subito  $\Phi(n)$  e determinare anche  $d$ .
- Determinare  $\Phi(n)$  direttamente senza determinare prima  $p, q$ , come prima ci permette di calcolare subito  $d$
- Determinare direttamente  $d$  senza trovare prima  $\Phi(n)$ .

Il problema principale é quello di fattorizzare  $n$  nei suoi due fattori primi, infatti determinare  $\Phi(n)$  é equivalente a fattorizzare  $n$ . Possiamo quindi vedere i tempi di fattorizzazione di  $n$  come «benchmark» della sicurezza di RSA. Per un  $n$  grande con grandi fattori primi, la fattorizzazione é un problema difficile ma non quanto lo era prima. Nella seguente tabella sono misurati i tempi in MIPS-years ovvero un anno in cui un processore che può eseguire un milione di istruzioni al secondo processa senza interruzioni. Il numero di cifre fa riferimento al numero di cifre della chiave.



Col tempo sono stati scoperti diversi algoritmi per la fattorizzazione dei numeri ma in generale, per ora, una chiave nel range di 1024-2048 bits è ritenuta sicura. Inoltre i ricercatori hanno dato qualche restrizione aggiuntiva per rendere meno semplice la fattorizzazione di  $n$ :

- $p$  e  $q$  devono avere una lunghezza simile ma non troppo piccoli.
- Sia  $(p - 1)$  che  $(q - 1)$  devono avere un fattore primo grande, questo perché se hanno solo fattori primi piccoli possono essere ricavati subito da un algoritmo chiamato **Pollard's  $p - 1$**
- Il M.C.D.  $(p - 1, q - 1)$  deve essere piccolo, questo perché se è grande significa che genera meno chiavi possibili.

Inoltre è stato dimostrato (**Michael Wiener**) che se  $e < n$  e  $d < n^{\frac{1}{4}}$  allora  $d$  può essere facilmente determinato.

#### 5.4.1.2. Timing Attacks

Come detto prima si basano sul tempo di esecuzione dell'algoritmo di decifratura, **Paul Kocher** ha dimostrato che in alcuni casi è possibile determinare la chiave privata solamente tenendo traccia di quanto tempo ci mette il computer a decifrare il messaggio.

Per usare questo attacco si indovina la chiave bit a bit, procedendo da sinistra verso destra:

- L'attaccante crea un messaggio cifrato e lo invia, questo messaggio è costruito in modo tale che quando il computer arriva al bit che lui sta cercando di indovinare, se quel bit è 1 allora l'operazione sarà lentissima mentre se è 0 allora sarà normale.
- Si cronometra il tempo totale che impiega il computer a rispondere
- Se il tempo è stato estremamente lungo allora può impostare quel bit a 1 altrimenti a 0. Si passa al bit successivo.

Esistono tre difese contro questo tipo di attacchi:

- **Constant Exponentiation Time:** Si forza il computer ad impiegare sempre lo stesso tempo massimo per ogni bit, ovviamente andiamo a peggiorare le prestazioni.
- **Random Delay:** Si aggiunge del «rumore» facendo fare delle piccole pause casuali al processore, il difetto è che l'hacker può inviare il messaggio tante volte e fare una media dei tempi di risposta rilevando i tempi reali.
- **Blinding:** Consiste nel modificare il messaggio ricevuto per imbrogliare il cronometro dell'hacker.

In alcuni RSA è implementato il Blinding nel seguente modo:

1. Genera un segreto casuale  $r$  tra 0 e  $n - 1$ .
2. Calcola  $C' = C(r^e) \bmod n$  dove  $e$  è l'esponente pubblico.
3. Calcola  $M' = (C')^d \bmod n$  come il classico RSA
4. Calcola  $M = M' \cdot r^{-1} \bmod n$

Ovvero:

1. Viene scelto un numero casuale
2. Lo cifriamo e moltiplichiamo per il messaggio ricevuto.
3. Decifriamo questo valore ottenuto andando a imbrogliare il cronometro dell'hacker
4. Moltiplichiamo il valore ottenuto con l'inverso del numero inventato prima per ottenere il messaggio originale.

Questa operazione di blinding diminuisce le performance di un 2-10%.

## 5.5. Diffie-Hellman

L'algoritmo permette a due utenti di scambiarsi una chiave segreta in modo sicuro.

Vengono scelti due elementi pubblici:

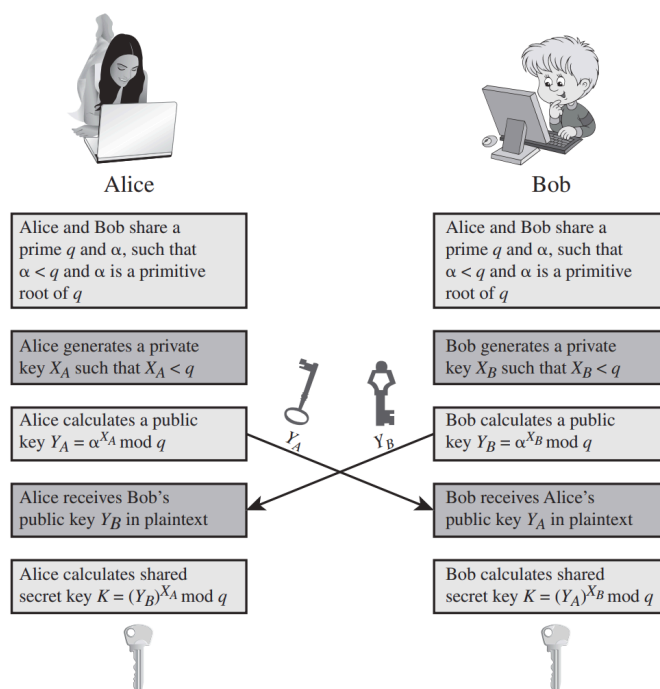
- Un numero primo  $q$
- $\alpha < q$  e  $\alpha$  deve essere una radice primitiva di  $q$

Una radice primitiva modulo  $n$  è un numero  $g$  che elevato a potenza riesce a generare da solo tutti i numeri possibili di quel modulo. Ad esempio se  $n = 7$  allora 3 è una radice primitiva infatti elevando 3 a tutte le potenze da 1 a 6 e calcolando il resto della divisione con 7 otteniamo tutti i numeri da 1 a 6. Non tutti i numeri possiedono radici primitive, cioè avviene solo per  $2, 4, p^k$  o  $2p^k$  dove  $p$  è un primo dispari.

- L'utente A genera le sue chiavi, prima seleziona una chiave privata  $X_A$  t.c.  $X_A < q$  e poi calcola la chiave pubblica  $Y_A = \alpha^{X_A} \bmod q$
- L'utente B genera le sue chiavi nello stesso modo quindi  $X_B < q$  e  $Y_B = \alpha^{X_B} \bmod q$
- Devono scambiarsi la loro chiave pubblica e poi possono calcolare la chiave in comune:
  - A la può calcolare eseguendo  $K = (Y_B)^{X_A} \bmod q$
  - B la può calcolare eseguendo  $K = (Y_A)^{X_B} \bmod q$
- Questa chiave  $K$  è il valore che si sono «scambiati» e che possono usare per le loro future comunicazioni.

### 5.5.1. Sicurezza del Diffie-Hellman

Prendiamo questo esempio dove un algoritmo utilizza Diffie-Hellman per scambiare le chiavi:



Questo tipo di comunicazione non è sicura contro attacchi del tipo **man-in-the-middle**. Alice e Bob devono scambiarsi la chiave e Darth è l'avversario:

1. Darth prepara l'attacco generando due chiavi private casuali  $X_{D1}$  e  $X_{D2}$  e genera anche le corrispettive chiavi pubbliche.
2. Alice trasmette la sua chiave pubblica a Bob.
3. Darth intercetta  $Y_A$  e trasmette  $Y_{D1}$  a Bob, inoltre calcola anche  $K2 = (Y_A)^{X_{D2}} \bmod q$
4. Bob riceve  $Y_{D1}$  e calcola  $K1 = (Y_{D1})^{X_B} \bmod q$
5. Bob trasmette  $Y_B$
6. Darth intercetta  $Y_B$  e trasmette  $Y_{De}$  ad Alice, inoltre calcola anche  $K1 = (Y_B)^{X_{D1}} \bmod q$
7. Alice riceve  $Y_{D2}$  e calcola  $K2 = (Y_{D2})^{X_A} \bmod q$

A questo punto Alice e Bob sono convinti di essersi scambiati le chiavi ma invece tutte le loro future comunicazioni saranno lette da Darth che potrà anche modificare i messaggi. Questo algoritmo é vulnerabile a questo tipo di attacchi perché non c'è autenticazione del mittente, é possibile quindi risolvere questi problemi utilizzando firme digitali o certificati a chiave pubblica.

## 6. User Authentication

Il processo di autenticazione per gli utenti avviene in due step:

- **Identification:** Identificare la persona
- **Authentication:** Provare che questa persona è realmente chi dice di essere.

Ovviamente utilizziamo il termine persona ma questa potrebbe tranquillamente essere un altro sistema.

L'autenticazione può basarsi su:

- **Something the individual knows:** Ad esempio una password, risposte a delle domande o un PIN
- **Something the individual possesses:** Chiavi elettroniche, smart cards, ci si riferisce a questi strumenti come *token*
- **Something the individual is:** Dati biometrici come impronta digitale o retina.
- **Something the individual does:** Si basa sempre su dati biometrici ma includono azioni come ad esempio parlare, scrivere o il ritmo di scrittura.

### 6.1. Authentication with something you know

Le password offrono sicurezza nell'autenticazione ma questa viene compromessa dal comportamento umano, le principali difficoltà nel loro utilizzo sono:

- Per garantire la massima sicurezza andrebbe utilizzata una password diversa per ogni «oggetto» al quale accediamo ma è scomodo.
- Se qualcuno di non autorizzato viene a conoscenza della password sarà necessario cambiarla e informare altri utenti autorizzati del cambio avvenuto.
- La password potrebbe andare persa e gli amministratori di sistema dovranno fornirne un'altra.

In alcuni studi hanno indicato i 12 step che, di solito, un attaccante usa per violare una password:

- Inserire una password vuota
- Password uguale allo user ID
- Se è derivata dal nome dell'utente
- Contenuta in un piccolo dizionario di password
- Contenuta in un dizionario completo Inglese
- Contenuta in un dizionario non inglese
- Contenuta in un dizionario inglese considerando anche lettere grandi o sostituzioni con numeri, ad esempio O-0, E-3...
- Ottenuta tramite brute force con tutte le combinazioni alfanumeriche
- Ottenuta tramite brute force con tutte le combinazioni dal set di caratteri completo.

Il principale ostacolo è quello del tempo richiesto per indovinare la password, infatti qualsiasi password può essere indovinata e quello che ci protegge è il numero di tentativi richiesti.

I principali attacchi alle password sono:

- **Dictionary Attack:** Sono delle raccolte di parole e frasi provenienti ad esempio da film, serie TV, mitologia ecc... che spesso vengono utilizzate anche come password. Questi dizio-

nari non vengono soltanto usati da parte degli attaccanti ma anche dagli amministratori di sistema per individuare utenti che inseriscono password deboli.

Per indovinare una password si può provare anche a rimpicciolire il set di possibilità andando ad analizzare l'utente, infatti è molto probabile che una password sia un dato personale come ad esempio il nome di un animale domestico o che derivi da queste informazioni.

Per verifica un utente il sistema operativo controlla che ID e password forniti dall'utente combacino all'interno del suo database, questa tabella potrebbe essere letta da utenti malintenzionati e per questo i sistemi operativi non salvano le password in chiaro ma una versione cifrata di queste. Quindi quando un utente prova ad autenticarsi il sistema operativo applica la stessa funzione di cifratura all'input e verifica che combaci con quello memorizzato da lui. (non sempre si usa la cifratura ma comunque operazioni simili.) Questo ovviamente non ci protegge da attacchi brute force ma è possibile renderli più difficili ad esempio inserendo un tempo di attesa tra una password e un'altra.

- **Rainbow Table:** Cifrare le password ha comunque una vulnerabilità ovvero che password identiche avranno la stessa password cifrata, quindi se un utente si impossessa di una tabella con password cifrata può comunque capire chi sta utilizzando la stessa password, anche se non sa quale sia.

Per risolvere questo problema si utilizza il **salt** ovvero aggiungere dei dati insieme alla password prima di cifrarla, questi dati possono essere ad esempio il momento di iscrizione o altri dati randomici.

Vediamo adesso come costruire password forti e resistenti anche ad **exhaustive attack** ovvero quando l'attaccante prova tutte le combinazioni possibili:

- Utilizzare anche i numeri oltre alle lettere a-z e usare anche varianti maiuscole, in questo modo portiamo a 62 il set di caratteri disponibili per ogni carattere della password.
- Altra roba ma non credo serva per il corso.

## 6.2. Authentication Based on Biometrics: Something you are

Qui utilizziamo caratteristiche biometriche dell'utente per autenticarlo, ad esempio:

- Impronta digitale
- Geometria della mano
- Retina
- Voce
- Scrittura
- Riconoscimento facciale

Uno dei vantaggi rispetto alle password è che i dati biometrici non possono essere persi, rubati, dimenticati e sono sempre disponibili. Questi però presentano anche alcuni difetti:

- Sono molto invasivi e non tutti sono d'accordo sul condividere questo tipo di dati.
- Hanno costi di implementazione molto alti.
- Possono diventare un *single point of failure*, infatti con le password se questa viene dimenticata o altro possiamo cambiarla o resettarla ma questo non avviene per i dati biometrici, se il sensore legge male le impronte allora l'utente rimane bloccato.



- Tutti i sensori implementano dei *threshold* per l'accettazione dei dati e anche funzionalità per riconoscere volti o altro anche se questi sono inclinati o altre variazioni.
- Possono verificarsi *falsi positivi* ovvero quando un utente viene accettato quando non dovrebbe e *falsi negativi* ovvero quando un utente viene rifiutato quando non dovrebbe. Spesso ridurre i falsi positivi porta ad un aumento dei falsi negativi e viceversa.

Un sistema dovrebbe avere un rate di falsi positivi dello 0.001% e un rate di falsi negativi dell'1%. Anche queste piccole percentuali se usate con tantissimi utenti potrebbero però portare a problemi.

I test di screening devono essere in grado di valutare il grado di efficacia dei loro *matching*, devono quindi poter determinare se stiano identificando efficacemente le persone ricercate senza danneggiare coloro che non lo sono. Quando un sistema confronta un dato in suo possesso con un dato che sta misurando sta effettuando quello che chiamiamo *dichotomus test*: o c'è un match o non c'è. Possiamo descrivere questo test usando un Reference Standard, lo standard di riferimento è l'insieme di regole che determina quando un test positivo corrisponde a un risultato positivo. L'obiettivo è evitare due tipi di errori: i falsi positivi e i falsi negativi.

Possiamo misurare il successo dell'analisi usando 4 misure standard: **sensitivity**, **prevalence**, **accuracy** e **specificity**, per vedere come funzionano assegniamo delle variabili ai match in questo modo:

|                  | Is the Person Claimed | Is Not the Person Claimed |
|------------------|-----------------------|---------------------------|
| Test is Positive | True Positive = $a$   | False Positive = $b$      |
| Test is Negative | False Negative = $c$  | True Negative = $d$       |

- Sensitivity: È il grado di sensibilità con il quale il sensore fa match o no. Rappresenta la proporzione di risultati positivi tra tutti i possibili match corretti e si calcola come:

$$\frac{a}{a + c}$$

- Specificity: Misura la proporzione di risultati negativi fra tutte le persone non ricercate in quel momento, si calcola come:

$$\frac{d}{b + d}$$

- Accuracy: Misura la sensibilità con la quale il test identifica correttamente le analisi, si misura come:

$$\frac{a + d}{a + b + c + d}$$

- Prevalence: Misura quanto è comune una certa condizione, si misura come:

$$\frac{a + c}{a + b + c + d}$$

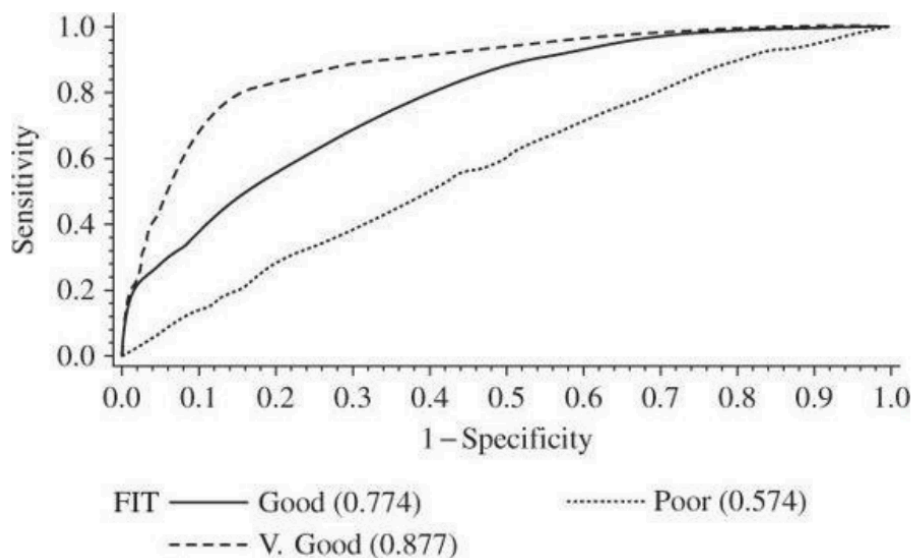
Sensitivity e specificity sono inversamente proporzionali, se una sale allora l'altra scende, per questo infatti non possiamo semplicemente dire che vogliamo rimuovere i falsi positivi, questo porterebbe ad un aumento dei falsi negativi, va trovato il giusto bilanciamento fra i due. Per aiutarci calcoliamo il *positive predictive value* del test, un numero che ci dice quante volte un match positivo rappresenta effettivamente l'autenticazione della persona cercata in quel momento. Questo si calcola come:

$$\frac{a}{a + b}$$

Con lo stesso ragionamento possiamo calcolare il *negative predictive value* come:

$$\frac{d}{c + d}$$

Il **receiver operating characteristic (ROC) curve** è una rappresentazione grafica del trade-off tra falsi negativi e falsi positivi, di base questo mostra il rate di falsi positivi (1 - specificity) sull'asse delle x e il rate dei veri positivi (sensitivity oppure 1 - rate dei falsi negativi) sull'asse y. Vogliamo cercare di rimanere il più a sinistra e il più in alto possibile, questo significa avere tanti veri positivi e un basso rate di falsi positivi. Nel grafico più l'area sotto la curva è grande più avviene quello scritto prima:



L'amministratore di sistema, in base all'utilizzo che verrà fatto di questo test deciderà il giusto compromesso tra i due valori.

### 6.3. Authentication Based on Tokens: Something you have

Si basa appunto su degli oggetti che possediamo, possiamo dividere i token in:

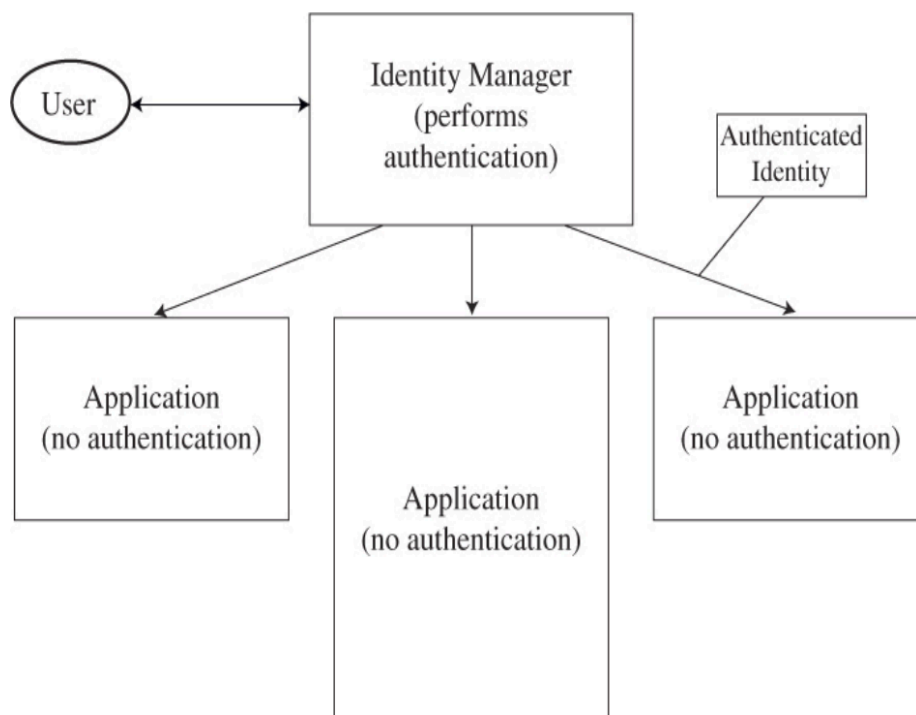
- **Token passivi:** Sono semplici oggetti che non fanno nulla, foto è un esempio di token passivo che appunto contiene un'informazione che non cambia mai.
- **Token attivi:** Possono cambiare o anche interagire con l'ambiente, ad esempio un abbonamento elettronico per il trasporto pubblico che va usato sui tornelli.

Un'altra differenza è:

- **Static token:** I dati che contengono rimangono fissi. Questi sono deboli ad un tipo di attacco chiamato **skimming** ovvero utilizzare un dispositivo per copiare i dati di autenticazione del token ed inviarli all'attaccante.
- **Dynamic Token:** I dati cambiano nel tempo, ad esempio esistono dei dispositivi che generano dei codici a intervalli di tempo e questi codici vanno utilizzati per effettuare il login. Questo tipo di token elimina il problema dello skimming.

## 6.4. Federated Identity Management (FIM)

È l'unione di diversi sistemi di autenticazione e identificazione, invece di mantenere diversi profili utente questi sistemi mantengono un singolo profilo con un solo metodo di autenticazione e i sistemi collegati a questo sistema non richiederanno autenticazione.



Ad esempio lo SPID fa parte di questi sistemi.

Un approccio simile invece è quello del **Single Sign On** questo permette ad un utente di accedere a diversi servizi utilizzando un solo set di credenziali, una volta loggati il sistema si «fida» e possiamo accedere a tutti i servizi collegati. Ad esempio quando facciamo il login con Google / Apple.

Infine troviamo la **multifactori authentication** che combina più autenticazioni prima di dare l'accesso all'utente. Esempio classico è quando facciamo login con email e password ma poi ci viene chiesto un codice inviato sul nostro telefono. Sicuramente avere più fattori di autenticazione aiuta la sicurezza ma più che il numero di questo, quello che protegge è la sicurezza dei sistemi adottati. Possiamo avere anche un'autenticazione a 100 fattori ma se sono tutti uguali e deboli sono più inutili di un singolo fattore più sicuro.

## 6.5. Authentication Security Issues

- **Client Attack:** L'attaccante fa finta di essere un utente autorizzato, cerca di ottenere i dati senza accedere all'host.
- **Host Attack:** È un attacco ai file dell'host dove sono memorizzati i dati di login degli utenti come ad esempio le password.
- **Eavesdropping Attack:** Un attaccante cerca di ottenere la password «studiando» l'utente
- **Trojan Horse Attack:** Un software o un dispositivo mascherato da applicazione autorizzata che in realtà ha lo scopo di rubare dati sensibili come password.
- **Replay Attack:** Un attaccante cattura la richiesta di un utente e la ripete.
- **Denial of Service Attack:** Disabilitare i sistemi andandoli a sovraccaricare di richieste, ad esempio mandare offline il sistema di autenticazione con migliaia di richieste.

## 7. Access Control

Sono le regole che ci permettono di dare determinati poteri agli utenti: *A **subject** is permitted to access an **object** in a particular **mode** and only such authorized accesses are allowed.*

Notiamo quindi che l'access control si basa su 3 elementi principali:

- **Object:** Una risorsa sulla quale dobbiamo fare access control, di solito sono entità che vengono usate per ricevere ed inviare informazioni.
- **Subject:** Un'entità in grado di accedere agli oggetti
- **Access Mode:** Descrive in che modo il subject può accedere ad un oggetto (scrittura, lettura, esecuzione...)

### 7.1. Access Policies (AP)

Guidano l'implementazione dei meccanismi di access control. Quindi si parte da un'idea (l'access policy) su quali accessi dovrebbero essere approvati e in quali circostanze e poi si implementa il meccanismo di access control sul sistema, spesso tramite una tabella.

Proteggere gli objects prevede anche altri obiettivi complementari:

- *Check every access right periodically:* I privilegi degli utenti cambiano nel tempo e in futuro potrebbe essere possibile che ad un utente vadano rimossi dei permessi o aggiunti di nuovi.
- *Enforce least privilege:* Il principio del **least privilege** prevede che un subject abbia accesso al più piccolo numero di object necessari per svolgere i suoi tasks.
- *Verify Acceptable Usage:* Anche se a un utente vengono garantiti dei permessi è importante che utilizzi bene quest'ultimi, ad esempio la navigazione su internet.

Ma implementare una corretta policy non significa aver terminato il lavoro, spesso queste vanno riviste o comunque controllate per vedere se stanno lavorando come previsto, magari qualche utente ha memorizzati tanti accessi a cose di cui ormai non ha più bisogno. Questo si chiama **tracking**.

Un altro concetto importante è la **granularity**, ovvero quanto vogliamo «accurato» il nostro access control. Più è alta la granularity e quindi più è alto il controllo che abbiamo sui file più sarà alto il numero di decisioni da fare e quindi abbasseremo anche le performance. Tipicamente un file, un programma o dei dati sono l'unità più piccola che si considera per l'access control.

Per un amministratore è importante vedere cosa hanno fatto gli utenti, è per questo che si utilizza l'**access log**, chiamato anche **audit log**. Ci sono vari motivi per cui è importante effettuare analisi sui log di questo tipo:

- Aiutano a vedere quali risorse sono utilizzate di più e quindi quali hanno bisogno di upgrade.
- Se il sistema crasha, i record possono mostrare quali accessi erano in corso e aiutare a identificare la causa del crash.
- Se un utente utilizza in modo scorretto delle risorse possiamo vederlo.
- Se subiamo un attacco informatico i log possono aiutarci a vedere da dove gli attaccanti sono entrati nella rete e a quali dati hanno avuto accesso.

Ultimo passo è il **limited privilege** ovvero restringere i poteri di utenti e processi in modo che i danni che possono causare sono limitati. Ovviamente non possiamo limitare al massimo

gli utenti ma nemmeno lasciargli completa libertà, quello che cerchiamo è quindi il giusto compromesso fra i due.

## 7.2. Implementing Access Control

Spesso viene implementato dal Sistema Operativo stesso ad esempio proteggendo i file necessari a lui per funzionare, le funzionalità offerte dal sistema operativo però non bastano quindi l'AC viene implementato o dall'applicazione o dalla rete.

### Reference Monitor

L'Access Control dipende da una combinazione di hardware e software che deve essere:

- Sempre disponibile in modo da poter validare ogni tentativo di accesso.
- Immune alle manomissioni.
- Corretta.

Il Reference Monitor non è uno strumento plug and play ma un concetto e per renderlo efficace dobbiamo convertire in modo giusto le AC policies.

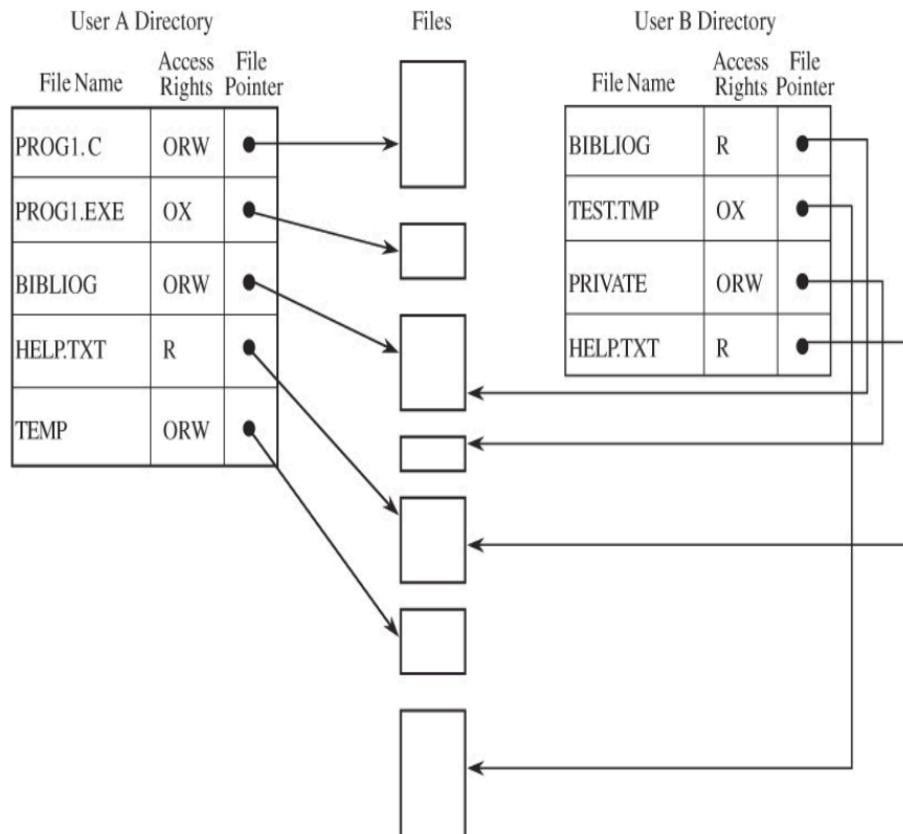
Adesso vediamo diversi modelli per gestire l'AC sfruttando anche il reference monitor.

### 7.2.1. Access Control Directory

Immaginiamo di dover proteggere dei files da degli utenti di un sistema, ogni file ha un proprietario che ha il controllo sul file, ogni utente ha una file directory che elenca tutti i file a cui l'utente ha accesso.

Ovviamente nessun utente ha accesso in scrittura sulla file directory, è quindi compito del sistema operativo quello di gestirle tutte in base ai comandi che riceve dai possessori dei file.

*Esempio*



Un vantaggio di questo sistema è sicuramente la facile implementazione, infatti dobbiamo soltanto fare una lista per ogni utente in base a cosa può accedere. Ci sono anche diversi contro però:

- La lista potrebbe avere dimensioni enormi se abbiamo tantissimi file.
- Revocare gli accessi è costoso se un utente A vuole rimuovere accesso a tutti gli utenti, inoltre se A dà un permesso a B e B lo trasferisce a C allora A non sa niente di questo trasferimento. **propagation of access rights problem**.
- Gli pseudonimi sono un problema. Se A possiede F e anche B possiede un file chiamato F ed entrambi danno accesso a C su questo file allora bisogna tenere traccia di due files diversi per C.
- Rinominare un file è un problema perchè si potrebbero creare inconsistenze nei permessi.

### 7.2.2. Access Control Matrix

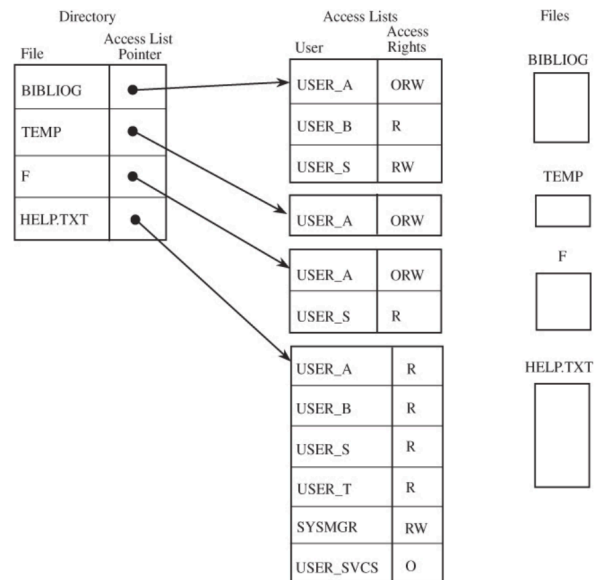
È una tabella dove sulle righe abbiamo gli utenti mentre sulle colonne le risorse a cui devono accedere, all'interno delle celle abbiamo i poteri corrispondenti, ad esempio:

|          | objects |                      |              |
|----------|---------|----------------------|--------------|
|          | File A  | Printer              | System Clock |
| subjects | User W  | Read<br>Write<br>Own | Read         |
|          | Admin   | Write<br>Control     | Control      |

Questa implementazione viene usata raramente dato che cercare i permessi all'interno di questa struttura è spesso inefficiente.

### 7.2.3. Access Control List

Questa rappresentazione corrisponde alle colonne della Control Matrix, c'è una lista per ogni oggetto e questa mostra per tutti gli utenti quali permessi hanno su quel file:



Anche se sembra simile, in realtà è molto diverso dalla directory list, qui infatti abbiamo una access control list per oggetto che porta molti vantaggi. Ad esempio possiamo implementare dei permessi di default all'interno della access control list del file senza andare a creare una entry per ogni utente (ad esempio per file pubblici.)

In questa struttura è inoltre permesso l'utilizzo delle wildcards ("\*") ovvero un «segnaposto» che sta a indicare «tutti gli utenti» o comunque la categoria in cui viene messa, ad esempio in Linux abbiamo la divisioni in permessi per utente-gruppo-world. Quando le liste vengono ordinate prima appariranno tutti i record con dei nomi validi e poi le wildcard, in questo modo non andiamo a perdere i match.

### 7.2.4. Priviledge List

Questa invece è una riga della access matrix che mostra quindi tutti i privilegi di un utente, un vantaggio di questa implementazione è la facilità nel rimuovere i permessi, se un utente viene rimosso dal sistema allora la privilege list che mostra a quali oggetti ha accesso può essere rimossa dall'oggetto.

## 7.3. Capability (non ho capito molto dal libro / slide)

Abbiamo visto soltanto implementazioni che lasciano tutto il lavoro e la responsabilità al sistema, possiamo lasciare del lavoro anche all'utente ad esempio fornendogli un ticket o un pass necessario ad effettuare l'accesso. Definiamo come **capability** un token non falsificabile che fornisce al suo possessore dei permessi su un oggetto. La rappresentiamo come una tripla di soggetto, oggetto e permessi



|        |                      |                  |              |
|--------|----------------------|------------------|--------------|
|        | File A               | Printer          | System Clock |
| User W | Read<br>Write<br>Own | Write            | Read         |
| Admin  |                      | Write<br>Control | Control      |



Come rendiamo il token non falsificabile? 1) Fornirlo direttamente al SO o al meccanismo di AC e non all'utente:

- Il SO mantiene tutti i token per gli utenti
- Il SO ritorna all'utente una sua struttura interna che è collegata all'utente
- Può essere creato solo da una specifica richiesta dall'utente al sistema operativo e ciascuno di questi identifica un accesso.

2) Tramite cifratura con una chiave disponibile soltanto al SO o al sistema di AC.

Tramite la capability introduciamo il trasferimento o la propagazione dei permessi. Un utente con permessi di propagazione può passare copie di capabilities ad altri utenti. Ad esempio l'utente A può passare la copia di una capability a B che poi può passarla a C. B può prevenire ulteriori propagazione omettendo il permesso di propagazione alla copia che sta passando a C.

Come funziona invece per la revoca dei permessi?

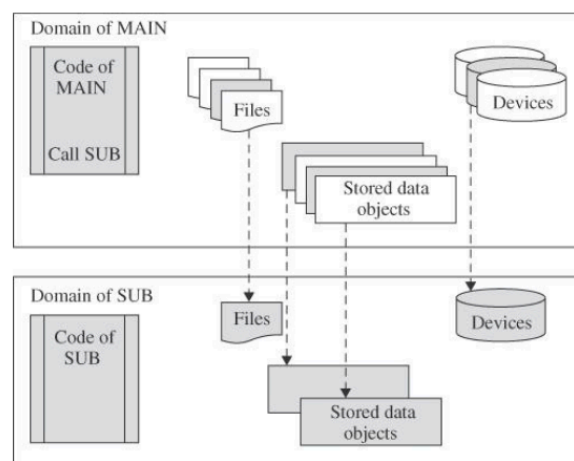
Quando un utente toglie una capability non devono essere permessi ulteriori accessi con quella capability, una tabella dovrebbe contenere puntatori alle capabilities attive nate da questa revocata in modo che il SO può tracciarle e vedere quali accessi eliminare.

## 7.4. Domain

È la collezione di oggetti ai quali i processi hanno accesso:

- La collezione di capabilities degli oggetti nel dominio definiscono il dominio.

I processi però vengono eseguiti per gli utenti e il dominio per un utente include programmi, files, dati e dispositivi di I/O.



Il dominio di una sottoprocedura non è per forza lo stesso di chi l'ha chiamata:

- La chiamante potrebbe passare soltanto alcuni oggetti al dominio della sottoprocedura
- La sottoprocedura potrebbe avere permessi diversi e quindi non poter accedere a tutti gli oggetti della chiamante.
- La chiamante potrebbe passare non tutti i suoi permessi di accesso.

## 7.5. Capabilities at Work

Le capabilities sono un ottimo metodo per tenere traccia degli accessi da parte degli utenti durante l'esecuzione:

- Le capabilities richiedono l'utilizzo delle ACL o ACM
- La capability per un oggetto viene creata a runtime, ogni volta che un processo vuole utilizzare un nuovo oggetto il SO esamina la lista e determina se questo è accessibile, se sì, crea la capability per quell'oggetto.

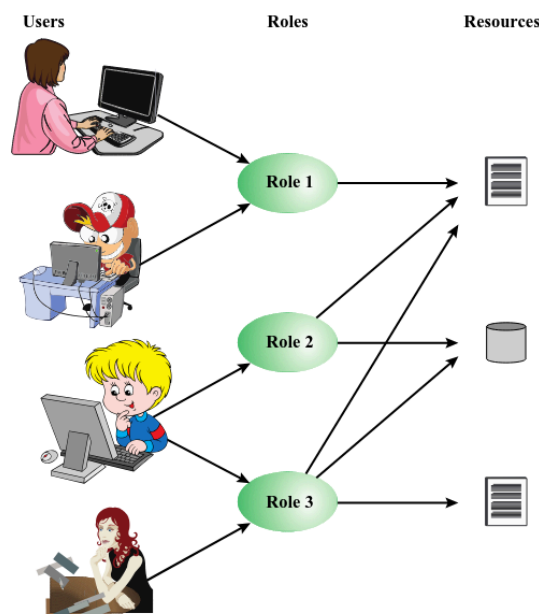
Le capabilities vanno salvate in zone di memoria non accessibili ai normali utenti.

## 7.6. Procedure Oriented Access Control

Le procedure garantiscono che l'accesso agli oggetti siano effettuati tramite un'interfaccia sicura, ad esempio nessun utente e nemmeno il sistema operativo dovrebbero avere accesso diretto alla tabella degli utenti, l'unico accesso garantito dovrebbe essere quello attraverso le tre procedure di: aggiunta utente, cancellazione utente e verifica utente. Ognuna di queste procedure avrà i suoi controlli per verificare che la chiamata sia legittima.

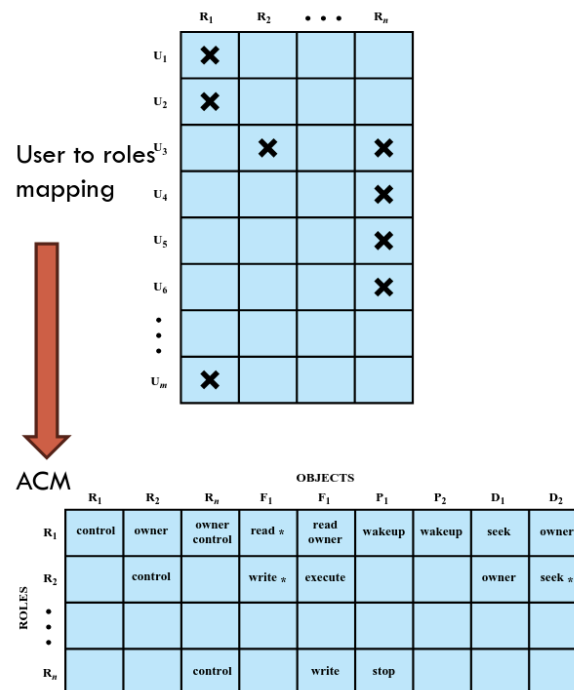
## 7.7. Role-Based Access Control

Invece di basarsi su chi è l'utente, si basa più sul ruolo che copre questo utente, i permessi infatti vengono dati in base al ruolo e non agli utenti specifici.

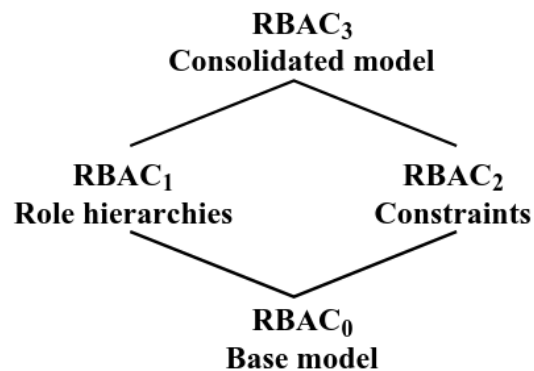


Ogni ruolo dovrebbe contenere il minimo numero di permessi richiesti da quel ruolo (least privilege).

Per implementare questo meccanismo abbiamo quindi bisogno di una tabella che ci dice per ogni utente a che ruolo o ruoli appartiene e poi una ACM che per ogni ruolo indica quali poteri ha:



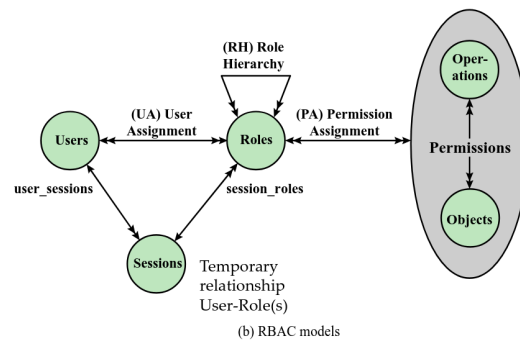
Esistono diversi modelli di Role-Based Access Control, ad esempio quello basato sull'ereditarietà:



(a) Relationship among RBAC models

Questo permette ad un ruolo di ereditare permessi da un altro ruolo.

Oppure ruoli basati sulla sessione, ad utente vengono forniti dei permessi in base a quello che deve fare in quel momento:



## 7.8. Attribute-Based Access Control

Definisce condizioni sia sugli oggetti che sugli utenti, ad esempio prendiamo una configurazione dove ogni risorsa ha un attributo che identifica il soggetto che l'ha creata, in questo modo una singola regola può indicare il permesso di possesso per tutte le risorse rispetto a tutti i possessori. Il punto forte dell'ABAC è infatti la flessibilità.

Ci sono 3 elementi chiave nel modello ABAC:

- Attributi
- Policy Model
- Architecture Model

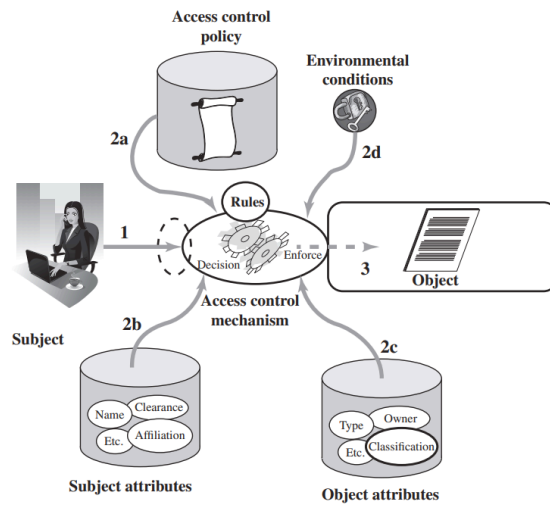
Analizziamoli tutti

### 7.8.1. Attributes

Gli attributi definiscono caratteristiche del soggetto, oggetto o ambiente al quale sono assegnati, esistono infatti diversi tipi:

- **Subject Attributes:** Un soggetto è un'entità attiva, gli attributi in questo caso possono identificare ad esempio il suo ruolo, il dipartimento ecc...
- **Object Attributes:** In questo caso possono indicare ad esempio il tipo di file, il proprietario ecc...
- **Environment Attributes:** Ad esempio dove si trova, il livello di sicurezza della rete ecc...

ABAC verifica l'accesso agli oggetti considerando le regole su tutti gli attributi. Quando un utente prova ad accedere ad una risorsa la richiesta viene analizzata da un meccanismo di access control. Questo meccanismo è guidato da delle regole definite dalle policy e basandosi su queste regole il meccanismo definisce gli attributi del soggetto, della risorsa e dell'ambiente per determinare l'autorizzazione.



## 8. Programs And Programming

Vediamo i problemi di sicurezza legati alla programmazione, per prima cosa diamo delle definizioni:

- **Error:** Un errore umano compiuto da uno sviluppatore
- **Fault:** Un passaggio, un comando, un processo o definizione di dati scorretti all'interno di un programma.
- **Failure:** Quando il comportamento del sistema non rispetta i requisiti richiesti.
- **Flaw:** Indica un'omissione non intenzionale di controlli da effettuare.

I difetti di un programma possono causare un **fallimento di integrità** andando ad intaccare non solo la correttezza ma anche l'accuratezza e la consistenza dei dati.

Un difetto nato per cause innocue come un semplice errore di distrazione può trasformarsi in una vulnerabilità se un malintenzionato scopre come sfruttarlo per manipolare il comportamento del programma. Tra le falle più comuni troviamo: **Buffer Overflow, Incomplete Mediation, TOCTTOU, Backdoor, Race Condition, ecc...**

### 8.1. Buffer Overflow

Si verifica quando un programma tenta di memorizzare dati che eccedono lo spazio pre-allocato per il buffer, nonostante sia un problema noto da molti anni è diffusissimo ancora oggi.

Vediamo come funziona fissando alcuni concetti:

- **Data as Code:** Per il processore, codice e dati non sono altro che sequenze di bit in memoria. Un attaccante sfrutta questo concetto causando un overflow dei dati nello spazio di memoria adiacente andando ad immettere in queste regioni dei dati specifici li renderemo delle istruzioni valide per il processore.
- **Gestione dello Stack:** Quando una funzione ne chiama un'altra tutti i suoi parametri, indirizzi di ritorno e vecchio puntatore dello stack vengono salvati nello stack, se un utente malintenzionato causa un overflow sullo stack può sovrascrivere il vecchio puntatore o l'indirizzo di ritorno, in questo modo a fine funzione il programma non tornerà nel punto di prima ma eseguirà del codice dannoso scelto dall'attaccante.
- **Stack Smashing:** Si utilizza l'**Heap** per fare overflow sullo stack. L'heap è l'area di memoria in cui le variabili vengono create e crescono in modo dinamico, se non controllato l'heap può espandersi in maniera anomala finendo per collidere con lo stack e schiacciarlo.

In che modo possiamo combattere lo stack overflow?

- **Contromisure di base:** Controllare sempre le lunghezze dei dati prima di effettuare scritture, confermare i limiti degli array, usare utilità di gestione stringhe sicure e limitare i privilegi assegnati ai programmi.
- **Scelta del linguaggio:** Linguaggio come C e C++ permettono libero accesso alla memoria mentre linguaggi come Java o Python controllano automaticamente i limiti.
- **Tecnica del Canary:** A livello di Sistema Operativo viene applicato un livello protettivo attorno al frame dello stack inserendo un valore esca chiamato *canary*.

Se avviene un overflow che tenta di arrivare all'indirizzo di ritorno o al *program counter*, inevitabilmente sovrascriverà prima il canary. Il codice di terminazione della procedura controllerà

questo valore e se è stato modificato fermerà l'esecuzione. Questo funziona finché l'attaccante non sa come falsificare anche quel valore.

## 8.2. Incomplete Mediation

La «mediazione» è l'atto di verificare l'autorizzazione di un soggetto a eseguire un'azione, la falla avviene quando questo controllo non è esaustivo. Un esempio è un sito di e-commerce che usa URL in chiaro per i parametri d'acquisto (`&qy=20&price=10`). Il server si affida al browser dell'utente per questi dati quindi se l'utente modifica l'URL mettendo `&price=1` e il backend non convalida questi dati l'attaccante pagherà di meno i prodotti. Per risolvere questo problema dobbiamo sempre convalidare i dati che riceviamo con quelli che abbiamo nel nostro database.

## 8.3. Time-of-Check to Time-of-Use - TOCTTOU

Sfrutta l'intervallo di tempo tra il momento in cui un sistema controlla un'autorizzazione e il momento in cui usa effettivamente il file, è noto anche come **Bait and Switch**. Un esempio non nel mondo dell'informatica è quando un malintenzionato mette sul banco dei soldi per pagare e mentre il cassiere è distratto tu togli delle monete, il cassiere convinto ancora che ci siano i soldi di prima li prende e viene truffato. Come contromisura dobbiamo implementare una **integrità seriale** ovvero senza interruzioni tra il check e l'utilizzo, oppure copiare i dati dell'utente in un'area sicura dell'OS ed eseguire la validazione direttamente sulla copia, in questo modo l'utente non potrà modificarli in corso d'opera.

## 8.4. Undocumented Access Point / Backdoor

Durante lo sviluppo software gli sviluppatori lasciano spesso delle «porte di servizio» o modalità di esecuzione veloci per testare il software che non passano per i controlli di sicurezza. Il problema nasce quando si scordano di rimuoverle nella versione in produzione, questo è molto pericoloso dato che **non sono documentate** e di conseguenza non appaiono in nessun manuale o specifica di sistema e passano inosservate anche a chi esegue revisioni tecniche. Queste possono anche venir lasciate da utenti malintenzionati dopo aver eseguito un attacco e ottenuto accesso alla macchina.

## 8.5. Race Condition

È un difetto di sincronizzazione e si verifica quando l'esito di un programma dipende dalla sequenza temporale con cui vengono eseguiti due o più processi che competono verso la stessa risorsa.

- È un problema difficile da rilevare perché il bug non è costante e si manifesta in base al carico del sistema, alla memoria occupata e alla priorità in quel preciso istante.
- Un esempio classico è il sistema di prenotazione, se non ci sono meccanismi di «lock» due utenti possono vedere lo stesso posto come disponibile in contemporanea e prenotarlo simultaneamente andando a creare conflitti.

Questo causa un **fallimento di integrità**.

## 9. Malware

Esistono diversi tipi di Malware e si differenziano in base a come si propagano.

- **Virus:** È un programma che si replica infettando altri programmi non maligni andando quindi a modificarli inserendo il proprio codice. La sua caratteristica fondamentale è che **non può funzionare né replicarsi senza un programma ospite**. I virus possono essere **transient** ovvero vivono e muoiono con l'esecuzione dell'ospite oppure **resident** ovvero si insediano nella memoria del sistema e rimangono attivi finché il sistema non si spegne.
- **Worm:** È un programma che diffonde copie di sé stesso attraverso la rete. A differenza dei virus non hanno bisogno di ospiti ma sono appunto **standalone**.
- **Trojan Horse:** È un malware che oltre al suo scopo primario nasconde un secondo effetto dannoso. Un esempio pratico è una schermata di login che permette sì il login alla piattaforma ma nel frattempo salva le tue credenziali e le invia ad un attaccante.

Esistono anche altre tipologie che hanno nomi in base all'azione che eseguono:

- **Logic / Time Bomb:** Si attivano quando si verifica una certa condizione logica o a un orario prestabilito.
- **Ransomware:** Cifra i dati della vittima e chiede un riscatto per decifrarli.
- **Spyware:** Intercetta di nascosto le attività dell'utente
- **Rootkit:** Si installa nelle sezioni più privilegiate del sistema operativo ed è molto difficile da rilevare.

Adesso che conosciamo i principali tipi di malware possiamo vedere in che modo agiscono andando ad analizzare **Harm (danno), Trasmissione e Propagazione, Attivazione, Stealth**.

### 9.1. Harm

Il danno definisce come il malware colpisce utenti e sistemi. Possiamo dividerlo in **danni non distruttivi** che hanno come principale obiettivo quello di disturbare e creare panico; e in **danni distruttivi** che corrompono file o forzano l'hardware, di solito con questo tipo di danni si punta al furto di dati sensibili o all'accesso alla macchina per lanciare attacchi DoS. I malware possono diffondersi su milioni di sistemi portando a costi di recupero enormi compresi anche danni all'immagine dell'azienda che ha subito l'attacco.

### 9.2. Trasmissione e Propagazione

Una volta che il malware si trova sul disco, deve essere eseguito per diffondersi, può farlo in diversi modi:

- **Setup e Installer:** Il virus infetta i pacchetti di installazione di altri programmi. Una volta eseguito il pacchetto il malware continua a diffondersi senza bisogno di altri interventi umani.
- **File allegati:** Si possono nascondere malware in file apparentemente innocui. Questi sfruttano i linguaggi di programmazione dei documenti (Macro) per eseguire dei comandi.
- **Autorun:** I malware infettano i file che il sistema operativo avvia automaticamente durante il login.
- **Tecniche di iniezione:** Il virus può accodarsi a un programma originale e modificarne l'output oppure rimpiazzarne intere parti.



### 9.3. Attivazione

Si studia come il codice prende il controllo del sistema, in alcuni casi gli sviluppatori utilizzano delle vulnerabilità chiamate **zero-day** ovvero che non sono state ancora risolte. Il virus prende il controllo della macchina in diversi modi:

- Bypassare il target originale di un programma per eseguire altro codice
- Infettare la sequenza di caricamento iniziale del computer, il **Boot Sector**. In questo modo il virus sarà la prima cosa ad avviarsi sul sistema.
- Il virus si attacca a processi sempre attivi per eseguirsi ripetutamente.

### 9.4. Stealth

In che modo i virus si nascondono all'utente? L'utente come principale strumento utilizza gli **antivirus** ma questi hanno dei limiti infatti cercano le delle «firme» ovvero sequenze precise di bit che identificano un virus noto. Per ingannare gli antivirus i malware utilizzano la **steganografia** nascondendo il proprio codice e non far corrispondere mai la stessa firma.

Oppure esistono altre tecniche:

- **Virus Polimorfici**: Cambiano costantemente forma e riorganizzano il proprio codice per non far corrispondere mai la stessa firma.
- **Virus Cifrati**: Tutto il corpo del virus è cifrato e appare come dati casuali mantenendo in chiaro soltanto la chiave di decifratura.

### 9.5. Contromisure

Una delle principali contromisure è quella di **educare l'utente** ad esempio a non aprire allegati sospetti ed eseguire backup nel tempo, ma questo non basta.

Anche chi crea i sistemi deve eseguire pratiche sicure per prevenire le falle in partenza:

- **Modularità**: Dividere il codice in moduli piccoli, semplici e indipendenti. In questo modo isoliamo i problemi.
- **Information Hiding e Incapsulamento**: I moduli funzionano come «scatole nere» andando quindi a ostacolare gli attaccanti.
- **Mutual Suspicion**: I programmi non devono fidarsi reciprocamente, un modulo deve sempre verificare gli input che riceve da un altro.
- **Genetic Diversity**: Evitare l'eccessiva integrazione di prodotti dalla stessa origine, poiché un bug in un modulo rischia di far crollare tutto il resto.

Inoltre è importante rispettare i **Principi di Design della Sicurezza (Saltzer & Schroeder)**:

- **Privilegio Minimo**: Dare a processi e utenti solo i poteri strettamente necessari.
- **Design Aperto**: La sicurezza non deve basarsi sul mantenere segreto il codice ma resistere anche se l'attaccante ne conosce la struttura.
- **Mediazione Completa**: Ogni singolo tentativo di accesso deve essere verificato.
- **Basato sui Permessi**: Di default l'accesso deve essere negato, concedendolo solo esplicitamente.

### 9.6. Tecniche di Testing e Validazione

Infine come contromisure abbiamo le tecniche di testing. Il collaudo si divide in test **Black-Box** dove si testano input e output senza vedere il codice e **Clear-Box** dove invece si ha

accesso al design interno. Passare un test non dimostra l'assenza di guasti ma soltanto che quel guasto specifico non è occorso.

Esistono una tipologia di test chiamati **Penetration Testing** dove team di esperti **autorizzati** tentano di craccare il sistema per trovare delle vulnerabilità.

Delle pratiche da evitare durante questa fase sono 2:

- **Penetrate-and-patch**: Cercare vulnerabilità per poi sistemarle con delle «toppe». Spesso queste patch veloci non fanno altro che introdurre nuovi bug.
- **Security by Obscurity**: Credere che un sistema sia sicuro perché il suo funzionamento è tenuto nascosto al pubblico.

## 10. Internet Security

In questo capitolo ci si concentra sui pericoli con cui può scontrarsi un utente su internet. Iniziamo vedendo le criticità dei browsers ovvero i software che permettono agli utenti di collegarsi ad internet, di base questi devono:

- Collegarsi a degli indirizzi web
- Ricevere i contenuti da mostrare
- Trasmettere eventuali dati dell'utente all'indirizzo

Già da questi tre punti possiamo osservare diverse possibili criticità:

- Un browser spesso si connette anche ad altri indirizzi oltre a quello indicato nella barra di navigazione
- Il software del browser può essere corrotto per eseguire azioni malevole
- Alcuni browser permettono l'installazione di estensioni di cui non possiamo fidarci al 100%.
- Altro...

Con altri tipi di programmi è possibile limitare i file a cui hanno accesso o togliere l'accesso ad internet esterno, queste limitazioni non sono applicabili ad un browser e, anche per questo, è uno dei principali vettori di attacco dove vengono anche scoperte molte vulnerabilità.

### 10.1. Browser Attacks

Ci sono 3 principali modi per attaccare un browser:

- Modificare il sistema operativo in modo da compromettere il corretto funzionamento del browser.
- Modificare il browser o uno dei suoi componenti per alterarne il comportamento.
- Intercettare e/o modificare le comunicazioni da e verso il browser.

#### 10.1.1. Man-in-the-Browser

In questo tipo di attacco si utilizza del codice per infettare il browser, questo codice può leggere, copiare e condividere tutto quello che l'utente inserisce nel browser. Il pericolo qui è che l'attaccante può recuperare le credenziali dell'utente, dati bancari o altre informazioni sensibili. Uno dei più famosi è stato il **Silent Banker** nel 2008 dove questo trojan era presente in alcuni browser come «componente aggiuntivo» e quando rilevava che il browser era collegato a determinati siti bancari iniziava a registrare tutti gli input dell'utente e inviarli all'attaccante. Nemmeno la crittografia poteva proteggere i dati perché questi venivano intercettati prima di essere crittografati. Inoltre, SilentBanker oltre a rubare i dati modificava anche le azioni compiute dall'utente, se ad esempio veniva effettuato un bonifico alla banca A allora il trojan modificava l'operazione per effettuare un bonifico alla banca B, dove magari era ospitato un conto dell'attaccante.

#### 10.1.2. Keystroke Logger

Ha una funzione simile all'attacco di prima infatti non registra soltanto i tasti inseriti nel browser ma in tutto il sistema, per farlo però c'è bisogno di un dispositivo USB esterno inserito nel PC.

### 10.1.3. Page-in-the-Middle

In questo caso non viene alterato il browser ma la pagina web, questa potrebbe presentare ad esempio delle finte schermate di login per inviare i dati agli attaccanti.

### 10.1.4. Program Download Substitution

Si effettua insieme al page in the middle, infatti viene sostituito il link di download di un software con del software malevolo.

### 10.1.5. User-in-the-Middle

Molti siti si difendono dai bot utilizzando i CAPTCHA, dei piccoli test che i bot non sono in grado di risolvere. Degli attaccanti magari vogliono un «esercito» di bot per acquistare biglietti o altro ma non possono mandare liberamente i loro bot per via dei captcha, allora creano dei siti che non riguardano l'attacco vero e proprio, magari per distribuire software o risorse gratuite e i bot, in automatico, inoltrano le foto dei captcha su questi siti per farle risolvere agli utenti ed inoltrare le risposte ai siti originali. In questo modo l'utente umano crede di aver risolto il captcha per scaricare il software ma in realtà ha aiutato l'attaccante. Per risolvere questi problemi i browser si sono spostati su tecnologie più nuove come i CAPTCHAv3 che non osservano più la risposta ma come l'utente si comporta.

## 10.2. How Browser Attacks Succeed

La falla principale degli attacchi visti sopra è l'autenticazione non corretta, se  $A$  non può assicurarsi che il mittente del messaggio sia veramente  $B$  allora  $A$  non può fidarsi dell'autenticità di nulla all'interno del messaggio.

Ci sono diversi punti possono portare ad una mancata autenticazione:

- **Usability and Accuracy** possono andare in conflitto, un sistema più usabile potrebbe essere meno preciso ma gli utenti vogliono un sistema semplice.
- **Computer-to-computer interaction allows limited bases for authentication.** Computer Authentication è basata su qualcosa che il computer «conosce», questo significa che è memorizzata in qualche dato computabile, questi dati possono essere trovati da processi non autorizzati e calcolati anche da altri computer.
- **Malicious software can undermine authentication by eavesdropping** authentication data.

### 10.2.1. Successful Identification and Authentication

Ci sono, però, diversi modi per garantire un'identificazione ed un'autenticazione avvenute con successo.

**Shared Secret** - È un qualcosa che soltanto le due entità che si scambiano i messaggi dovrebbero conoscere, nessun altro che si intromette nella comunicazione deve saperlo.

**One-Time Password** - Sono dei codici validi una sola volta e che durano per un determinato periodo di tempo.

**Out-of-Band Communication** - Si tratta dell'autenticazione a due fattori (2FA) ovvero spezzare l'autenticazione in più mezzi diversi.

## 10.3. Web Attacks Targeting Users

*(Gran parte di questi stanno descritti già prima non capisco come è organizzato sto libro di merda :D)* La prima categoria di attacchi riguarda i contenuti falsi mentre la seconda vuole danneggiare l'utente.

### 10.3.1. False or Misleading Content

L'attacco più semplice è il **Defaced Web Site** ovvero quando un attaccante rimpiazza o modifica il contenuto di una pagina web legittima. Spesso l'obiettivo di questi attacchi è soltanto quello di «ridicolizzare» la vittima o mostrare le loro debolezze in campo di sicurezza, ad esempio mettendo messaggi di prese in giro nella pagina per mostrare che si è riusciti a bucare il sito.

### 10.3.2. Fake Web Site

Uguale a prima praticamente (?)

### 10.3.3. Fake Code

Si tratta di creare e distribuire software malevolo ma che compie anche le azioni che si aspetta l'utente.

#### 10.3.3.1. Protecting Web Sites Against Change

Esistono diversi modi, oltre alla crittografia che qui non è adatta, per proteggere i siti web.

##### 10.3.3.1.1. Integrity Checksums

È una funzione matematica che riduce un blocco di dati in un piccolo numero di bits. Cambiare, anche di poco, il blocco in input fa variare di molto il risultato rendendo praticamente impossibile cambiare i dati in modo che il risultato sia uguale a quello dei dati originali. Utilizzando un checksum ci assicuriamo che i cambiamenti sul sito cambino anche il valore del checksum.

##### 10.3.3.1.2. Signed Code or Data

Utilizzare un integrity checker aiuta l'amministratore del sito a verificare che i dati sono intatti, inoltre firmare il codice con un certificato da una CA garantisce integrità e autenticazione della fonte. Bisogna fare attenzione però perché anche un'azienda malintenzionata può ricevere un certificato valido e distribuire quindi codice.

### 10.3.4. Web Bug

Si tratta di sfruttare un'immagine minuscola, spesso una GIF 1x1 trasparente invisibile all'utente per tracciare la sua navigazione e inviare i dati all'attaccante, nello specifico l'attaccante può sapere, ad esempio, se hai aperto la mail contenente questa foto e sapere quindi che la tua casella di posta è attiva.

## 10.4. Furto di Dati: Scripting e SQL Injection

Questi attacchi colpiscono la logica delle applicazioni web sfruttando le vulnerabilità nella validazione dell'input dell'utente.

- **Cross-Site Scripting (XSS):** Sfruttano il fatto che le comunicazioni HTTP trasmettono in chiaro script incorporati, in questo modo costringono il browser o il server ad eseguire codice malevolo mascherato da contenuto legittimo. Un esempio è l'inserimento di uno script malevolo nei commenti di un blog, chiunque carichi quella pagina eseguirà lo script senza accorgersene.
- **SQL Injection (SQLi):** Sfrutta la scarsa sanitizzazione degli input per inviare comandi SQL direttamente al database backend. L'attacco interrompe prematuramente la stringa attesa dal database utilizzando il singolo apice ' e aggiunge un nuovo comando, inserisce poi alla fine il simbolo del commento per omettere tutto il resto del codice legittimo.
  - **Inband Attacks:** Sfruttano lo stesso canale di comunicazione per iniettare l'attacco e ricevere i risultati, mostrandoli direttamente sulla pagina web.

Questi comprendono tecniche di **tautology** ovvero che inseriscono una condizione sempre vera come `OR 1=1` per aggirare l'autenticazione e le **Query Piggybacked** ovvero accodare una query distruttiva come `DROP table` a una query legittima.

- **Inferential Attacks (Blind SQLi):** Non c'è alcun trasferimento visibile di dati, l'attaccante inietta query specifiche e ricostruisce la struttura e il contenuto del database osservando i comportamenti del sistema o gli errori logici raccogliendo informazioni per futuri attacchi.
- **Out-of-Band Attacks:** Utilizzati quando le informazioni non possono tornare indietro via HTTP ma il database possiede connettività in uscita vulnerabile per inviare i dati tramite altri canali.

La **contromisura principale** è la **sanitizzazione dell'input** che rimuove o codifica caratteri pericolosi prima di elaborarli, e il rigoroso controllo degli accessi sul server backend per limitare l'estrazione dei dati.

## 10.5. Sicurezza delle Applicazioni Mobile (App)

A differenza dei siti web, le app risiedono fisicamente sul dispositivo ed interagiscono in profondità con hardware e software, inoltre **funzionano continuamente in background** e spesso raccolgono e trasmettono tanti dati senza dirlo all'utente.

Le vulnerabilità delle app derivano dal fatto che ci sono tantissimi utenti da sfruttare e l'architettura stessa di queste, l'utente infatti non sa cosa fanno realmente e possono ottenere privilegi enormi agendo come da *spyware* esportando tantissimi dati non necessari al loro funzionamento. Tra i problemi più comuni troviamo:

- **Insecure Direct Object References:** Le app non dovrebbero esporre la struttura interna delle risorse ma agire da intermediario traducendo i comandi dell'utente.
- **Mancanza di Logging e Monitoring:** L'app è l'unico ponte tra utente e risorse, se fallisce nel registrare e generare allarmi per attività sospette, blocca l'indagine sulla compromissione dei dati.
- **Problemi di Crittografia:** Un'errata gestione o memorizzazione delle chiavi crittografiche o il mancato utilizzo di funzioni di hashing per verificare l'integrità dei dati.
- **Componenti vulnerabili o non aggiornati:** Molte app si basano su librerie di terze parti, se una di queste è vulnerabile l'app diventa vulnerabile, o almeno finché lo sviluppatore non la aggiorna.

- **Errori di configurazione:** Mantenere la password predefinita o esporre informazioni involontariamente.

È necessario quindi che gli sviluppatori implementino rigidi controlli automatizzati e utilizzino sistemi per l'analisi statica e strumenti di *Interactive Application Security Testing (IAST)* per individuare i difetti durante l'esecuzione.

## 10.6. Attacchi tramite Email e Messaggi

L'indirizzo email del mittente è facilmente falsificabile e gli attaccanti sanno che le persone riconoscono le mail false ma puntano sui grandi numeri, inviando milioni di mail basta una minima percentuale di vittime per generare profitti.

È importante utilizzare **Crittografia PGP (Pretty Good Privacy)** per garantire confidenzialità, integrità e autenticazione nelle comunicazioni.

- **Confidenzialità:** Il sistema genera una chiave di sessione casuale simmetrica per cifrare il corpo del messaggio, poi la chiave di sessione viene cifrata utilizzando la chiave pubblica del destinatario e viene allegata al messaggio.
- **Integrità e Autenticazione:** Viene creato un hash del messaggio che viene firmato cifrandolo con la chiave privata del mittente.
- **Distribuzione delle chiavi:** Si basa su certificati convalidati gerarchicamente (formato X.509) permettendo al mittente e al destinatario di scambiarsi informazioni in sicurezza senza doversi scambiare materialmente le chiavi in anticipo, purché condividano un'autorità di certificazione (CA) fidata.

## 11. OS Security

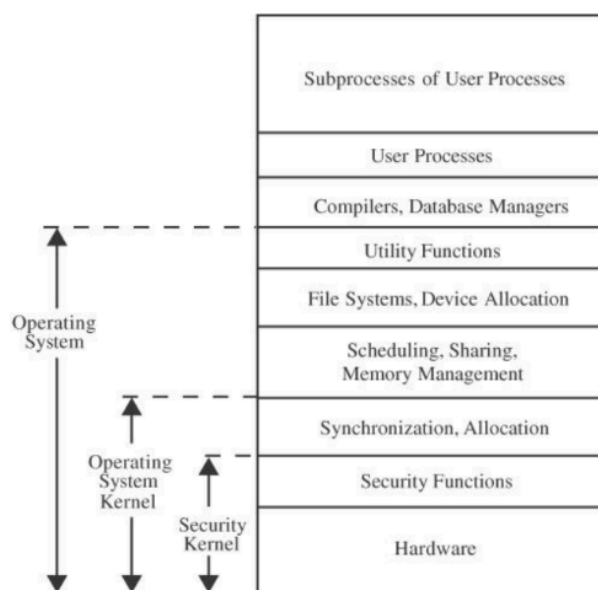
Il sistema operativo è la prima linea di difesa in un computer questo infatti serve a:

- Proteggere gli utenti l'uno dall'altro
- Proteggere le aree di memoria da processi non autorizzati
- Esegue identificazione e autenticazione delle operazioni
- Garantisce una condivisione equa delle risorse hardware

Proprio perché è la prima linea di difesa però, è anche uno dei principali bersagli degli attacchi. Ci sono diverse funzionalità dei sistemi operativi che coinvolgono la sicurezza informatica:

- **Enforced Sharing:** Garantisce l'integrità e la coerenza quando le risorse vengono rese disponibili agli utenti.
- **Comunicazione Interprocesso e sincronizzazione:** Il Sistema Operativo fa da ponte per le richieste asincrone tra i processi mediando la comunicazione tramite tabelle di controllo degli accessi.
- **Protezione dei dati critici del sistema:** Se i dati attraverso cui l'OS fa rispettare la sicurezza non sono protetti allora crolla l'intero sistema.
- **Garantire un servizio equo:** Tramite algoritmi di scheduling si assicura un controllo che previene la «starvation» dei processi
- **Interfacciamento protetto all'hardware:** Controlla l'accesso ai dispositivi I/O e al file system.
- **Autenticazione Utente**
- **Protezione della memoria e autenticazione:** Mantiene separati lo spazio utente e quello di sistema per garantire la confidenzialità e l'integrità.

Gli oggetti che l'OS deve proteggere includono la memoria, reti, i dati, file condivisibili e l'hardware. L'architettura tipica di protezione si basa su una **visione a strati (layered view)** dove l'hardware fisico è avvolto da un livello di sicurezza detto **security kernel** circondato a sua volta dal kernel generico dell'OS e infine dai servizi e processi utente.



Il Sistema Operativo, data la sua natura multicomponente, deve integrare moduli di diversa provenienza per proteggersi e non sempre questi sono sicuri, questo significa che l'OS deve



protegersi sia da programmi utente che da moduli o add-on difettosi. Se l'OS é scritto per un hardware specifico allora la protezione potrebbe essere implementata direttamente su quest'ultimo, cosa invece non possibile se ;“OS deve girare su hardware generico.

Per fare questo l'OS utilizza dei meccanismi software:

- **ACL, Access Control List**
- **Events Logging:** L'OS registra numerosi eventi in file di log che sono essenziali per mostrare cosa é successo durante un attacco informatico e pianificare le difese future.
- **Virtualizzazione:** Permette all'OS di presentare a ogni utente solo le risorse pertinenti tramite Macchine Virtuali. L'**hypervisor** agisce da filtro eseguendo le istruzioni privilegiate e l'accesso alla memoria in modo controllato.
- **Sandbox:** Sono degli ambienti isolati dove i processi non fidati hanno un impatto inferiore sulle risorse esterne.
- **Honeypot:** Sono delle vere e proprie esche che si offrono agli attaccanti per monitorarli e studiarli in tempo reale, spesso vengono utilizzate nel networking ma é possibile anche negli OS tramite VM.

## 11.1. Separation and Sharing

L'OS gestisce la concorrenza tra utenti attraverso i due concetti di **separazione e condivisione**. La separazione puó avvenire in 4 modi:

- Fisica: Utilizzando dispositivi distinti
- Temporale: Eseguendo processi a rischio in momenti differenti
- Logica: Confinando gli accessi in modo che ogni processo abbia l'illusione di operare da solo
- Crittografica: Cifrando i dati per renderli illegibili da altri processi.

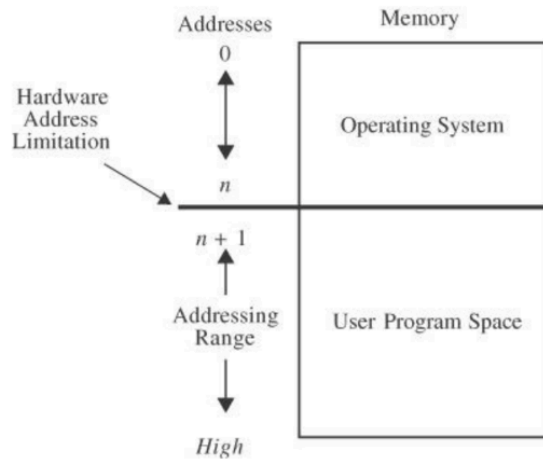
La separazione però é solo una faccia della medaglia, vogliamo anche poter condividere alcuni oggetti fra gli utenti. In scenari complessi si utilizza una condivisione che limita gli accessi o che limita l'uso di un oggetto

## 11.2. Hardware Protection of Memory

Per proteggere la memoria si utilizzano meccanismi hardware specifici:

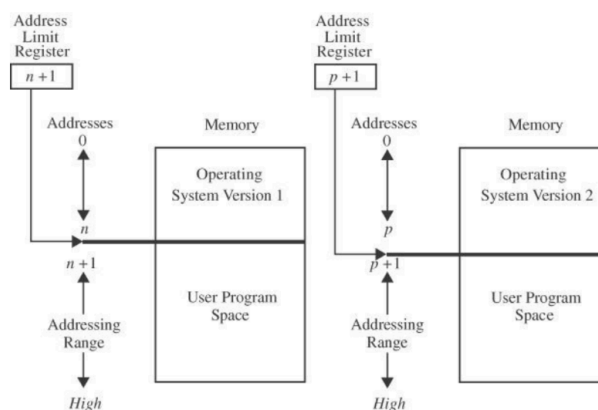
- **Fence:** Impongono limiti hardware statici o dinamici (tramite i Fence Register) per definire dove finisce l'OS e dove inizia lo spazio utente.

Per quelli statici abbiamo una situazione simili a questa:



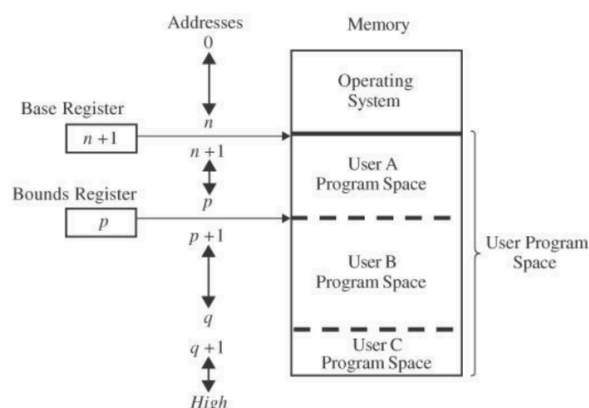
Ovvero impongono una **limitazione hardware fissa**, quindi il confine tra memoria del sistema e lo spazio dei programmi utente è stabilito a un indirizzo di memoria specifico e immutabile.

Nei Fence Register dinamici abbiamo questa situazione:



Utilizza uno specifico registro hardware chiamato **Address Limit Register** che memorizza l'indirizzo di confine, questo indirizzo però può cambiare permettendo al limite tra lo spazio di sistema e quello utente di variare nel tempo.

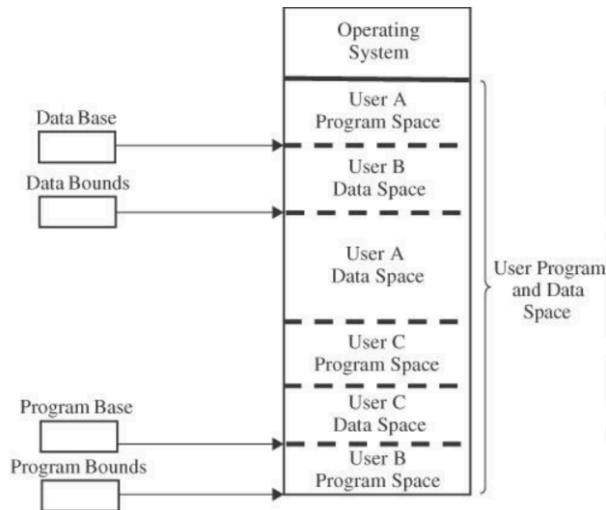
- **Base/Bound registers:** Sono coppie di registri utilizzati per tracciare l'indirizzo di base e il limite superiore consentito.



Questa foto illustra il meccanismo base ovvero:

- Il *Base Register* memorizza l'indirizzo di memoria da cui inizia lo spazio assegnato a uno specifico programma utente.
- Il *Bounds Register* memorizza l'indirizzo che stabilisce il limite superiore di tale spazio. Ogni volta che il programma tenta di accedere alla memoria, l'hardware verifica che l'indirizzo richiesto tra il valore base e il valore bound.

Poi abbiamo un altro meccanismo che usa questi registri:



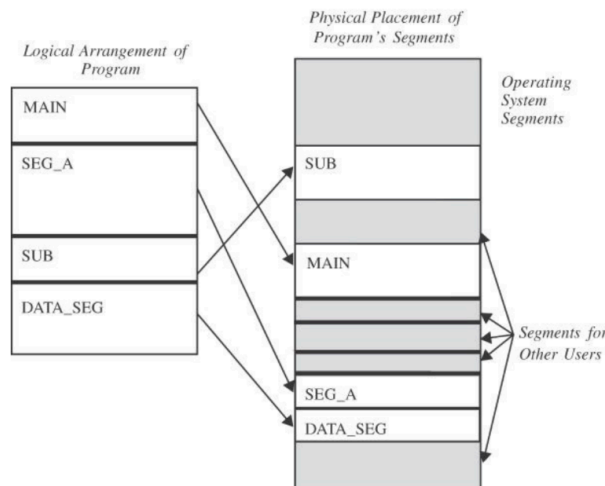
In questo caso lo spazio utente non viene più trattato come un blocco unico ma viene diviso e protetto da due coppie distinte di registri:

- *Program Base e Program Bounds*: Delimitano unicamente l'area di memoria in cui risiede il codice eseguibile.
- *Data Base e Data Bounds*: Delimitano un'area separata in cui risiedono i dati (le variabili dei programmi).

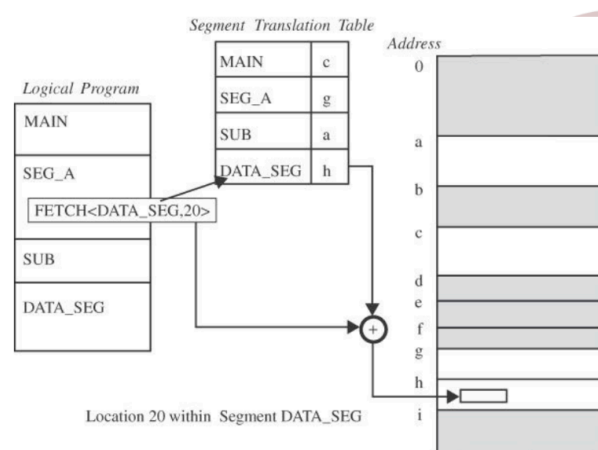
Questo meccanismo permette al SO di trattare le aree di memoria in modo indipendente garantendo una sicurezza maggiore, ad esempio gli permette di imporre regole più precise.

- **Tagged Architecture**: Associa a ogni singola parola di memoria una etichetta di permessi esplicita.

Questo concetto ci porta alla **Virtual Memory** e alla **Segmentazione**, vediamo cosa succede ad un programma utente:



Il programma «logicamente» é un insieme di blocchi contigui ma nella memoria fisica questi vengono posizionati in aree diverse. Questo approccio permette di dividere il programma in porzioni aventi diritti di accesso differenti l'una dall'altra. Alla base della segmentazione c'è la «traduzione degli indirizzi», quando il programma in esecuzione fa una richiesta l'indirizzo fisico reale in cui si trova quel dato é nascosto al programma e l'operazione passa per la **Segmentation Translation Table**:



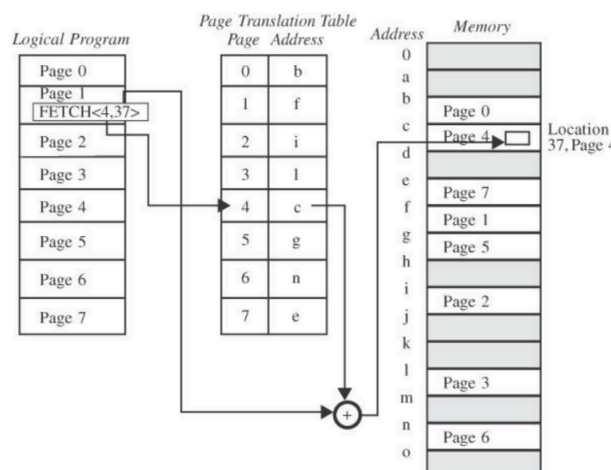
La tabella fornisce l'indirizzo di partenza del dato, nella foto viene richiesta la ventesima parola del segmento **DATA\_SEG**, la tabella sa che il segmento inizia nella cella **h**, una volta posizionatosi all'inizio della cella somma l'offset richiesto, in questo caso 20.

Il fatto che ogni indirizzo passi attraverso il SO prima di toccare la memoria comporta molti benefici alla sicurezza:

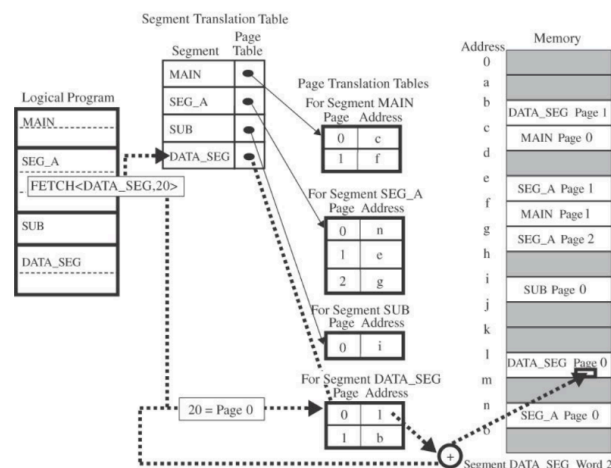
- Ogni riferimento in memoria é controllato per assicurarsi che non superi la dimensione del segmento stesso.
- Dati diversi all'interno della stessa applicazione possono ricevere diversi livelli di protezione
- Due o più utenti possono condividere l'accesso a un determinato segmento pur avendo diritti di accesso diversi
- L'utente non può in alcun modo generare l'accesso verso un segmento che non é autorizzato a vedere.
- Il SO può spostare i segmenti a piacimento o rimuoverli dalla memoria se non sono in uso.

Un grande difetto però risiede nella **dimensione del segmento**, essendo i segmenti di dimensione variabile il sistema è costretto a eseguire complessi controlli per verificare continuamente che l'indirizzo generato non vada oltre la fine del segmento correntemente in uso. Per risolvere questo problema introduciamo il concetto di **paging**.

Con questa tecnica il programma logico non viene più diviso in segmenti variabili ma in *pagine* tutte uguali, anche la memoria fisica viene divisa in *page frames* della stessa misura delle pagine. Il processo di traduzione è simile ma vengono mappate pagine logiche su pagine fisiche, questo approccio offre gli stessi vantaggi visti nella segmentazione ma con una gestione della memoria più efficiente.



In realtà però si utilizza un meccanismo più robusto chiamato **Paged Segmentation** unendo i due concetti ovvero concatenando le due tabelle:



Quando chiediamo l'indirizzo `DATA_SEG, 20` la prima tabella non punta più direttamente alla memoria ma ad una seconda tabella dedicata unicamente al segmento `DATA_SEG`. Il sistema poi converte l'offset in un numero di pagina in base a quanto è grande una pagina, in questo caso la posizione 20 si trova all'interno della pagina 0. A questo punto possiamo vedere dove si trova la pagina 0 del segmento dati nella memoria fisica, in questo caso all'indirizzo `1` e poi sommarci l'offset residuo.

### 11.3. Security in the Design of Operating System

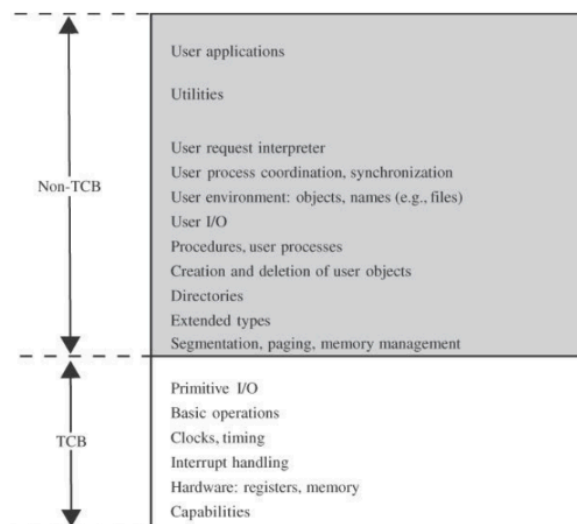
La sicurezza nei Sistemi Operativi va implementata rispettando dei principi:

- Semplicità
- Design a layer
- **Least Privilege**
- Reference Monitor per la mediazione delle richieste.

Un Sistema Operativo progettato in modo sicuro é definito **Trusted System**, questi sono capaci di garantire:

- **Correttezza Funzionale:** Fa quello per cui é stato programmato.
- **Rispetta l'integrità:** É impossibile eludere i privilegi.

A questo scopo si crea la **Trusted Computing Base (TCB)** ovvero l'insieme di tutti i meccanismi, hardware e processi primitivi racchiusi in un guscio protetto il cui unico scopo é far rispettare la policy di sicurezza. La TCB deve ispezionare l'attivazione dei processi, i cambi di dominio, la protezione della memoria e l'I/O. Inserire troppe funzionalità nella TCB però distruggerebbe la modularità del SO e ostacolerebbe la capacità di analizzarne il codice, l'approccio ideale é quello di sviluppare prima un piccolo **Security Kernel** interfacciato con l'hardware e poi costruirci intorno il resto delle funzionalità.



## 12. Network Security

### 12.1. Interception

Le comunicazioni di rete possono essere intercettate in vari modi a seconda del mezzo trasmissivo utilizzato.

- **Sniffing:** Viene utilizzato con i cavi di rame e consiste nell'installazione di dispositivi appositi, *pachet sniffer*, per catturare i pacchetti su una LAN. Questi dispositivi si inseriscono nella rete attraverso il **Cable Splicing** ovvero il taglio fisico del cavo per inserire lo sniffer oppure l'uso delle radiazioni elettromagnetiche, **radiation**, emesse dal cavo per intercettare il segnale. I cavi in fibra ottica non emettono radiazioni elettromagnetiche e inoltre sono molto difficili da tagliare per inserire dispositivi esterni, questo li rende praticamente immuni agli attacchi precedenti.
- **Microonde:** Questi segnali non sono confinati, li rende facilissimi da intercettare utilizzando dei dispositivi nel raggio di azioni. Per le comunicazioni satellitari (LEO, MEO, GEO) è ancora più semplice visto il loro enorme campo d'azione sulla terra (footprint).

### 12.2. Network Security Attacks

Le minacce si dividono in 3 categorie principali:

- **Intercettazione:** Perdita di confidenzialità (eavesdropping)
- **Modifica:** Perdita di integrità
- **Denial of Service**

#### 12.2.1. Intercettazione

Per quanto riguarda l'intercettazione le principali contromisure sono:

- Cifratura
- Sicurezza Fisica, ovvero la protezione delle linee di comunicazione
- Linee dedicate
- Controlled Routing, assicurarsi che la comunicazione passi solo in determinati nodi

Quali sono invece i motivi che espongono una rete maggiormente a questo pericolo?

- **Anonymity:** A differenza di intrusioni fisiche, in una rete un attaccante può lanciare un attacco da chilometri di distanza senza mai entrare in contatto con il sistema permettendo quindi a chi intercetta i dati di nascondere la propria identità.
- **Più punti di accesso e condivisione risorse:** Il principale obiettivo delle reti è quello di condividere le risorse tra tantissimi dispositivi e ogni dispositivo in una rete rappresenta un potenziale punto di accesso.
- **Complessità del sistema:** Visto che in una rete possono esserci moltissimi dispositivi, tutti completamente diversi fra loro sia in hardware che in software, è molto difficile garantire sicurezza.
- **Più percorsi possibili e sconosciuti:** Quando inviamo un messaggio non abbiamo alcun controllo dell'instradamento di questo, anche se sappiamo che un host è compromesso non abbiamo modo di evitare che il nostro pacchetto passi per quel nodo.

Per tutti questi motivi la cifratura rimane il metodo principale di sicurezza nelle trasmissioni.

### 12.2.2. Data Corruption

Avviene quando una comunicazione viene modificata durante la trasmissione, può avvenire in più modi:

- Modificare i dati mentre sono sul percorso (Modification Attack)
- Creare nuove comunicazioni (Insertion Attack)
- Ripetere una vecchia comunicazione (Replay Attack)

I due approcci principali ai modification Attacks sono:

- **Sequencing:** Avviene quando un attaccante riesce a far arrivare un frammento di dati più recente prima di un frammento precedente sfasando l'ordine della comunicazione. Anche se il protocollo TCP è progettato per garantire il corretto ordine, le applicazioni non sempre lo sono aprendo la strada a vulnerabilità.
- **Substitution:** Consiste nel prendere un pezzo di comunicazione e inserirlo all'interno di un'altra. È più semplice da effettuare quando le comunicazioni hanno una formattazione ovvero una struttura rigida.

Per quanto riguarda i reply attack invece, avvengono quando una comunicazione legittima, di solito anche senza nessuna modifica, viene riutilizzata da un attaccante anche senza rompere la cifratura. Le principali contromisure in questo caso sono:

- Utilizzo di numeri di sequenza in modo che il server accetti soltanto il messaggio più recente

### 12.2.3. Denial of Service

Per interruzione del servizio non intendiamo necessariamente un blocco totale ma anche parziale di solo alcune funzioni. I principali motivi dei blocchi sono:

- **Hardware Failures:** Qui la difesa principale è la ridondanza dei dispositivi, se uno fallisce deve esserci sempre un backup pronto.
- **Routing:** I protocolli di instradamento servono a garantire un uso efficiente della rete, se queste regole di routing sono configurate in modo errato o manipolate il traffico non raggiunge più la destinazione.
- **Excessive Demand:** Sono i classici attacchi DoS, il traffico totale in entrate non è gestibile dal server.
- **Guasti introdotti da malware:** Alcuni software possono colpire componenti specifici mandando in crash applicazioni o esaurendo le risorse del sistema.

## 12.3. Port Scanning

Serve ad effettuare una prima «perlustrazione» su un indirizzo IP per vedere quali porte rispondono a delle query per programmare poi i successivi attacchi. Per fare questo si utilizzano i **Port Scanner** come *Nmap*, questo ci fornisce un output simile a questo:



```

Nmap scan report
192.168.1.1 / somehost.com (online) ping results
address: 192.168.1.1 (ipv4)
hostnames: somehost.com (user)
The 83 ports scanned but not shown below are in state: closed
Port      State    Service Reason      Product  Version  Extra info
21        tcp     open    ftp        syn-ack   ProFTPD  1.3.1
22        tcp     filtered ssh        no-response
25        tcp     filtered smtp       no-response
80        tcp     open    http       syn-ack   Apache   8.2.3    (CentOS)
106       tcp     open    pop3pw     syn-ack   poppassd
110       tcp     open    pop3       syn-ack   pop3d
111       tcp     filtered rpcbind   no-response
113       tcp     filtered auth      no-response
143       tcp     open    imap       syn-ack   imapd    17.3
443       tcp     open    http       syn-ack   Apache   8.2.3    (CentOS)
465       tcp     open    unknown   syn-ack
646       tcp     filtered ldap      no-response
993       tcp     open    imap       syn-ack   imapd    17.3
995       tcp     open    unknown   syn-ack
2049      tcp     filtered nfs       no-response
3306      tcp     open    mysql      syn-ack   MySQL    5.0.45
8443      tcp     open    unknown   syn-ack
34 sec. scanned
1 host(s) scanned
1 host(s) online
0 host(s) offline

```

Dove un attaccante può capire: porte aperte, sistema operativo utilizzato, applicazioni esposte con le relative versioni. È inoltre possibile effettuare dei ping sui dispositivi della rete e tramite la latenza capire se si trovano nello stesso segmento o no, una latenza simile significa appunto che i dispositivi molto probabilmente si trovano nello stesso segmento di rete.

I port scanners sono un pericolo quindi?

- In realtà no dato che le porte aperte sono un'informazione che possiamo ricavare anche senza di questi.

Andrebbero proibiti in qualche modo?

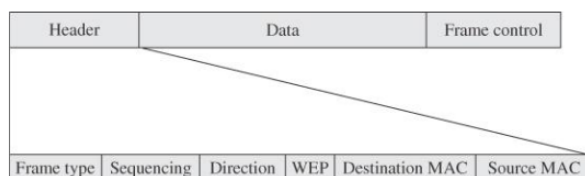
- Sarebbe troppo tardi per implementare delle regole simili dato che un qualsiasi programmatore con una conoscenza dei protocolli di rete potrebbe scriverne uno basico in poco tempo.

Questi non causano problemi di DOS e inoltre possono anche essere utilizzati per amministrazione e aumentare la sicurezza della rete.

## 12.4. Wireless Security

Le trasmissioni WiFi avvengono tramite **frame**, questi contengono tre campi principali:

- MAC header
- Payload (i dati)
- FCS (una sequenza di controllo)



Il MAC Header contiene dei campi con lunghezza fissa, fra cui:

- Frame Type: Control, Management, Data. Ad esempio per il tipo management possiamo fare 2 distinzioni:
  - *Beacon*: Ogni Access Point manda periodicamente un beacon frame per annunciare la sua presenza nella rete.
  - *Authentication*: Viene mandato quando una scheda di rete vuole interagire con un Access Point, invia appunto un authentication frame con informazioni riguardo la sua identità.

Dopo di questo vengono mandati altri frame dalla NIC per accordarsi con l'AP riguardo i parametri della connessione e la cifratura da utilizzare.

- ToDS, FromDS: Indica la direzione del frame, se va verso l'Access Point o viene da esso.
- Bits per il controllo di sequenza
- WEP (Wired Equivalency Privacy) bit: Encryption, spiegata meglio dopo
- Fino a 4 indirizzi MAC, 2 per mittente e destinatario e altri 2 opzionali per dei punti di filtraggio.

In una rete i vari Access Point si identificano tramite l'**SSID (Service Set Identifier)**, questo viene inviato all'interno dei beacon frame e viene utilizzato appunto dalle NIC per collegarsi. In una determinata area deve essere quindi univoco per ogni AP.

Le principali vulnerabilità WiFi riguardano:

- **Integrità:** Un attaccante in questo caso non fa in tempo a ricevere, modificare e reinviare un segnale prima che l'originale arrivi a destinazione, però può comunque prendere il controllo della comunicazione andando ad inserire nella comunicazione un segnale radio più forte, le antenne WiFi infatti fra due segnali preferiranno quello più potente.
- **Availability:** Oltre ai classici errori Hardware e Software, la disponibilità WiFi è minacciata anche da eventi atmosferici che causano interferenze e quindi la perdita di pacchetti. Possiamo trovare anche gli hotspot pubblici che si offrono un servizio ma spesso sono molto deboli per quanto riguarda la sicurezza degli utenti.

Vediamo le 3 fasi di accesso ad una rete WiFi e le sue debolezze:

1. L'AP invia un beacon broadcast per annunciare la sua presenza. La minaccia qui è che chiunque può intercettarlo e rispondere.
2. Il client risponde chiedendo l'autenticazione. Spesso l'autenticazione non è rigorosa e l'AP accetta qualsiasi dispositivo lo contatti.
3. Il client chiede di stabilire la connessione. Qualsiasi AP può negoziare e accettare l'associazione.

Altre vulnerabilità strutturali del WiFi si basano sul SSID, o meglio, come la sua gestione comporta dei rischi, possiamo configurarlo in due modi:

- **Open Mode:** L'AP dichiara pubblicamente il suo SSID, chiunque può quindi vedere la rete.
- **Closed Mode:** L'SSID rimane nascosto, in questo caso sono i client che devono inviare dei beacon alla ricerca dell'AP, questo però espone i client, un attaccante potrebbe essere in ascolto e capire quali parametri sono necessari per creare un falso AP e ingannare la vittima.

### Mac Spoofing

Alcuni AP potrebbero consentire direttamente la connessione ad alcuni dispositivi se questi hanno il MAC address incluso in una whitelist. Un attaccante però se conosce almeno uno di questi indirizzi potrebbe cambiare l'indirizzo sul suo dispositivo (Mac Spoofing) e connettersi bypassando i principali sistemi di sicurezza.

Come detto prima il protocollo WEP veniva inizialmente utilizzato ma in realtà è completamente inefficace per garantire privacy e autenticazione:

- **IV Collision:** Il protocollo utilizza chiavi da 64 o 128bit ma i primi 24 sono un vettore di Inizializzazione (IV) trasmesso in chiaro nel pacchetto, questo significa che i bit reali di crittografia sono solamente 40 o 104. Inoltre dato che il vettore ha solo 24bit significa che dopo circa 16milioni di pacchetti genererà obbligatoriamente dei dopppioni, questo ciclo crea appunto le IV Collisions che permettono di ricavare la chiave crittografica della rete.
- Non utilizza controlli di integrità avanzati e quindi non rileva alterazione dei pacchetti in transito.

Per risolvere questi problemi viene introdotto il **WPA (WiFi Protected Access)**:

- Introduce il **TKIP** ovvero un sistema che cambia la chiave in modo dinamico per ogni pacchetto e inoltre stabilisce una chiave per ogni sessione fra AP e dispositivo.
- Per l'autenticazione utilizza **EAP (Extensible Authentication Protocol)** in modo che l'autenticazione possa essere effettuata tramite password, token, certificati ecc...
- Utilizza un algoritmo di cifratura più potente, l'AES, anche se supporta ancora il RC4 del WEP.
- Garantisce integrità con un controllo a 64bit cifrato.
- Utilizza il **four-way handshake** per generare chiavi ad ogni sessione da utilizzare anche per la cifratura.

Nonostante questo, non rimane immune ad alcune tipologie di attacchi, tipo il **Man-In-The-Middle**:

- Sfruttando il MAC spoofing l'attaccante impersona il vero AP e invia un frame di «disassociazione» al client costringendolo a disconnettersi. Il client quindi è costretto a rinegoziare la connessione ma nel mentre l'attaccante cambia il suo MAC fingendosi il client e rubandogli la sessione con l'AP. Questo è possibile perchè a differenza del più avanzato WPA2 questi frame non hanno protezione dell'integrità.

Altri attacchi sono **Incomplete Authentication**, il client infatti deve dimostrare la sua identità all'AP ma l'AP non deve dimostrarla al client, un finto AP può quindi ingannare l'utente e sottrarre dati sensibili di autenticazione.

Infine c'è l'**Exhaustive Key Search e Chopchop**, infatti se si utilizzano password semplici basta un attacco a dizionario per trovarla. Il chopchop invece sfrutta una vulnerabilità dei controlli di integrità, l'attaccante tronca e sostituisce un byte alla volta in un pacchetto e analizza la reazione della rete a queste piccole modifiche, da queste può ricavare il pacchetto decifrato ed iniettarne di propri nella rete.

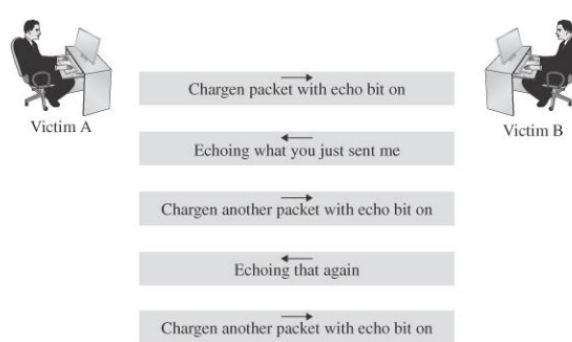
Tutti questi attacchi sono inutili contro l'ultimo protocollo WPA3.

## 12.5. Denial of Service

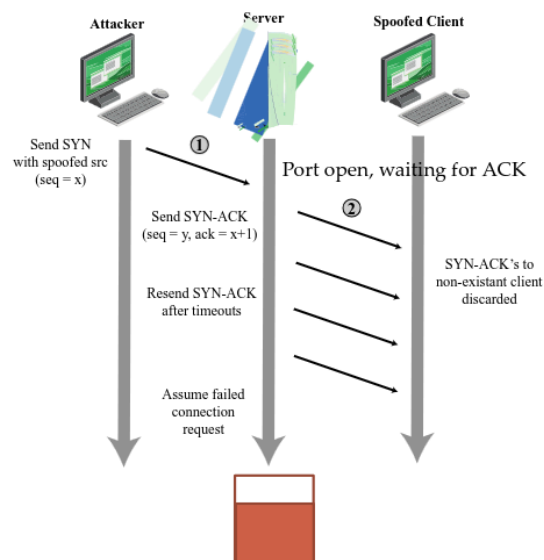
Un attacco DoS non è sempre un blocco totale ma può anche indicare un forte rallentamento del servizio, tra i principali metodi di attacco abbiamo:

- **Ping of Death / Flooding:** Se l'attaccante possiede più banda della vittima può saturare la rete inviando innumerevoli richieste di ping
- **Smurf Attack:** L'attaccante invia un pacchetto broadcast finto (con l'IP della vittima) alla rete, tutti i computer quindi rispondono contemporaneamente verso l'indirizzo della vittima congestionandola.

- **Echo-Chargen:** Instaura un loop infinito:



- **SYN-flood (TCP):** Il protocollo TCP prevede un *three-way* handshake (SYN → SYN-ACK → ACK). L'attaccante manda tantissime richieste SYN fornendo un IP falso, il server risponde con SYN-ACK ma l'ACK di conferma non arriverà mai lasciando la porta del server aperta in attesa ed esaurendo velocemente le risorse disponibili.



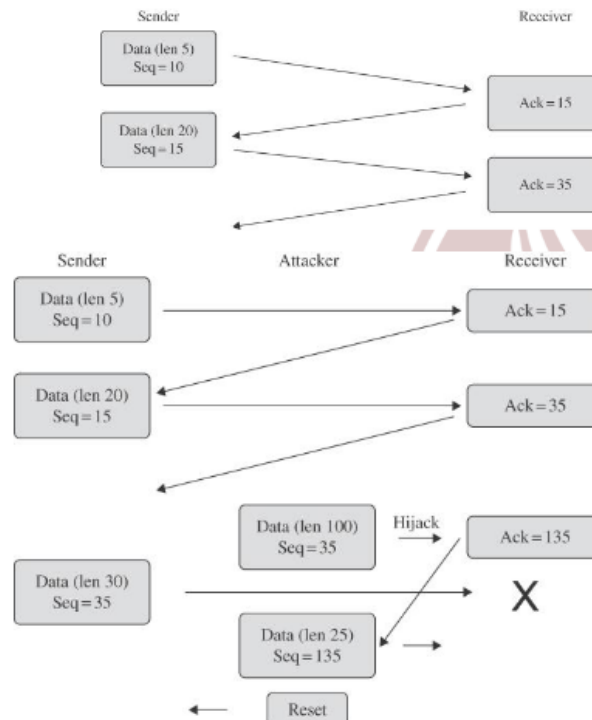
- **Teardrop (IP Fragmentation):** Spesso le comunicazioni in datagrammi li dividono in frammenti, il ricevente tramite le informazioni dei frammenti ovvero il loro inizio e quanto sono lunghi sono in grado di riassemblarli, un attaccante può inviare frammenti che si sovrappongono fra loro rendendo impossibile la ricostruzione e causando un rallentamento nel sistema.

Ci sono altri attacchi a livello Applicativo e di Routing ovvero:

- **DNS Poisoning e Spoofing:** Il poisoning avviene quando un attaccante inietta dati fittizi nella cache dei DNS, quindi se ad esempio il computer chiede google.com molto probabilmente avrà l'indirizzo nella cache ma questa é stata compromessa inserendo un IP voluto dall'attaccante. Lo spoofing invece funziona come un Man In The Middle infatti l'attaccante intercetta la risposta DNS e invia prima la sua alla vittima, mandandolo in siti trappola.

Questi attacchi sono appunto Top-Level Domain Attack e intaccano l'availability dell'utente dato che non gli permettono di risolvere indirizzi IP.

- Session Hijack: L'attaccante monitora l'handshake e invia un pacchetto con i numeri di sequenza e ACK corretti, in questo modo può inviare comandi ed escludere la vera sorgente dalla connessione.



### 12.5.1. Routing Attacks

Si possono manipolare le routing tables per rendere possibile il reinstradamento del traffico verso un router specifico, infatti dato che:

- I router si fidano l'uno dell'altro in modo implicito
- Ogni router comunica agli altri quali reti può raggiungere e in quanti hops
- Un router R può dire agli altri che è in grado di raggiungere tutte le reti nel miglior percorso possibile.

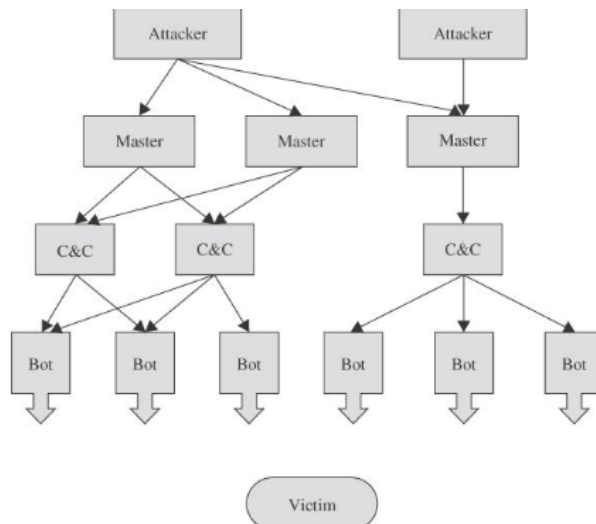
Questo rende possibili 2 attacchi:

- MiM attack: R riceve tutto il traffico e può quindi agire da MiM
- DoS attack: R viene saturato per via di tutte le richieste che riceve.

### 12.5.2. Distributed Denial of Service

Questo tipo di attacchi cambiano il «bilanciamento» tra vittima e attaccante, quest'ultimo infatti infetta altre macchine con dei virus che non per forza porta dei danni effettivi alla macchina, anzi questa deve rimanere operativa, e utilizza queste macchine per coordinare un attacco verso una vittima. Questa rete di «zombie» prende il nome di **botnet**. Questa botnet non ha un collegamento diretto fra attaccante e bot ma segue una struttura a cascata ridondante pensata per proteggere chi ha organizzato l'attacco ma anche di funzionare in caso di crash di alcune porzioni della rete. L'attaccante invia i comandi ad un primo livello di macchine chiamate Server Master questi a loro volta li inviano ai server C&C che sono quelli a stretto contatto con i bot. I bot possono ricevere i comandi in due modi:

- Push: Il comando viene inviato dal server al bot
- Pull: È il bot che periodicamente controlla se ci sono comandi in attesa.



Come ci si difende da questi attacchi?

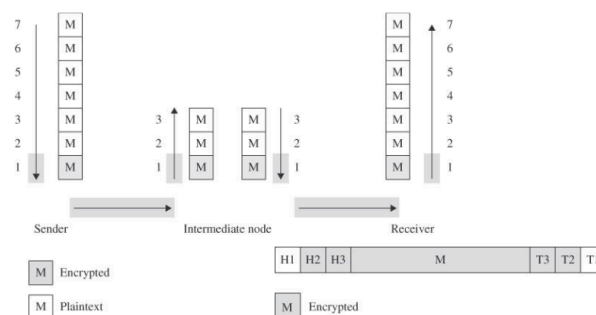
- Autonomous Mobile Protective Agents: Un programma simile a quelli usati dagli attaccanti ovvero un Trojan Horse che però ha il compito di diffondere una «controinfezione» per disabilitare il vero Trojan Horse.
- Risolvere continuamente le vulnerabilità
- Alcune tecniche come **tuning** dei server attivi, **load balancing**, **shunning** (ridurre il numero di richieste ricevibili da alcuni range di indirizzi), **blacklisting** (rifiutare la connessione da determinati indirizzi).

## 12.6. Contromisure

### 12.6.1. Network Cryptography

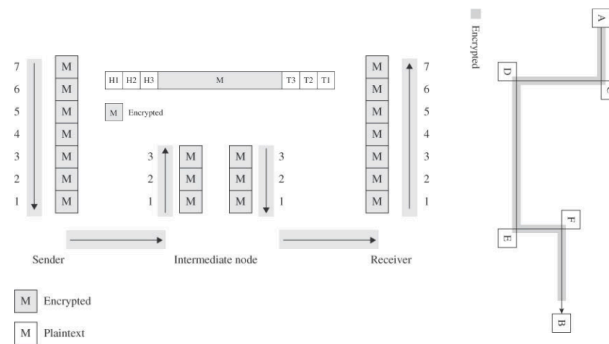
Esiste una grande differenza architeturale.

#### Link Encryption:



Il pacchetto viene cifrato all'origine poi decifrato e ricifrato a ogni singolo router intermedio, questo autentica i nodi ma espone parzialmente i dati in chiaro in quei passaggi.

#### End-to-End Encryption:



Il pacchetto rimane cifrato fino alla fine rendendo illeggibile il contenuto in tutti i router intermediari.

### SSH Encryption

Secure Shell (SSH) fornisce un collegamento autenticato e cifrato alla shell del sistema da un'altra macchina. Il protocollo SSH permette ai due terminali di stabilire:

- Algoritmo di cifratura da utilizzare
- Autenticazione (chiavi pubbliche)

### SSL and TLS Encryption

È un protocollo di cifratura utilizzato per proteggere i dati trasmessi in internet, principalmente per proteggere la confidenzialità. Può utilizzare tanti algoritmi di cifratura e certificati per l'autenticazione e lo scambio di chiavi. È implementato al quarto livello OSI e opera fra le applicazioni e il protocollo TCP/IP.

Nel tempo si è evoluto da SSL1.0 fino a SSL3.0 chiamato appunto TLS, quando si parla di HTTPS si intende infatti HTTP + TLS.

#### 12.6.2. IPsec

È uno standard progettato per gestire i dati crittografati e affrontare principalmente 3 minacce: spoofing, intercettazione e il session hijacking. Dato che opera al livello di rete (3 del modello OSI) significa che tutto quello che viaggia ai livelli superiori quindi anche i dati di controllo TCP e UDP vengono incapsulati e protetti automaticamente senza che le applicazioni si preoccupino di nulla.

Vediamo i concetti principali della sua architettura.

**Security Association** - Sono tutti i parametri che i due terminali devono scambiarsi e sono necessari a stabilire una connessione sicura:

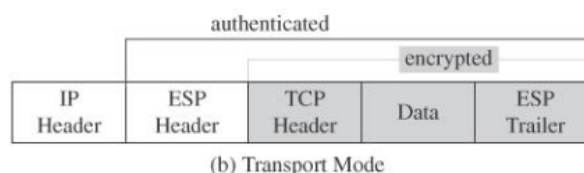
- Algoritmo di cifratura e la relativa chiave crittografica
- I parametri di crittografia come il vettore di inizializzazione
- Protocollo e chiave per l'autenticazione
- La durata dell'associazione ovvero il periodo dopo il quale le chiavi devono essere rinnovate
- Livello di sensibilità dei dati

**Gestione delle chiavi** - Il punto critico della crittografia è quello di scambiarsi le chiavi in modo sicuro, IPsec risolve il problema utilizzando l'architettura **ISAKMP (Internet Security Association Key Management Protocol)** implementata operativamente tramite il protocollo **IKE (ISAKMP Key Exchange)**, questo processo avviene in più fasi:

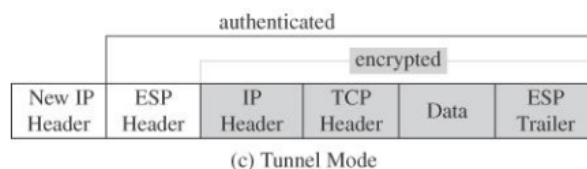
1. Si utilizza lo schema di Diffie-Hellman per generare matematicamente un «segreto condiviso» tra le due parti senza mai trasmetterlo sulla rete.
2. Utilizzando questo segreto le due parti si scambiano le proprie identità e i rispettivi certificati digitali per autenticarsi a vicenda.
3. Derivano la vera chiave crittografica e instaurano la Security Association definitiva.

**Modalità di Funzionamento** - IPsec deve alterare il pacchetto originale per proteggerlo, può farlo in due modi a seconda della modalità di funzionamento:

- **Transport Mode:** È la normale modalità operativa tra i due host, solo il payload viene blindato e inserito all'interno del campo crittografato ESP. L'IP Header originale rimane in chiaro permettendo ai router intermedi di leggere l'indirizzo del destinatario e instradare quindi il pacchetto.



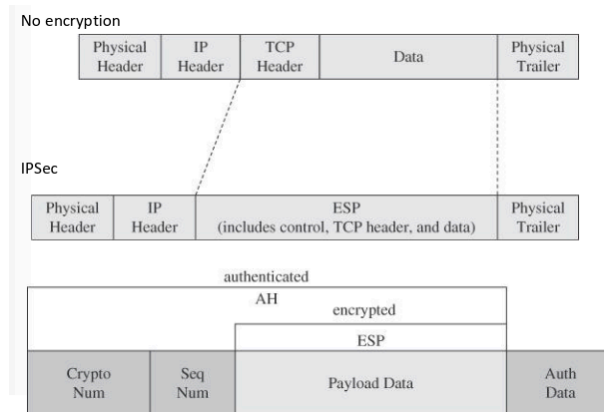
- **Tunnel Mode:** Offre una sicurezza più elevata per mascherare anche la topologia della rete nascondendo infatti l'indirizzo del vero destinatario. In questa modalità l'intero pacchetto originale viene cifrato e infilato nel campo ESP. IPsec aggiunge però un nuovo IP Header in chiaro all'esterno che punta ad un dispositivo remoto che si occuperà di ricevere il pacchetto, rimuovere la crittografia e smistare il pacchetto originale verso la vera destinazione interna.



**Header IPSec** - I campi aggiunti dal protocollo difendono dagli attacchi specificati prima:

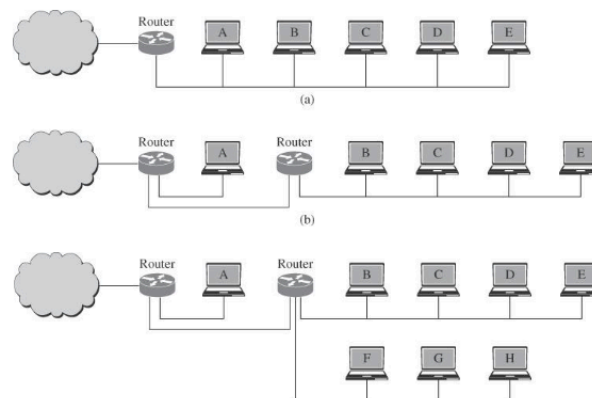
- **Crypto Num:** Contiene le informazioni sulla suite crittografica concordata dai terminali
- **Seq Num:** L'inserimento di numeri di sequenza è la contromisura diretta contro i Replay Attack
- **Auth Data:** Un blocco dedicato alla gestione dell'autenticazione assicurando che i dati provengano effettivamente dal mittente dichiarato e non siano stati alterati.





## 12.7. Firewalls

Il primo concetto principale è quello della segmentazione della rete in sottoreti, ognuna gestita da un router, in questo modo se una viene compromessa non significa che anche le altre debbano cadere automaticamente.



- Nell'immagine **a** tutti gli host sono visibili e una vulnerabilità può propagarsi
- Nella **b** le reti sono separate, rimangono quindi esposte a vulnerabilità ma queste non si espandono fra le reti.

Questo meccanismo però controlla soltanto il movimento dei dati e non risolve il problema del controllo degli accessi. Per questo è necessario un **firewall**.

Un firewall è un dispositivo che esegue un codice per filtrare tutto il traffico tra una rete interna e una esterna, un firewall ben progettato deve rispettare i tre principi del **Reference Monitor**:

- **Always Invoked:** Deve essere l'unico punto di passaggio obbligato per il traffico
- **Tamperproof:** A prova di manomissione, solitamente infatti è implementato su un computer dedicato ed isolato in modo da essere altamente immune alle modifiche esterne.
- **Small and Simple:** Deve far girare un sistema operativo ridotto con servizi minimi, meno codice c'è e più sarà facile da analizzare per trovare falle offrendo anche meno strumenti di attacco ad un attaccante.

Il firewall decide cosa passa e cosa no basandosi su un set di regole, per creare queste regole gli amministratori si basano su due filosofie:

- **Default Permit:** Tutto quello che non è esplicitamente vietato è permesso.

- **Default Deny:** Tutto ciò che non è esplicitamente permesso è vietato.

*Esempio di regole:*

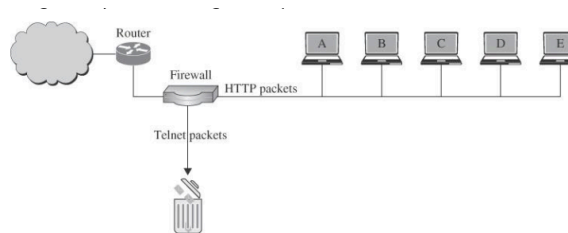
| Rule | Type | Source Address | Destination Address | Destination Port | Action |
|------|------|----------------|---------------------|------------------|--------|
| 1    | TCP  | *              | 192.168.1.*         | 25               | Permit |
| 2    | UDP  | *              | 192.168.1.*         | 69               | Permit |
| 3    | TCP  | 192.168.1.*    | *                   | 80               | Permit |
| 4    | TCP  | *              | 192.168.1.18        | 80               | Permit |
| 5    | TCP  | *              | 192.168.1.*         | *                | Deny   |
| 6    | UDP  | *              | 192.168.1.*         | *                | Deny   |

Il firewall legge le regole dall'alto verso il basso, nell'esempio quindi prima vengono permesse le connessioni sulla porta 80 o 25 verso certi IP per poi avere una regola che blocca tutte le connessioni.

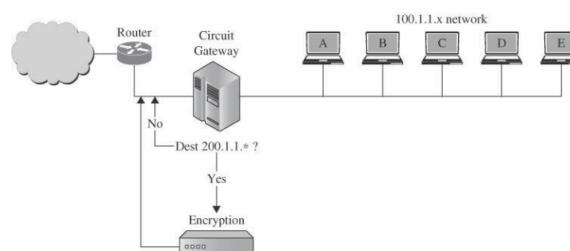
### 12.7.1. Tipologie di Firewall

Possiamo classificare i firewall in base al livello di profondità con cui ispezionano i pacchetti:

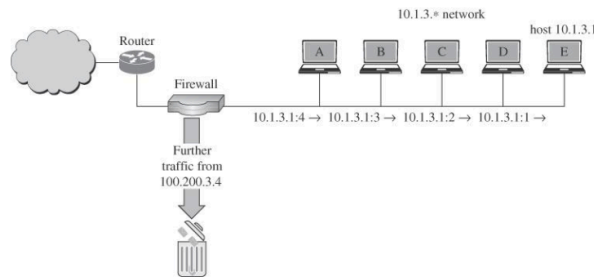
- **Packet / URL Filtering:** Il più semplice e veloce, analizza un pacchetto per volta guardando soltanto gli indirizzi IP e le porte senza memoria del passato.



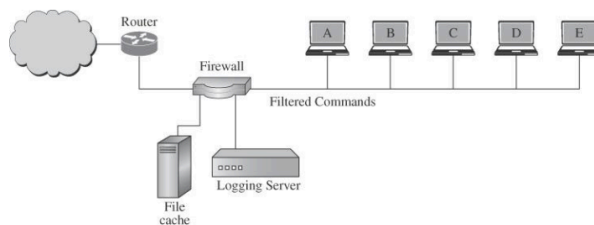
- **Circuit Level Gateway:** Opera al livello 5 OSI, il firewall verifica e autorizza la connessione logica nel momento in cui viene creata, ma una volta stabilita lascia passare i dati successivi senza ispezionarli. Viene utilizzato per collegare due reti di cui ci si fida, ad esempio nelle VPN.



- **Stateful Inspection:** Mantiene in memoria lo «stato» della connessione. Correla le informazioni dei pacchetti precedenti con quelli successivi, riuscendo a bloccare complessi schemi come una scansione di porte anche estesa nel tempo.



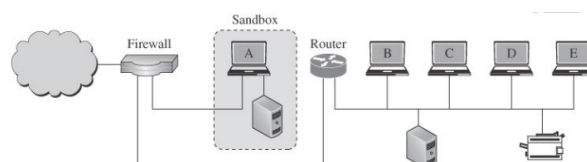
- **Deep Packet Inspection (DPI):** Oltre agli indirizzi IP, ispeziona attivamente il contenuto dei pacchetti, può anche alterare i dati in uscita ad esempio anonimizzando il traffico.
- **Application Proxy Gateway (Bastion Host):** Lavora al livello più alto ovvero quello di Applicazione. Si comporta come un intermediario infatti dall'esterno sembra lui la destinazione finale, riceve i dati, li analizza in base a come si comporta l'app e poi crea una nuova connessione verso la macchina interna. Offre molta sicurezza ma è lento.



- **Guard:** Un proxy molto sofisticato, permette complesse regole programmabili. Un guard non si limita ad osservare il traffico ma interpreta a fondo i pacchetti e ne genera di nuovi identici o modificati per garantire maggiore sicurezza. In base allo storico delle richieste riesce a capire se autorizzare o meno determinate azioni, la sua caratteristica principale infatti è che è «limitato soltanto da ciò che è computabile», questo significa che teoricamente è in grado di applicare qualsiasi regola noi riusciamo a programmare. Alcuni esempi:
  - Controllo delle email: Limitare il numero di email che riceviamo da un determinato utente
  - Throttling: Limitare attivamente la velocità del traffico a disposizione di utente
  - Paywall: Gestire l'accesso a determinati servizi solo dopo un pagamento
  - Scansione antivirus: Può ispezionare i file che gli utenti stanno scaricando e passarli attraverso un antivirus direttamente sul firewall e bloccandoli prima ancora che arrivino sul PC del destinatario.

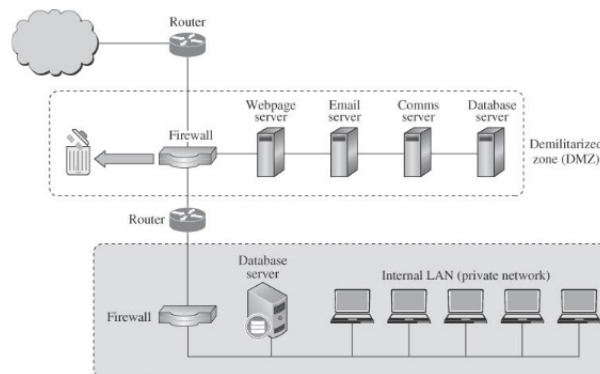
Tutta questa sua flessibilità porta ad un'altissima complessità e per questo motivo risulta difficile implementarne uno in modo accurato.

- **Sandbox:** Se il firewall nota un codice in entrata sospetto, lo esegue in ambiente isolato limitandogli l'accesso alle risorse. Se il codice non si comporta in modo strano allora l'amministratore può approvare l'ingresso.



Il firewall può essere posizionato in diversi punti in una rete, la topologia più recente è quella della **DMZ (Demilitarized Zone)**, qui si usano due firewall:

- Un primo firewall esterno che fa passare il traffico pubblico verso la DMZ ovvero un'area cuscinetto dove risiedono i server esposti al pubblico come quello Web o Email.
- Un secondo firewall interno più rigido che separa la DMZ dalla LAN privata proteggendo risorse come i Server Database.



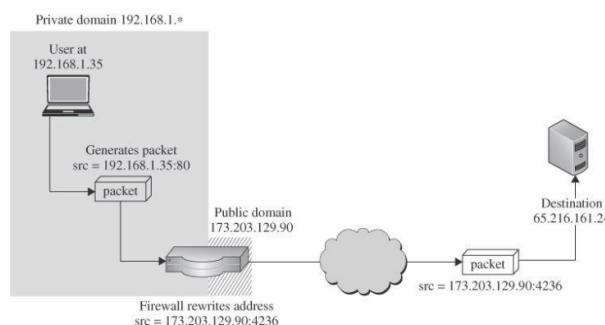
### 12.7.2. Limitazioni dei Firewall

Ovviamente questi non risolvono tutti i problemi delle reti:

- Riescono a proteggere soltanto l'ingresso principale della rete, se un dipendente con dispositivi infetti si collega via wifi interno, ha bypassato tutto il firewall
- Una volta che i dati superano il firewall in modo legittimo per uscire non sono più protetti
- Essendo lo scudo principale ed anche visibile, sono uno dei principali bersagli degli attaccanti
- Richiedono una manutenzione costante da parte degli amministratori, anche una sola regola sbagliata rende inutile tutto il firewall. Per questi motivi si consiglia la **Defense in Depth** ovvero avere più strati di sicurezza interni per non basare tutto sulla sola resistenza del firewall.

### 12.7.3. Network Address Translation (NAT)

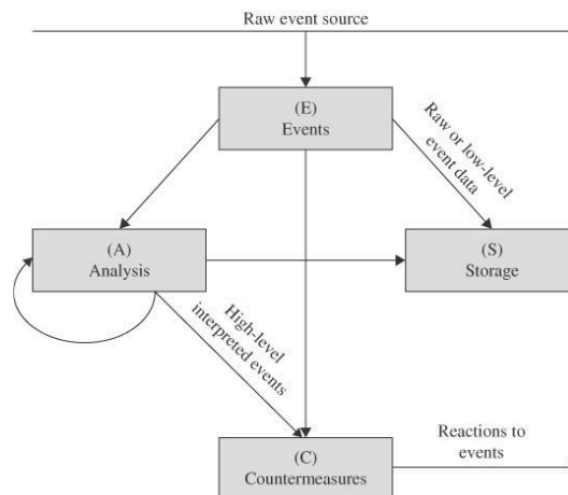
Viene spesso integrato nel firewall, serve a riscrivere gli indirizzi IP interni dei computer sostituendoli con l'IP pubblico del firewall verso l'esterno. Questo nasconde la vera topologia della rete privata impedendo agli attaccanti di conoscere, mappare e sfruttare direttamente gli IP interni. Utilizzato anche per risolvere i problemi dei «pochi» indirizzi.



## 12.8. Intrusion Detection and Prevention

Un IDS è un dispositivo dedicato esclusivamente a monitorare le attività per identificare eventi sospetti o malevoli, le sue funzioni includono:

- Monitoraggio degli utenti
- Auditing delle configurazioni per scovare vulnerabilità
- Controllo integrità dei file
- Analisi statistica per scovare comportamenti anomali e, se necessario, l'installazione di trappole per registrare informazioni sugli intrusi.



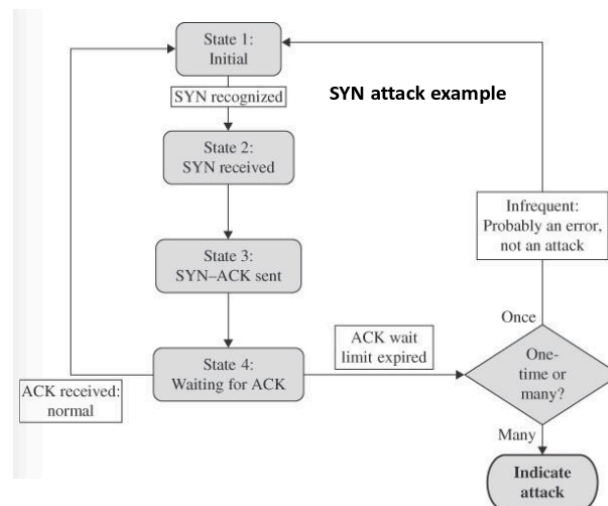
I suoi obiettivi principali sono l'essere veloce, accurato, semplice e operare in tempo reale nascondendo la propria presenza agli attaccanti. Il problema più grande che un IDS deve gestire è la sua **Accuratezza (Sensitivity)** misurata tramite due errori:

- **False Positive:** L'IDS lancia un allarme per un evento che non è un attacco, troppi falsi allarmi fanno sì che gli amministratori perdano fiducia nel sistema portandoli a ignorare le vere emergenze
- **False Negative:** L'IDS non lancia l'allarme e un vero attacco passa inosservato. L'amministratore deve configurare attentamente la sensibilità dell'IDS per bilanciare questi due problemi.

Ci sono due principali metodologie di rilevamento.

- **Signature Based:** L'IDS esegue un pattern-matching cercando corrispondenze con «firme» (sequenze specifiche di dati) di attacchi noti. Ad esempio può accorgersi di un Port Scan se nota tanti pacchetti TCP SYN diretti a porte diverse dello stesso IP in brevissimo tempo. Un problema di questi IDS è che se la firma non esiste nel database, l'IDS è cieco e inoltre l'attaccante può ingannare l'IDS inserendo pacchetti spazzatura per sfasare la firma.

Questo IDS non è adatto a intercettare attacchi di frammentazione IP come il teardrop perché l'IDS dovrebbe riassemblare e tenere in memoria quasi tutto il traffico di rete prima di capire che i frammenti sono malevoli. Neanche il SYN-flood è facile da rilevare poiché l'handshake SYN-ACK fa parte del traffico TCP normale. L'unico modo che ha di superare questi limiti è tracciando lo stato delle connessioni ma mantenere lo stato di migliaia di connessioni simultanee diventa molto costoso e complesso.



- **Heuristic / Anomaly-Based:** In questo caso invece di cercare firme note, l'IDS costruisce un modello del «comportamento normale» e fa scattare gli allarmi ogni volta che avviene un'eccezione a questo modello. Per farlo impara nel tempo tramite tecniche di Machine Learning e Intelligenza Artificiale. Come funziona?
  - Inference Engine: L'euristica lavora col concetto di «sporczia», il sistema analizza l'andamento da uno stato *Clean* a *Dirty*, ad esempio l'host A che scansiona l'host B è leggermente sospetto ma accettabile ma se poi l'host A richiede i file pubblici e tenta di scaricarli tutti allora diventa sospetto. La somma di più azioni fa superare la soglia e scattare l'allarme
  - Approcci: L'inferenza può essere *State-based* (osserva il cambiamento di stato del sistema) o *Model-based* (classifica tutto il traffico o i log in buono, sospetto o sconosciuto in base ai modelli)
  - Pro e Contro: Il grande vantaggio è che l'IDS impara e si adatta col tempo, lo svantaggio è che le sue decisioni sono buone solo quanto la quantità e la qualità delle informazioni normali su cui si è allenato in precedenza.

Alcuni esempi di rilevamento:

- Land Attack: Un attaccante invia dei pacchetti SYN dove l'indirizzo del mittente è lo stesso del destinatario. Per rilevarlo controlla la variabile *Land*, se vale 1 significa che l'indirizzo IP mittente e destinatario sono identici e svela quindi il pacchetto malevolo.
- Teardrop: Guarda la feature *Wrong Fragment* che conta la somma dei pacchetti con checksum errati.
- Smurf: Rileva tantissimi pacchetti di «ICMP echo reply» in entrata verso la vittima senza che questa abbia mai generato i relativi pacchetti di richiesta.
- Ipsweep: È un attacco che serve a determinare quali hosts sono in ascolto sulla rete mandando molti pacchetti di ping. Per rilevarlo controlla la frequenza dei pacchetti ICMP in base alla durata temporale, aggregando le risposte tramite la destinazione.

Da notare che gli IDS lavorano a livelli di rete diversi, un IDS configurato per analizzare gli header IP (livello 3) difficilmente riuscirà ad analizzare i payload a livello applicativo (7) a meno che non esegua un costoso riassettaggio dei dati.

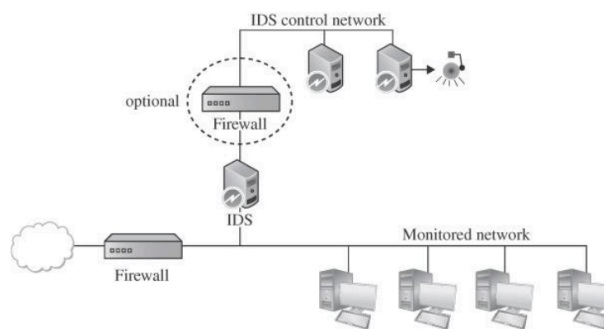
### 12.8.1. Posizionamento e Tipologie di Implementazione

Un IDS può essere implementato in modi e posizioni diverse per coprire diverse vulnerabilità.

- Front-end IDS: Si posiziona prima o assieme al firewall esterno, vede tutto il traffico globale in entrata ma proprio per questo è visibile e diventa un bersaglio primario.
- Internal IDS: Controlla la rete dall'interno, fondamentale per rilevare attacchi causati da minacce interne o macchine già compromesse.

Per quanto riguarda l'implementazione invece:

- HIDS (Host-based IDS): È installato su un singolo computer, raccoglie i log di quel sistema operativo, controlla le richieste di accesso e gli orari. Essendo un processo software che gira sulla macchina, se la macchina viene bucata l'HIDS può essere individuato e spento dall'attaccante
- NIDS (Network-based IDS): Intercetta i dati dell'intera rete, il suo enorme vantaggio è che può funzionare in **stealth mode**. La scheda di rete usata dall'IDS per ascoltare il traffico non ha un indirizzo IP configurato, non può quindi ricevere pacchetti diretti né essere contattata. Per lanciare gli allarmi il NIDS usa una seconda scheda di rete collegata a una rete di controllo gestionale completamente isolata.



### 12.9. Intrusion Prevention Systems (IPS) e Tipi di Risposta

Se un IDS ha la capacità di agire e bloccare attivamente i danni di cui si accorge, prende il nome di IPS, questo può rispondere in 4 modi:

- Monitor: Non interviene attivamente ma aumenta l'acquisizione dei dati per studiare l'attaccante senza farsi notare.
- Signal: Manda allerte ad altri componenti protettivi della rete.
- Call a Human: Richiede l'intervento di un amministratore.
- Protect: Modifica in tempo reale la rete per fermare la minaccia. L'IPS può tagliare l'accesso ad un file, scartare il traffico da un certo IP, reindirizzare il traffico verso una sandbox o honeypot, bilanciare il traffico o, in casi estremi, disconnettere intere porzioni di rete. Queste azioni potrebbero far capire all'attaccante di essere stato scoperto portandolo a fermare l'attacco o cambiare strategia.